

Методика автоматической трансляции программного кода между языками с сохранением семантики безопасности.

1. Введение

Одной из основных проблем, связанных с программным кодом, является его безопасность. Один из подходов работы с безопасностью кода - это обработка свойств безопасности кода. Мы можем собрать свойства безопасности кода некоторой программы, отследить как они изменяются во время выполнения этой программы и выделить множество свойств, которые сохраняются - это будет отпечаток безопасности этой программы. Но мы можем не только отслеживать свойство безопасности кода во время выполнения некоторой программы, мы можем отслеживать эволюцию свойств безопасности во время трансформации исходного кода и предсказывать, как они изменятся (и соответственно, как изменится безопасность трансформированного кода). Одной из таких важных и сложных трансформаций является преобразование (трансляция) исходного кода на другой язык программирования.

При разработке программного обеспечения достаточно часто возникает задача о переносе программного кода между языками (который мы называем трансляцией кода, см. далее [глава 1.8]). При этом, если о соответствии функциональным требованиям у перенесенного (транслированного) кода помнят почти всегда (обычно проблемы возникают с соответствием поведения, связанного с тонкостями того или иного языка/платформы), то вот о безопасности кода при переносе обычно часто даже и не думают. Особенно эта проблема актуальна, если перенос (трансляция) кода - это не одноразовое явление, а периодический или массовый процесс.

Процесс периодического/массового переноса (трансляции) кода бывает нужен в следующих случаях:

- 1) Некоторая компания разрабатывает и выпускает свой продукт (библиотеку) на некотором определенном языке под некоторую платформу (например, на C# под платформу .NET). Но при этом есть желание выйти с данным продуктом (библиотекой) на другие языки и платформы: например, на Java, на Python, на C++ и т.д. Примером такой компании может быть компания Aspose, которая разрабатывает библиотеку компонентов для работы с документами разнообразных форматов (word, excel, pdf и т.д.).
- 2) Есть некоторая функциональность (библиотека), реализованная на некотором языке под некоторую платформу (например, некоторая open source функциональность), у которой периодически выходят новые версии. Мы разрабатываем некоторое ПО на другом языке и под другую платформу, и мы нуждаемся в версии этой функциональности под язык и платформу, для которой разрабатывается ПО.
- 3) У нас есть большое количество ПО, созданного на одном языке и под одну платформу, а нам (например, по причине импортозамещения) необходимо перевести весь этот объем существующего ПО под другой язык и другую платформу.

Сделать перенос (трансляцию) кода можно двумя способами:

- Вручную, но для этого нужно достаточно много разработчиков, которые бы хорошо знали как исходные язык программирования, так и язык программирования, на который будет транслироваться исходный код; при этом это будет не самый быстрый процесс, т.к. при ручном переносе (трансляции) кода, транслируемый код фактически создается заново, особенно с учетом разницы между языками (см. далее [глава 4.3]).
- С помощью средств автоматизации: это либо специально созданные утилиты, либо LLM-агенты. Но и в этом случае все достаточно не просто. Специально созданные утилиты обычно нужно конфигурировать под каждую переносимую программу; к тому же далеко не весь код может быть вообще перенесен в автоматическом режиме (как, например, переносить определение шаблонов с языка C++ на любой другой язык, где подобной концепции вообще нет) с одного языка на другой (см. далее [глава 4.4]). А LLM-агенты периодически “врут” (особенно это проявляется на большом объеме кода).

Очевидно, мы хотим, чтобы перенос (трансляция) кода с одного языка на другой осуществлялся автоматически, т.к. несмотря на все проблемы, которые возникают при автоматическом переносе кода (см. далее [глава 4.4]), обычно все-таки это более быстрый процесс, чем перенос (трансляция) кода вручную.

И вот здесь, на стыке требований к безопасности кода и нашим желанием уметь автоматически осуществлять перенос (трансляцию) кода с одного языка на другой возникают следующие вопросы. Если исходный код обладает всеми необходимыми для нас свойствами безопасности, то что можно сказать в этом случае о коде после переноса (трансляции)? Будет ли он обладать всеми теми же свойствами безопасности, что и исходный код, или же какими-то из свойств придется “пожертвовать”? Будет ли осуществляться сохранение свойств безопасности автоматически или же для сохранения некоторых из них придется делать дополнительные действия?

1.1 Актуальность. Почему проблема важна.

Как уже было сказано выше, периодически возникают задачи о переносе (трансляции) кода с одного языка на другой, как вручную, так и с помощью средств автоматизации. При этом важную роль играет не только безопасность полученного кода, а также то, как она связана с безопасностью исходного кода.

1.2 Объект исследования.

Объектом исследования в данной работе является процесс автоматического переноса (трансляции) программного кода между различными языками программирования. Под данным процессом понимается преобразование кода, написанного на одном (исходном) языке программирования, в эквивалентный код на другом (целевом) языке программирования с сохранением их функционального поведения. Такой перенос может осуществляться как между языками высокого уровня, например, между C# и Python, так и в направлении понижения уровня абстракции, например, при генерации низкоуровневого кода с использованием инфраструктур вроде

LLVM (но в данной работе рассматривается перенос только между языками высокого уровня).

Особое внимание в рамках исследования уделяется вопросам корректности и надежности трансляции, поскольку даже при формальном сохранении синтаксической структуры программы могут возникать изменения в ее семантике, влияющие на поведение при выполнении. Это связано с различиями в моделях памяти, системах типов, механизмах управления ресурсами и особенностях выполнения программ в разных языках. В связи с этим процесс трансляции рассматривается не только как техническая задача преобразования кода, но и как сложная формальная проблема, требующая учёта семантической эквивалентности, а также обеспечения свойств безопасности и устойчивости программ в целевой среде выполнения.

1.3 Предмет исследования.

Предметом исследования являются методы выявления, классификации и сохранения свойств безопасности при автоматизированном переносе программного кода между языками программирования. В рамках данной работы под свойствами безопасности понимаются характеристики кода, которые мы можем получить исходя из анализа этого кода на основе списка Common Weakness Enumeration (CWE) [101], а также на основе спецификации языка. Примером таких свойств безопасности могут служить следующие: “проверка входных аргументов в функции/методе”, “типобезопасность”, “сохранение инкапсуляции” и другие. Существенное внимание уделяется разработке подходов, позволяющих формально выявлять эти свойства на уровне исходного кода, а также определять, какие из них должны быть сохранены в процессе трансляции и что для этого необходимо сделать.

1.4 Задача исследования

Задачей исследования является следующее:

- разработка подхода (методики) выявления и классификации свойств безопасности;
- определение требований (условий) при которых мы можем считать, что семантика безопасности сохраняется при автоматизированной трансляции на основе классификации свойств безопасности;
- разработка метода симплификации исходного кода с целью упрощения структуры автоматизированного транслятора;
- создание симплификатора кода для языка C# и транслятора кода с языка C# на язык Python;
- проведение эксперимента для проверки разработанной методики.

1.5 Цель.

Разработка методики автоматического переноса (трансляции) кода между языками, обеспечивающего контролируемое сохранение семантики безопасности.

1.6 Гипотеза

При фиксированной паре языков (в нашем случае, это C# - Python) и при использовании анализа Security Fingerprint, классификации L1–L4 (с применением компенсирующих мер для свойств L2) и симплификации риск потери значимых свойств безопасности (инвариантов безопасности) снижается по сравнению с переносом без такой методики.

1.7 Новизна

Задача трансляции кода с одного языка на другой (несмотря на то, что в некоторых компаниях является коммерчески применимой и успешной) сама по себе содержит достаточно новизны, т.к. результаты ее решения широко не распространены. Но в данной работе к этой задаче были применены модель работы со свойствами безопасности (инвариантов безопасности) и Security Fingerprint, была предложена классификация свойств безопасности L1–L4 относительно трансляции кода (с применением компенсирующих мер для свойств L2), выведены критерии успешности трансляции (относительно сохранения семантики безопасности) и был использован подход с симплификацией исходного кода для упрощения кода транслятора.

1.8 Теоретическая и практическая значимость

В данной работе предлагается подход (методика) для централизованного управления безопасностью кода в тех случаях, когда один и тот же продукт (например, библиотека) выпускается на разных языках программирования. Для этого при трансляции мы обеспечиваем сохранение семантики безопасности: строим отпечаток безопасности исходного кода и отслеживаем его эволюцию во время транслирования. Также, для упрощения создания транслятора, мы используем подход с симплификацией исходного кода: мы изменяем код так, чтобы AST исходного кода по максимуму походила на AST кода, полученного в результате – при таком подходе процесс трансляции кода сильно упрощается. Такая централизация управления безопасностью кода уменьшает шансы того, что версии одного и того же продукта на разных языках будут иметь разную семантику безопасности (разные потенциальные уязвимости).

Теоретическая значимость работы заключается в формализации понятия семантики безопасности при автоматической трансляции программного кода и разработке модели “отпечатка безопасности” как набора инвариантов, подлежащих сохранению при переносе между языками.

В рамках работы вводятся критерии сохранения защитных свойств, классификация инвариантов по уровням сохранности, а также рассматривается понятие допустимого ослабления семантики безопасности в случаях, когда особенности целевого языка не позволяют полностью воспроизвести гарантии исходного языка.

Полученные результаты расширяют представления о трансформации программ с сохранением нефункциональных требований и могут быть использованы при разработке формальных подходов к оценке безопасности при миграции программных систем. Практическая значимость работы заключается в разработке и реализации прототипа инструмента автоматической трансляции программного кода (из языка C# в язык Python), демонстрирующего возможность сохранения защитных свойств при переносе между языками.

Предложенная методика позволяет: снизить вероятность появления новых уязвимостей при миграции кода; обеспечить контролируемость процесса переноса за счет фиксации и сравнения «отпечатков безопасности»; уменьшить трудоемкость поддержки мультиплатформенных решений; централизовать управление безопасностью исходного кода.

Результаты работы могут быть применены в организациях, осуществляющих миграцию программных систем, разработку библиотек для нескольких языков программирования или переход на новые технологические платформы.

1.9 Положения на защиту

Во время трансляции кода между какой-либо парой языков, мы можем сформировать Security Fingerprint для исходного кода, провести классификацию свойств безопасности из сформированного Security Fingerprint, применить компенсирующие меры для свойств безопасности класса L2 - это снизит потерю значимых свойств безопасности и позволит предсказать Security Fingerprint кода после трансляции.

Симплификатор - это программа, которые эти преобразования кода на исходном языке реализует. В результате ее работы получается код также на исходном языке, но более простой для процедуры трансляции. В результате применения симплификатора уменьшается количество случаев, которые нужно обрабатывать в трансляторе, т.е. уменьшается цикломатическая сложность кода транслятора. Как итог, уменьшается вероятность ошибки/дефекта в трансляторе.

Связка симплификации исходного кода и автоматической/полуавтоматической трансляции позволяет (во многих случаях) достаточно быстро транслировать код между двумя языками с предсказуемыми свойствами безопасности транслированного кода.

1.10 Что такое трансляция кода.

Трансляция кода - это перевод кода с исходного языка на некоторый другой (например с C# на Python) один к одному (ну или почти один к одному) - с сохранением структуры исходного кода. При этом должны быть сохранены все функциональные требования и максимально возможное количество нефункциональных требований (см. далее [глава 1.9]). Может осуществляться как вручную, так и с использованием средств автоматизации. Полностью сохранить структуру исходного кода не всегда возможно - это зависит как от сущностей, которые использовались в исходном коде, так и от

возможностей языка, на который происходит трансляция кода. Например, если мы будем транслировать код с ООП языка на язык, в котором нет поддержки ООП, то очевидно, что структура после трансляции несколько изменится; однако, в транслированном коде все-равно так или иначе будет видна структура исходного кода. Если же перевод кода осуществляется таким образом, что его структура не сохраняется, то это уже не трансляция - это создание нового кода (новой кодовой базы) на основе существующего кода.

Автоматическая трансляция кода - это трансляция кода с использованием средств автоматизации: либо утилиты для автоматической трансляции (портирования, транспилиции) кода, либо LLM агентов (более подробно см. далее [глава 4.2]).

1.11 Трансляция кода с точки зрения функциональных и нефункциональных требований.

С точки зрения функциональных требований все просто: трансляция кода должна возвращать такой код, который с точки зрения функциональных требований полностью эквивалентен исходному коду. Так как в противном случае код, полученный в результате трансляции, ведет себя иначе и/или результат его выполнения отличается от результата выполнения исходного кода - очевидно, что такой транслированный код никому не будет нужен.

Точно также, поведение транслируемого кода должно быть эквивалентно поведению исходного кода. Однако с этим есть ряд тонкостей/сложностей: в разных языках поведение похожих конструкций может быть разным. Это означает, что мы должны проверять на эквивалентность исходный код и код, полученный в результате трансляции, не только с точки зрения функциональных требований, но и с точки зрения спецификаций языков (на котором написан исходный код и на который он транслируется). Это же касается и случаев, когда мы хотим создать аналог некоторой конструкции, которая есть в исходном языке, но нет в языке, на который происходит трансляция.

Рассмотрим несколько примеров, демонстрирующих, что учет поведения при трансляции должен выполняться максимально аккуратно.

Пример 1:

Пусть у нас есть следующее определение функции на C# (здесь приведен фрагмент кода):

```
void fun1(T1 arg1, T2 arg2, T3 arg3, T4 arg4) {...}
```

Порядок вычисления аргументов *arg1*, *arg2*, *arg3* и *arg4* перед вызовом функции *fun1* четко определен спецификацией языка C#: они вычисляются слева направо.

Предположим, что мы эту функцию транслировали на язык C++. В нем порядок вычисления аргументов *arg1*, *arg2*, *arg3* и *arg4* перед вызовом функции *fun1* не определен - каждый компилятор может вычислять их в своем порядке. Это является потенциальной проблемой, если вычисление значений одних аргументов прямо или косвенно могут влиять на вычисление значений других аргументов. В этом случае

(если не предпринять специальных шагов) мы можем получить разное поведение в исходном коде на языке C# и в коде, полученном в результате трансляции на язык C++.

Пример 2:

В C-подобных языках есть тернарный оператор, который выглядит следующим образом (здесь приведен фрагмент кода):

```
cond ? expr1 : expr2
```

В языке Golang аналога тернарному оператору нет. Однако, некоторые разработчики реализуют его аналог следующим образом (здесь приведен фрагмент кода):

```
func TernOp[T any](cond bool, expr1 T, expr2 T) T {  
    if cond { return expr1 }  
    return expr2  
}
```

После чего его использование выглядит так (здесь приведен фрагмент кода):

```
value := TernOp(a == 1, 13, 666)
```

На первый взгляд может показаться, что поведение аналога совпадает с поведением тернарного оператора, однако это не так. Например, значение следующего выражения с тернарным оператором (здесь приведен фрагмент кода):

```
int x = 0;  
int value = x != 0 ? 13 / x : 1;
```

когда значение переменной *x* равно 0 будет определено (вычислено).

А при вычислении значения аналогичного выражения в языке Golang (здесь приведен фрагмент кода):

```
x := 0  
value := TernOp(x != 0, 13 / x, 1)
```

когда значение переменной *x* равно 0 мы получим панику из-за деления на 0.

Основное различие между тернарным оператором и его попыткой реализацией на языке Golang в том, что при вычислении значения тернарного оператора выражения *expr1* и *expr2* вычисляются лениво, в зависимости от значения выражения *cond*; тогда как в приведенной реализации на языке Golang, выражения *expr1* и *expr2* вычисляются всегда, вне зависимости от значения выражения *cond*.

С нефункциональными требованиями все сильно сложнее - давайте рассмотрим некоторые из них:

1) Одно из самых важных нефункциональных требований - это безопасность. К ней относятся как требования по защите данных, шифрованию, контролю доступа и требования к безопасности кода. Требования по защите данных, шифрованию, контролю доступа реализуются с помощью библиотечного кода (с помощью АПИ платформы); поэтому можно считать, что с этим обычно проблем не бывает.

А вот безопасностью исходного кода обычно не занимаются: многие разработчики считают, что если трансляция кода происходит один к одному (ну или

почти один к одному), то безопасность транслированного кода не ухудшается. Основана эта уверенность на том, что наиболее очевидные механизмы безопасности кода являются составляющими частями самого кода: проверки входных аргументов и пользовательского ввода, очистка неиспользуемых ресурсов и т.д. Но многие механизмы не такие очевидные и связаны больше с возможностями языков программирования и платформ (сред выполнения); и вот они часто при трансляции кода не сохраняются, если не предпринимать для этого специальных действий. Так, например, в языке Python отсутствуют модификаторы доступа и, как следствие, инкапсуляция; таким образом, при трансляции кода с языка C# на Python теряются такие свойства безопасности, как инкапсуляция и контроль доступа к членам [глава 3.5].

2) Другое, не менее важное нефункциональное требование - это производительность. И вот с производительностью при транслировании кода все достаточно неоднозначно: в одних случаях результат транслирования кода будет удовлетворять требованиям по производительности, в других нет и потребуются ручная кастомизация кода (см. далее [глава 4.7]), а в некоторых случаях результат трансляции кода вообще не сможет удовлетворить требованиям по производительности к исходному коду из-за ограничений языка/платформы. В данной работе мы подробно останавливаться на этом не будем.

3) С требованиями к масштабируемости, надежности и доступности обычно проблем не бывает - они реализуются с помощью распределенной (микросервисной) архитектуры, а также с помощью оркестраторов контейнеров, таких как Kubernetes (K8s). Поэтому подробно останавливаться на этом мы не будем.

1.12 Ограничения в рамках работы.

В рамках данной работы мы часто будем обсуждать разнообразные языки и проблемы, которые возникают при трансляции между произвольными парами таких языков. Однако, это будут больше теория, т.к. главная пара языков, на которой сконцентрировано внимание в данной работе - это C# и Python. В рамках данной работы мы главным образом будем изучать трансляцию кода, написанного на языке C#, в код на языке Python, и сохранение свойств безопасности при такой трансляции.

2. Обзор подходов

Давайте сделаем краткий обзор подходов, которые используются для обеспечения безопасности при компиляции (трансляции).

2.1 Secure compilation (безопасная компиляция)

Подход secure compilation (безопасной компиляции) представляет собой направление исследований в области теории компиляторов, компьютерной безопасности и формальных методов, направленное на обеспечение сохранения свойств безопасности программы при ее трансляции из исходного языка в целевой

язык (машинный код, байткод или некоторый низкоуровневый язык, такой как IR). В отличие от традиционных компиляторов, таких как GCC, LLVM и другие, которые ориентированы преимущественно на сохранение семантики и оптимизацию производительности, безопасная компиляция требует более сильных гарантий, связанных с поведением программы в условиях потенциально враждебного окружения.

Классическое понятие корректности компиляции предполагает, что поведение скомпилированной программы эквивалентно поведению исходной программы. Однако в контексте безопасности этого недостаточно, поскольку после компиляции программа может взаимодействовать с низкоуровневой средой исполнения, в которой отсутствуют гарантии, предоставляемые высокоуровневым языком, такие как контроль доступа, типовая безопасность или автоматическое управление памятью. В результате возможны атаки, использующие особенности целевого уровня, например, нарушения границ памяти или вмешательство в поток управления.

В рамках *secure compilation* формулируются более строгие критерии корректности. Одним из фундаментальных является критерий полной абстракции (*fully abstract compilation*), согласно которому две программы, неразличимые в исходном языке, должны оставаться неразличимыми после компиляции. Это означает, что компилятор не должен вводить дополнительные наблюдаемые различия, которые могли бы быть использованы атакующим для получения информации о внутреннем устройстве программы. Данный критерий обеспечивает сохранение поведенческой эквивалентности программ в присутствии произвольного контекста.

Более практичным подходом является сохранение свойств безопасности при взаимодействии с произвольным окружением (*robust property preservation*). В этом случае требуется, чтобы свойства, доказанные для программы на уровне исходного языка, сохранялись даже при выполнении в присутствии вредоносного или неконтролируемого кода. К таким свойствам относятся конфиденциальность, целостность данных и безопасность управления потоком выполнения. Данный подход учитывает реалистичные сценарии эксплуатации программ, в которых они редко выполняются в полностью доверенной среде.

Реализация безопасной компиляции может осуществляться различными способами. Один из подходов связан с использованием аппаратной поддержки безопасности: архитектура CHERI [102], реализующей модель *capability-based* доступа, технология защищенных анклавов, таких как Intel SGX [103] и т.д. Эти решения позволяют изолировать память и ограничивать доступ к ресурсам на уровне аппаратного обеспечения, тем самым усиливая гарантии безопасности скомпилированного кода.

Другой подход основан на программных методах изоляции, включая *Software Fault Isolation* и “выполнение в песочнице” (*sandbox*). В качестве примера безопасной целевой платформы часто рассматривается *WebAssembly*, обеспечивающий строгую модель исполнения с контролем памяти и ограничением взаимодействия между модулями. Такие механизмы позволяют ограничить последствия возможных ошибок или атак внутри отдельных компонентов системы.

Существенную роль играют также методы, основанные на формальной верификации и криптографии. В частности, подход *proof-carrying code* (PCC) [104]

предполагает, что программа сопровождается формальным доказательством своей корректности и безопасности, которое проверяется перед выполнением. Это позволяет перенести часть ответственности за безопасность на этап компиляции и статического анализа.

Отдельным направлением является типобезопасная компиляция, в рамках которой компилятор гарантирует сохранение типовых свойств исходной программы на уровне целевого кода. Это особенно важно для языков с развитой системой типов, таких как C#, Java, Rust, где многие гарантии безопасности основаны именно на типовой системе.

Формальное обоснование корректности и безопасности компиляции требует использования строгих математических методов. Среди них выделяются логические отношения, операционная и денотационная семантика, а также различные виды симуляций и бисимуляций. Эти методы позволяют доказать, что поведение программы сохраняется при трансляции, а также что никакие дополнительные возможности для атакующего не возникают на уровне целевого кода.

На практике идеи secure compilation реализуются в ряде систем, таких как CompCert - формально верифицированный компилятор языка C [105], seL4 - микроядро операционной системы с доказанной корректностью [106], а также WebAssembly как безопасная платформа выполнения [107].

Несмотря на достигнутые результаты, secure compilation остаётся сложной и активно развивающейся областью. Основные проблемы включают высокую сложность формальной верификации компиляторов, дополнительные накладные расходы на обеспечение безопасности, а также трудности интеграции с существующими промышленными системами. Кроме того, остаётся открытым вопрос о полноте моделей угроз, особенно в контексте атак по побочным каналам.

Как результат, безопасная компиляция представляет собой важный шаг в направлении построения надежных программных систем, обеспечивая не только функциональную корректность, но и устойчивость программ к широкому спектру атак при их выполнении в небезопасной среде.

2.2 Translation validation (валидация трансляции)

Подход translation validation (валидация трансляции) представляет собой метод обеспечения корректности компиляции, при котором проверка соответствия исходной и скомпилированной программ выполняется не на уровне самого компилятора, а отдельно для каждого результата трансляции. Данное направление развивается в рамках теории компиляторов и формальных методов и рассматривается как альтернатива полной формальной верификации компилятора.

В отличие от традиционного подхода, реализованного, например, в системах наподобие CompCert [105], где корректность компилятора доказывается один раз и распространяется на все компилируемые программы, валидация трансляции предполагает, что результат каждой конкретной компиляции сопровождается отдельной проверкой. Таким образом, вместо доказательства корректности самого компилятора

осуществляется проверка того, что для данной пары «исходная код — целевая программа» сохраняется требуемое отношение эквивалентности.

Основная идея подхода заключается в следующем: после выполнения компиляции специальный валидатор анализирует исходную код и скомпилированную программы и проверяет, что они эквивалентны с точки зрения наблюдаемого поведения. При этом компилятор может рассматриваться как недоверенный компонент, а корректность обеспечивается за счет независимой процедуры проверки. Такой подход позволяет использовать сложные оптимизирующие компиляторы без необходимости их полной формальной верификации.

Формально задача валидации трансляции сводится к проверке семантического соответствия двух программ (исходного кода и скомпилированной программы), заданных на разных уровнях абстракции. В зависимости от модели вычислений и языка программирования могут использоваться различные критерии эквивалентности, включая эквивалентность трасс выполнения, сохранение вход-выходного поведения или более строгие формы наблюдаемой эквивалентности. Важным аспектом является учёт оптимизаций, выполняемых компилятором, которые могут существенно изменять структуру программы, сохраняя при этом ее поведение.

Для реализации валидации трансляции применяются методы формальной верификации, такие как символьное исполнение, проверка достижимости состояний, а также построение и сопоставление промежуточных представлений программ. Широко используются техники, основанные на SMT-решателях и автоматическом доказательстве теорем, что позволяет эффективно проверять эквивалентность даже в присутствии нетривиальных преобразований кода.

Одним из ключевых преимуществ подхода является его практическая применимость. В отличие от полной верификации компилятора, которая требует значительных усилий и сложной формализации, валидация трансляции может быть интегрирована в существующие цепочки компиляции с относительно небольшими затратами. Это делает её особенно привлекательной для использования в промышленных системах, где применяются сложные и постоянно развивающиеся компиляторы, например, такие как LLVM.

В то же время данный подход имеет ряд ограничений. Во-первых, проверка выполняется для каждого отдельного результата компиляции, что может приводить к дополнительным накладным расходам по времени. Во-вторых, сложность задачи эквивалентности программ (исходного кода и скомпилированной программы) может существенно возрасти при наличии агрессивных оптимизаций, побочных эффектов и взаимодействия с внешней средой. В-третьих, в некоторых случаях требуется ослабление критериев эквивалентности, что может снижать строгость гарантируемых свойств.

Как результат, валидация трансляции представляет собой компромисс между полной формальной верификацией компилятора и отсутствием формальных гарантий корректности. Она позволяет повысить доверие к результатам компиляции, обеспечивая проверку сохранения поведения программы на уровне отдельных трансляций, и является важным инструментом в разработке надежных и безопасных программных систем.

2.3 Verified compilation (верифицируемая компиляция)

Подход *verified compilation* (верифицируемая компиляция) представляет собой направление в области теории компиляторов и формальных методов, ориентированное на построение компиляторов, корректность которых доказана формально с использованием математических методов. В рамках данного подхода компилятор рассматривается как объект строгой спецификации, а его поведение доказывается соответствующим этой спецификации, что позволяет гарантировать сохранение семантики программ при трансляции.

В отличие от методов, основанных на тестировании или эмпирической проверке, верифицируемая компиляция предполагает построение формального доказательства того, что для любой допустимой исходной программы (исходного кода) результат компиляции эквивалентен ей с точки зрения наблюдаемого поведения. Это означает, что выполнение скомпилированной программы приводит к тем же результатам, что и выполнение исходной программы (как если бы она выполнялась с помощью интерпретатора), с учетом допустимых различий, например, связанных с неопределенным поведением в языках низкого уровня.

Формально корректность компилятора обычно выражается в терминах симуляции между семантиками исходного и целевого языков. Наиболее распространенным является доказательство прямой (*forward*) симуляции, гарантирующей, что каждый шаг выполнения исходной программы может быть воспроизведен соответствующей последовательностью шагов целевой программы. В ряде случаев используется также обратная (*backward*) симуляция или бисимуляция, позволяющая устанавливать более сильные свойства соответствия.

Практическая реализация данного подхода требует формализации как исходного языка программирования, так и целевого языка (машинный код, байткод или некоторый низкоуровневый язык, такой как IR), а также всех этапов компиляции, включая промежуточные представления. Для этого широко применяются системы автоматического доказательства теорем и интерактивные доказательные ассистенты, такие как Coq и Isabelle. В этих системах формализуются синтаксис, семантика и трансформации программ, после чего строится машинно-проверяемое доказательство корректности.

Одним из наиболее известных примеров является CompCert - компилятор языка C [105], корректность которого формально доказана в системе Coq. В данном проекте показано, что результат компиляции сохраняет поведение программы при выполнении на целевой архитектуре, что существенно повышает уровень доверия к программному обеспечению, созданному с его использованием.

Верифицируемая компиляция обеспечивает сильные гарантии корректности и надежности, что особенно важно для критически важных систем, включая авиацию, медицину и встроенные системы. В отличие от подхода *translation validation* (валидация трансляции), где корректность проверяется для каждой отдельной трансляции, здесь доказательство относится ко всему компилятору и распространяется на все программы, которые он обрабатывает (компилирует).

В то же время данный подход характеризуется высокой сложностью реализации. Формализация компилятора требует значительных усилий, а доказательства могут достигать большого объема и сложности. Это затрудняет поддержку и развитие таких систем, особенно в условиях необходимости внедрения новых оптимизаций и поддержки современных языковых конструкций. Кроме того, верификация обычно проводится для упрощенных моделей языков и архитектур, что ограничивает применимость результатов в полномасштабных промышленных компиляторах, например, таких как LLVM.

Как результат, верифицируемая компиляция представляет собой наиболее строгий подход к обеспечению корректности трансляции программ, обеспечивая формально доказанные гарантии сохранения их поведения. Несмотря на высокую стоимость разработки, данный подход играет важную роль в создании высоконадежных программных систем и служит основой для дальнейших исследований в области безопасной и доверенной компиляции.

2.4 Сравнение с используемым подходом.

Давайте сравним рассмотренные выше подходы, которые используются для обеспечения безопасности при компиляции (трансляции) с тем, что предлагается использовать в данной работе:

- 1) Все приведенные выше подходы рассматривают все-таки процесс компиляции, т.е. получение из исходного кода результата в виде программы. В данной работе рассматривается вопрос трансляции кода с одного языка на другой - т.е. фактически компилятор для этого не используется.
- 2) У нас никак не валидируется исходный код, т.е. мы считаем, что он уже того качества, которое нас устраивает. Наша задача: выделить интересующие нас свойства безопасности из исходного кода, классифицировать их и транслировать код так, чтобы, свойства безопасности определенных типов (согласно классификации) сохранились при трансляции (см. далее [глава 3.7]). При этом полученный транслированный код у нас тоже не валидируется.
- 3) Фактически, мы можем проверять как исходный код до трансляции с помощью одного из подходов, так и полученный код в результате трансляции - это никак не вступает в конфликт с тем, что мы делаем. С помощью трансляции кода с сохранением семантики безопасности мы можем гарантировать, что при трансляции исходного кода определенного качества (с некоторым набором свойств безопасности) мы можем гарантировать, что получим транслированный код, качество которого не хуже некоторого порога (т.е. транслированный код будет обладать подмножеством свойств безопасности исходного кода, которое мы сможем определить до начала трансляции).

Какой из этого можно сделать вывод. Перечисленные выше подходы в основном решают задачи сохранения поведения и корректности при компиляции и верификации преобразований исходного кода в некоторый целевой язык (машинный код, байткод или некоторый низкоуровневый язык, такой как IR), т.е. верификация преобразований кода на высокоуровневом языке в код на низкоуровневом языке. В нашем же случае, мы преобразуем код с одного языка высокого уровня на другой язык тоже высокого уровня

и контролируем сохранение свойств безопасности при этом процессе. Другими словами, мы решаем несколько другие задачи, чем перечисленные выше подходы и никак с этими подходами не конкурируем. Более того, для дополнительной верификации результата трансляции мы можем использовать, например, с translation validation (валидация трансляции).

3. Модель безопасности

3.1 Свойства безопасности кода

Если взять какой-либо код и проанализировать его, то можно выделить некоторое множество свойств этого кода. Часть из них относятся к функциональным требованиям - например, использование метода наименьших квадратов для обработки данных; часть из них относится к реализации - например, для реализации метода обработчика данных использовался шаблон (паттерн) проектирования “Стратегия” [108]; а есть ряд свойств, которые относятся к безопасности. Давайте рассмотрим некоторые из них более подробно:

- Проверка (и экранирование) входных данных от пользователей (внешних систем) - очень много уязвимостей связано с тем, что входные данные напрямую, без каких-либо проверок и/или экранирования используются для построения результата.
- Проверка входных аргументов в функции/методе (как минимум для публичных/экспортируемых функций/методов) - отсутствие проверок позволяет передавать такие аргументы, которые приводят к ошибкам, либо к непредсказуемому поведению функции/метода.
- Правильная работа с ресурсами (отсутствие утечек ресурсов) - если неаккуратно работать с ресурсами (файлы, сетевые подключения и т.д.), то это может привести к тому, что приложение будет блокировать ресурсы, которые ему уже не нужны; в результате этого либо другие приложения не смогут получить доступ к этим ресурсам, либо в системе начнется их нехватка (из-за ограничений системы).
- Правильная работа с памятью (отсутствие утечек памяти) - если неаккуратно работать с памятью, то можно прийти к ситуации, когда не используемая приложением память остается занятой этим приложением, что в итоге приводит к нехватке памяти. Тут следует упомянуть, что это проблема не только unmanaged языков, в managed языках точно также может быть утечка памяти.
- Корректная работа с границами выделенной памяти при доступе к этой памяти. Если не соблюдать границы памяти при записи, то легко получить ситуацию повреждения памяти (когда при записи за границами выделенной области памяти происходит порча данных, принадлежащих другим областям памяти), При чтении за границами памяти тоже возможен ряд проблем: от чтения мусора, до ошибок доступа к памяти (при попытке обратиться к области памяти, на которую у программы нет прав).

- Сохранение инкапсуляции в ООП коде; т.е. сохранение корректного доступа к приватным членам.
- Сохранение корректного доступа к унаследованным членам в ООП коде.
- Типобезопасность - свойство (языка программирования), гарантирующее, что операции выполняются только над совместимыми типами данных.
- Гарантия состояния программы к возникновению исключений; есть 4 варианта этого свойства - отсутствие гарантий, базовая гарантия, сильная гарантия и гарантия отсутствия исключений. Отсутствие гарантий означает, что любое исключение переводит состояние программы в неопределенное. Базовая гарантия означает, что все объекты остаются в валидном состоянии (но их содержимое может измениться) и ресурсы не утекают. Сильная гарантия означает, что, если операция прервана исключением, состояние программы остается таким же, каким было до начала вызова. Гарантия отсутствия исключений означает, что данный код никогда не выбрасывает исключений.

Из перечня свойств безопасности видно, что одни свойства безопасности кода определяется самим кодом (например, проверка входных аргументов у функций/методов), а другие - языком и/или платформой (например, типобезопасность). Очевидно, что это различие происхождения свойств безопасности нужно учитывать, если мы хотим, чтобы при трансляции кода с одного языка на другой те или иные свойства безопасности сохранялись (об этом далее [глава 3.5]).

Следует сказать, что в данной работы под свойствами безопасности мы понимаем именно свойство безопасности (инварианты безопасности) самого кода и среды его выполнения (runtime), которые могут влиять на риск появления уязвимостей, ослабления ограничений или нарушения защитных гарантий при трансляции кода с одного языка на другой. Мы не рассматриваем свойства безопасности в широком инфраструктурном смысле (например, свойство минимальности используемых привилегий).

Может возникнуть вопрос, откуда мы берем список свойств безопасности? Ответ на этот вопрос будет следующий: мы берем перечень свойств безопасности не “с потолка”, а генерируем их на основе списка Common Weakness Enumeration (CWE) [101] и спецификации исходного языка. Например, рассмотрим следующую уязвимость из списка CWE - CWE-787: Out-of-bounds Write [109]. Очевидно, что ее причина в том, что мы пишем за границами выделенного буфера. Поэтому свойство безопасности, соответствующее данному CWE-787 будет следующее: “Корректная работа с границами выделенной памяти при доступе к этой памяти”.

3.2 Понятие Security Fingerprint.

Набор всех (или интересующих нас) свойств безопасности для данной кодовой базы называется Security Fingerprint. Свойство безопасности может иметь одно из следующих значений (очевидно, что приведенный ниже набор значений покрывает все возможные случаи):

- поддерживается полностью;
- поддерживается частично (либо с какими-то ограничениями);

- не поддерживается.

Следует сказать, что у разных сущностей (функций, методов, типов) в кодовой базе могут быть разные значения для одних и тех же свойств безопасности. Например, в функции **Func1** проверяются все входные аргументы, в функции **Func2** проверяются только некоторые входные аргументы, а в функции **Func3** входные аргументы вообще не проверяются. В этом случае значение свойства безопасности “проверка входных аргументов” для функции **Func1** будет “поддерживается полностью”, для функции **Func2** - “поддерживается частично”, а для функции **Func3** - “не поддерживается”.

Из этого следует, что Security Fingerprint может быть либо детализированным (когда для каждой сущности указываются свои свойства безопасности, либо агрегирующими, когда указываются общие свойства всей кодовой базы. Для агрегирующего Security Fingerprint все свойства безопасности должны учитывать все возможные значения у всех таких же свойств среди всех сущностей кодовой базы - т.е. слияние (merge) их значений. В следующей таблице (Таблица [номер]) приведены правила слияния двух значений (которые легко могут быть обобщены на случай произвольного количества значений):

	Значение 2		
Значение 1	поддерживается полностью	поддерживается частично	не поддерживается
поддерживается полностью	поддерживается полностью	поддерживается частично	поддерживается частично
поддерживается частично	поддерживается частично	поддерживается частично	поддерживается частично
не поддерживается	поддерживается частично	поддерживается частично	не поддерживается

Также следует сказать, что родительские сущности не наследуют значения свойств безопасности при формировании Security Fingerprint. Пусть, например, класс **Class1** содержит метод **Method1**; пусть у метода **Method1** значение свойства безопасности “проверка входных аргументов” равно “поддерживается частично”. У класса **Class1** в этом случае не будет агрегирующего значения свойства безопасности “проверка входных аргументов” по всем методам (хотя, конечно, при необходимости его можно определить и ввести).

Давайте формализуем понятие Security Fingerprint (полную формализацию см. далее [глава 3.9]):

Агрегирующий Security Fingerprint (состоящий из N свойств безопасности) мы можем записать следующим образом:

$$F_a(P_A) = (i_1, i_2, \dots, i_n),$$

где:

i_k - свойство безопасности (инвариант безопасности)

Пусть e_j - некоторая сущность, определенная в исходном коде программы P_A :

$$e_j \in E(P_A),$$

где

$E(P_A)$ - множество всех сущностей, определенных в программе P_A .

Пусть $I(e_j)$ - множество всех свойств безопасности, определенных для сущности e_j ,

$$I(e_j) = \{i_1, i_2, \dots, i_m\},$$

где:

i_k - свойство безопасности (инвариант безопасности)

Тогда детализированный Security Fingerprint (состоящий из свойств безопасности M сущностей, определенных в исходном коде программы P_A), мы можем записать следующим образом:

$$F_d(P_A) = (I(e_1), I(e_2), \dots, I(e_m))$$

3.3 Сравнение Security Fingerprint.

Основная цель Security Fingerprint - это не просто сформировать набор свойств безопасности, а уметь сравнивать два набора. Для чего это нужно: в первую очередь для того, чтобы понимать, как изменился набор свойств безопасности при трансляции кода.

Первым делом давайте определим, как соотносятся между собой значения свойств безопасности (какое из значений считается более безопасным); эти правила приведены в следующей таблице (Таблица [номер]):

	Значение 2		
Значение 1	поддерживается полностью	поддерживается частично	не поддерживается
поддерживается полностью	одинаково безопасно	значение 1 более безопасно	значение 1 более безопасно
поддерживается частично	значение 2 более безопасно	не определено	значение 1 более безопасно
не поддерживается	значение 2 более безопасно	значение 2 более безопасно	одинаково безопасны

Следует отметить, что когда значения у сравниваемых свойств равны значению “поддерживается частично”, то результат сравнения мы считаем неопределенным. Пусть, например, мы говорим о свойстве безопасности “проверка входных аргументов” для функций **Func1** и **Func2**; у функции **Func1** проверяется только первый аргумент, а у **Func2** - только второй и третий аргументы. Какая из функций более безопасна?

Очевидно, что мы не можем сказать это с определенностью; именно поэтому для этого случая результат сравнения значений свойств безопасности не определен.

Также следует сказать, что бессмысленно сравнивать значения двух разных свойств. Например, если мы сравним значения свойств безопасности “проверка входных аргументов” и “сохранение инкапсуляции”, то что даст результат их сравнения? Очевидно, что ничего осмысленного.

А теперь давайте определим, как мы будем сравнивать два набора Security Fingerprint - один, сформированный для исходного кода, а другой - сформированный для транслированного кода.

Первым делом определим, как мы сопоставляем свойства из двух Security Fingerprint для того, чтобы сравнить их значения; возможно несколько вариантов:

- 1) В обоих Security Fingerprint есть свойство с одним и тем же именем; например, свойство безопасности “проверка входных аргументов”. В этом случае все очевидно - такие свойства сопоставляются.
- 2) В двух Security Fingerprint есть два свойства, имена которых совпадают по семантическому смыслу. Пусть, например, мы транслируем код с языка C# на язык Golang; в коде на языке C# у нас есть свойство безопасности “сильная гарантия к возникновению исключений”. Но в языке Golang нет исключений - вместо этого в языке Golang используются ошибки (обычно функция в языке Golang возвращает пару значение - ошибка); поэтому в коде на языке Golang будет свойство безопасности “сильная гарантия к возникновению ошибок”. Вот при сравнении двух Security Fingerprint, мы будем сопоставлять эти два свойства в случае трансляции кода с языка C# на язык Golang. В этом случае сопоставлять два свойства нужно аккуратно - только если на 100% есть уверенность, что эти два свойства семантически эквивалентны, т.к. в противном случае мы будем сопоставлять свойства, которые сопоставлять не имеет никакого смысла.
- 3) Во всех остальных случаях мы считаем, что свойства из двух Security Fingerprint сопоставить невозможно.

Давайте поговорим о сравнении двух агрегирующих Security Fingerprint: вначале мы сопоставляем свойства безопасности в этих Security Fingerprint, после чего сравниваем их согласно таблице, приведенной выше (Таблица [\[номер\]](#)). Так как мы сравниваем два агрегирующих Security Fingerprint от исходного кода и кода после трансляции, то мы можем считать, что набор свойств Security Fingerprint относительно их сопоставления одинаковый. В редких случаях, когда либо в одном, либо в другом Security Fingerprint появляется свойство, которому невозможно сопоставить пару, то мы просто будем дописывать наличие этого свойства (вместе с его значением) к результатам сравнения.

Теперь поговорим о сравнении детализированным Security Fingerprint. Детализированный Security Fingerprint содержит все сущности (типы, методы, функции и т.д.), определенные в коде и для них определяет свойства безопасности (см. [глава 3.1]). Поэтому мы сначала определяем соответствие между сущностями в исходном коде и в транслированном коде, потом для сущностей, для которых было найдено соответствие, сравниваем их свойства безопасности. В итоге мы получим

детализированный результат сравнения сущностей с дополнительным списком сущностей, для которых не удалось найти пару.

Вполне может возникнуть вопрос: а имеет ли смысл сравнивать два набора Security Fingerprint для двух кодовых баз, которые создавались независимо, но которые решают одну и ту же задачу. Если эти два набора Security Fingerprint будут агрегирующими, то смысл есть - мы можем оценить, насколько качественно то или иное решение с точки зрения свойств безопасности. А вот сравнивать два набора детализированных Security Fingerprint нет никакого смысла, что очевидно.

3.4 Что такое семантика безопасности кода.

Теперь мы готовы определить, что же такое семантика безопасности кода. Семантику безопасности можно определить как набор свойств безопасности (или инвариантов безопасности или Security Fingerprint), которые должны сохраняться (вместе с их значениями) во время всего жизненного цикла программы. Если какое-либо свойство безопасности в разное время жизненного цикла программы имеет разные значения, то необходимо построить его суммарное значение, пользуясь правилами слияния значений свойств безопасности (см. [глава 3.3]).

Семантика безопасности - это неотъемлемое свойство исходного кода. С ее помощью можно описывать безопасность этого кода - например, предсказывать потенциальные уязвимости. Также с ее помощью можно отслеживать как меняется безопасность кода при его трансформации: как свойства безопасности ослабляются или вообще исчезают. Это утверждение справедливо и для случая, когда мы говорим о трансляции кода; поэтому определение семантики безопасности кода является для нас одним из ключевых определений.

3.5 Классы безопасности. Эволюция Security Fingerprint во время трансляции.

Очень важно понимать, как себя ведут свойства безопасности при трансляции кода. Но первым делом следует показать, что свойства безопасности кода не обязаны сохраняться при трансляции с одного языка на другой. Давайте рассмотрим пример кода на языке C#:

```
namespace SomeNamespace
{
    public class ClassA
    {
        public void Do(string data)
        {
        }
    }

    public class ClassB
    {
```

```

        public void Do(string data)
        {
        }
    }

    public class Processor
    {
        public void Process(ClassA objA)
        {
            objA.Do("IDDQD");
        }
    }

    public class Program
    {
        public static void Main(string[] args)
        {
            ClassA objA = new ClassA();
            ClassB objB = new ClassB();
            Processor processor = new Processor();
            processor.Process(objA);
            //processor.Process(objB);
        }
    }
}

```

Данный код обладает свойством типобезопасности - в метод **Process** класса **Processor** можно передать только экземпляр (объект) класса **ClassA**; если в метод **Process** класса **Processor** попробовать передать экземпляр (объект) класса **ClassB**, то мы получим ошибку компиляции (именно поэтому последняя строка в методе **Main** класса **Program** закомментирована).

А теперь давайте посмотрим, как мог бы выглядеть подобный пример в результате трансляции на язык Python:

```

class ClassA:
    def Do(self, data: str) -> None:
        pass

class ClassB:
    def Do(self, data: str) -> None:
        pass

class Processor:

```

```

def Process(self, objA: ClassA) -> None:
    objA.Do("IDDQD")

class Program:
    @staticmethod
    def Main(args: list[str]):
        objA = ClassA()
        objB = ClassB()
        processor = Processor()
        processor.Process(objA)
        processor.Process(objB)

```

Данный код обладает уже меньшей типобезопасностью, т.к. в метод *Process* класса *Processor* можно передать как экземпляр (объект) класса *ClassA*, так и экземпляр (объект) класса *ClassB* (так и экземпляр любого класса, у которого есть метод *Do*). С точки зрения языка Python здесь все в порядке и никаких проблем нет, но вот с точки зрения исходного кода здесь есть одна проблема - в исходном коде в метод *Process* класса *Processor* можно было передать только экземпляр (объект) класса *ClassA*, тогда как в коде, полученном в результате трансляции, в метод *Process* класса *Processor* можно передать как экземпляр (объект) класса *ClassA*, так и экземпляр (объект) класса *ClassB*. Видно, что при трансляции произошло снижение типобезопасности кода.

Давайте попробуем классифицировать поведение свойств безопасности кода при трансляции:

- 1) Первый тип свойств безопасности - это свойства, которые автоматически переносятся при трансляции. К этому типу в основном относятся свойства, определенные самим кодом - проверки входных аргументов в функциях/методах и т.п.
- 2) Второй тип свойств безопасности - это свойства, которые не поддерживаются языком/платформой, но могут быть более-менее просто симитированы (реализованы вручную).

Давайте снова вернемся к предыдущему примеру - предположим, что мы транслировали исходный код в код на языке Python следующим образом:

```

class ClassA:
    def Do(self, data: str) -> None:
        pass

class ClassB:
    def Do(self, data: str) -> None:
        pass

class Processor:
    def Process(self, objA: ClassA) -> None:
        if not isinstance(objA, ClassA):
            raise ValueError("obj is not ClassA instance")

```

```

objA.Do("IDDQD")

class Program:
    @staticmethod
    def Main(args: list[str]):
        objA = ClassA()
        objB = ClassB()
        processor = Processor()
        processor.Process(objA)
        #processor.Process(objB)

```

В этом варианте мы добавили в метод *Process* класса *Processor* проверку (с помощью встроенной функции *isinstance* [110]), что тип объекта, передаваемого в аргумент *objA* - это именно *ClassA*. С такой проверкой в метод *Process* класса *Processor* уже не удастся передать экземпляр (объект) класса *ClassB* (именно поэтому последняя строка в методе *Main* класса *Program* закомментирована). Т.о. мы смогли симитировать поведение в коде на языке Python, поведение кода на исходном языке. А это означает, что при трансляции мы смогли сохранить такое свойство исходного кода, как типобезопасность.

3) Третий тип свойств безопасности - это свойства, которые не поддерживаются языком/платформой и не могут быть симитированы (либо их имитация крайне сложна). Рассмотрим, например, такой принцип, применяемый в ООП, как инкапсуляция. Если посмотреть на язык C#, то в нем инкапсуляция есть; поля, свойства, методы и т.д. объявленные с помощью модификатора доступа *private* доступны только внутри класса, в котором они объявлены. Да, конечно, к ним можно получить доступ с помощью рефлексии - .NET Reflection API [111], но это явно не стандартный механизм, а некоторый “черный” ход. С другой стороны, в языке Python никакого механизма инкапсуляции нет - есть соглашение, что имена, начинающиеся с двойного символа подчеркивания - это приватные члены; к таким именам применяется механизм искажения имен (name mangling) [112] - т.е. если член с именем *__member* объявлен в классе *SomeClass*, то его имя становится *_SomeClass__member*. Но это не настоящая инкапсуляция - программист может легко получить доступ к приватному члену просто преобразуя его имя и не используя никакое специальное АПИ для этого; скорее это способ избежать случайных конфликтов имен при наследовании. Поэтому такое свойство безопасности исходного кода, как инкапсуляция (а также ограничение доступа к членам для наследников) пропадает при трансляции. Можем ли мы его как-то симитировать? Для методов и свойств ответ будет скорее всего положительным: мы можем посмотреть стек вызова данного метода или свойства (например, с помощью функции *inspect.stack()* модуля *inspect* [113]) и проверить, имеет ли права на вызов данного члена вызывающий код (для приватного члена вызывающий код должен находится в одном из членов того же класса, в котором определен проверяемый приватный член). Однако для полей мы ничего сделать не можем. Таким образом, фактически мы не можем сохранить такое свойство безопасности исходного кода, как инкапсуляция при трансляции исходного кода с языка C# на язык Python.

4) Четвертый тип свойств безопасности - это свойства, которые отсутствовали в исходном коде, но появились в коде после трансляции, причем не важно из-за действий транслятора или из-за свойств языка/платформы. Предположим, что мы переносим код с языка C на язык Python; предположим, что в коде на языке C для работы со строками используются такие небезопасные функции, как *strcpy*, *strcat* и другие (т.е. функции, которые не проверяют границы буфера при записи данных) [114]. Очевидно, что в таком коде на языке C такое свойство, как безопасность строковых операций отсутствует. Однако, как только мы транслируем этот код на язык Python, так сразу вся работа со строками станет безопасной, т.к. это фундаментальное свойство языка Python и его среды выполнения. Поэтому после трансляции у нашего кода появится такое свойство безопасности, как безопасность строковых операций.

Можно показать, что приведенный выше набор классов покрывает основные типы поведения свойств безопасности кода при трансляции. Таким образом, мы можем считать нашу классификацию поведения свойств безопасности кода при трансляции достаточной для наших задач.

Обозначим типы (классы) свойств безопасности следующим образом:

- L1 - свойства безопасности, которые автоматически сохраняются при трансляции;
- L2 - свойства безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, но которые мы можем симитировать (с помощью дополнительного кода, генерируемого при трансляции);
- L3 - свойства безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, и которые мы не можем симитировать (либо их имитация крайне сложна);
- L4 - свойства безопасности, поддержка которых отсутствует в исходном коде, но которые появляются в коде после трансляции, причем не важно из-за действий транслятора или из-за свойств языка/платформы.

А теперь давайте посмотрим на эти типы (классы) свойств безопасности с точки зрения того, как мы будем обрабатывать при трансляции соответствующие свойства, принадлежащие тому или иному классу:

- L1 - для сохранения свойств этого класса ничего делать не нужно (мы их просто отслеживаем), т.к. они автоматически переносятся при трансляции кода.
- L2 - для сохранения свойств этого класса требуется генерация дополнительного кода (компенсирующих механизмов), т.к. в противном случае они не сохранятся при трансляции кода.
- L3 - свойства этого класса сохранить невозможно (без переработки исходного кода), поэтому мы можем только зафиксировать их исчезновение как допустимое или недопустимое для нас ослабление итогового Security Fingerprint.
- L4 - свойства этого класса являются “приятным бонусом” при трансляции кода; мы просто фиксируем их появление в коде после трансляции.

Можно оформить эту обработку в виде функции принятия решения следующего вида (Таблица [номер]):

Класс свойств безопасности	Действие над свойствами безопасности
L1	Ничего не делаем (отслеживаем).
L2	Генерируем дополнительный код (компенсирующие механизмы).
L3	Фиксируем допустимое или недопустимое (для нас) ослабление (исчезновение свойства безопасности).
L4	Фиксируем улучшение.

Давайте повторим одну из самых главных мыслей о классификации свойств безопасности еще раз: исчезновение свойств безопасности, принадлежащих классу L3, не является какой-либо ошибкой или недоработкой транслятора; это происходит из-за особенностей языка/платформы, на которую мы транслируем исходный код. Например, то, что исчезает инкапсуляция при трансляции кода с языка C# на язык Python - это фундаментальное свойство языка Python.

Следует сделать еще следующее замечание: класс (тип) какого-либо свойства безопасности зависит от пары языков - исходного и на который происходит трансляция; если пара языков изменится, то и класс (тип) этого свойства безопасности тоже может измениться. Так, например, свойство безопасности “сохранение инкапсуляции” для пары языков C# - Python имеет класс (тип) L3; однако это же свойство для пары языков C# - C++ имеет класс (тип) L1, т.к. в языке C++ есть полная поддержка инкапсуляции.

3.5.1 Попытка сохранения правильного доступа к членам при трансляции кода с языка C# на язык Python.

Так как мы часто упоминаем проблему с исчезновением инкапсуляции и модификаторов доступа при трансляции кода с языка C# на язык Python, то давайте рассмотрим этот пример более подробно. Вот исходный код на C#, который демонстрирует, что инкапсуляция и модификаторы доступа работают так, как от них это ожидается:

```
namespace SomeNamespace
{
    public class ClassA
    {
        public void Do(string data)
        {
            Do2(data);
            Do3(data);
            value1 = 1;
            value2 = 2;
        }
    }
}
```



```

        value3 = 3;
    }
    protected void Do2(string data)
    {
    }
    private void Do3(string data)
    {
    }
    public int value1;
    protected int value2;
    private int value3;
}

public class ClassB : ClassA
{
    public void Make(string data)
    {
        Do(data);
        Do2(data);
        //Do3(data);
        value1 = 111;
        value2 = 222;
        //value3 = 3;
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        ClassA objA = new ClassA();
        ClassB objB = new ClassB();
        objA.Do("IDDQD");
        //objA.Do2("IDDQD");
        //objA.Do3("IDDQD");
        objB.Do("IDKFA");
        objB.Make("IDKFA");
        Console.WriteLine(objA.value1);
        //Console.WriteLine(objA.value2);
        //Console.WriteLine(objA.value3);
        Console.WriteLine(objB.value1);
        //Console.WriteLine(objB.value2);
    }
}

```

```

    }
}
}

```

В связи с тем, что в языке C# инкапсуляция и модификаторы доступа поддерживаются полностью, часть строк в коде закомментирована; если их раскомментировать, то мы получим ошибку компиляции.

А теперь давайте посмотрим, как мог бы выглядеть подобный пример в результате трансляции на язык Python:

```

class ClassA:
    def __init__(self):
        self.value1 = 0
        self._value2 = 0
        self.__value3 = 0
    def Do(self, data: str) -> None:
        self._Do2(data)
        self.__Do3(data)
        self.value1 = 1
        self._value2 = 2
        self.__value3 = 3
    def _Do2(self, data: str) -> None:
        pass
    def __Do3(self, data: str) -> None:
        pass

class ClassB(ClassA):
    def Make(self, data: str) -> None:
        self.Do(data)
        self._Do2(data)
        self._ClassA__Do3(data)
        self.value1 = 111
        self._value2 = 222
        self._ClassA__value3 = 333

class Program:
    @staticmethod
    def Main(args: list[str]) -> None:
        objA = ClassA()
        objB = ClassB()
        objA.Do("IDDQD")
        objA._Do2("IDDQD")

```

```

objA._ClassA__Do3("IDDQD")
objB.Make("IDKFA")
print(objA.value1)
print(objA._value2)
print(objA._ClassA__value3)
print(objB.value1)
print(objB._value2)

```

Из транслированного примера на языке Python видно (по сравнению с исходным примером на языке C#), что в языке Python инкапсуляция не поддерживается совсем; модификаторов доступа как таковых нет, но разделение на публичные (public), защищенные (protected) и приватные (private) члены происходит с помощью префиксов имен (имена защищенных членов начинаются с одного символа подчеркивания, имена приватных членов - с двух). Как итог, в примере на языке Python мы имеем полный доступ ко всем членам, с учетом механизма искажения имен (name mangling) [112] для приватных (private) членов (для членов, имя которых начинается с двух символов подчеркивания).

Давайте модифицируем пример на языке Python так, чтобы поведение для методов соответствовало поведению для методов в исходном примере на языке C#:

```

import inspect

class ClassA:
    def __init__(self):
        self.value1 = 0
        self._value2 = 0
        self.__value3 = 0
    def Do(self, data: str) -> None:
        self._Do2(data)
        self.__Do3(data)
        self.value1 = 1
        self._value2 = 2
        self.__value3 = 3
    def _Do2(self, data: str) -> None:
        stack = inspect.stack()
        caller = stack[1].frame.f_locals.get('self') if len(stack)
> 1 else None
        if not (caller and isinstance(caller, ClassA)):
            raise ValueError("Bad caller")
        pass
    def __Do3(self, data: str) -> None:
        stack = inspect.stack()
        if len(stack) < 2:

```

```

        raise ValueError("Bad caller")
    caller_method = stack[1].function
    caller_code = stack[1].frame.f_code
    is_good_caller = False
    for current_cls in self.__class__.__mro__:
        if caller_method in current_cls.__dict__:
            method = current_cls.__dict__[caller_method]
            if hasattr(method, '__code__') and method.__code__
is caller_code:
                is_good_caller = current_cls is ClassA
                break
    if not is_good_caller:
        raise ValueError("Bad caller")
    pass

class ClassB(ClassA):
    def Make(self, data: str) -> None:
        self.Do(data)
        self._Do2(data)
        #self._ClassA_Do3(data)
        self.value1 = 111
        self._value2 = 222
        self._ClassA__value3 = 333

class Program:
    @staticmethod
    def Main(args: list[str]) -> None:
        objA = ClassA()
        objB = ClassB()
        objA.Do("IDDQD")
        #objA._Do2("IDDQD")
        #objA._ClassA__Do3("IDDQD")
        objB.Make("IDKFA")
        print(objA.value1)
        print(objA._value2)
        print(objA._ClassA__value3)
        print(objB.value1)
        print(objB._value2)

```

Из модифицированного примера на Python видно следующее (по сравнению с исходным примером на языке C#):

- Мы смогли за счет дополнительного кода получить такое же поведение для методов, которое в языке C# может быть получено с помощью модификаторов доступа public, protected, private.
- Этот дополнительный код работает через анализ стека вызовов; он достаточно сложен и не тривиален (и не факт, что быстрый).
- Самое что неприятное - защитить таким же способом доступ к полям в языке Python возможности нет

Таким образом, свойства безопасности “сохранение инкапсуляции” и “правильная работа модификаторов доступа” при трансляции кода с языка C# на язык Python имеет класс L3 по нашей классификации свойств безопасности.

3.6 Проблемы с внешними зависимостями

Отдельно следует рассмотреть проблему со внешними зависимостями (библиотеками) в разрезе свойств безопасности и их классов. Любой код (любая кодовая база) зависит так или иначе от некоторого набора внешних библиотек, с помощью которых он реализует свою функциональность. Даже в самом простом случае, у любого кода есть зависимость от базовой библиотеки (.NET Base Class Library, Python Standard Library, C++ Standard Library и т.д.), если не брать такие граничные случаи, как, например, ПО, которое напрямую работает с оборудованием.

С этими внешними зависимостями (библиотеками) есть одна большая проблема, связанная со свойствами безопасности - это наличие уязвимостей в этих зависимостях (библиотеках). Не для кого не является секретом, что во многих версиях многих зависимостей (библиотек) со временем исследователи или хакеры находят уязвимости той или иной степени критичности. Соответственно, как только это происходит, это сразу же меняет Security Fingerprint либо для исходного кода, либо для транслированного кода, либо для них обоих. Следует сказать, что при этом в классификации свойств безопасности новых классов не появляется. Также и состав (набор свойств) Security Fingerprint не меняется, но могут поменяться значения свойств. Например, значение свойства безопасности с “поддерживается” может измениться на “поддерживается частично” или “не поддерживается”, а значение “поддерживается частично” может измениться на “не поддерживается”. То есть, происходит “деградация” значений свойств безопасности. Соответственно, если мы не хотим, чтобы такая “деградация” Security Fingerprint со временем происходила, то мы должны периодически проверять используемые внешние зависимости (библиотеками) с помощью Software Composition Analysis (SCA) или анализа состава программного обеспечения [115]; после чего обновлять внешние зависимости (библиотеками) для которых обнаружены неприемлемые уязвимости. К сожалению, это далеко не такой простой процесс как может показаться:

- Никто не гарантирует, что более новая версия по АПИ или по поведению совместима со старой, особенно, если изменился мажорный номер версии внешней зависимости (библиотеки) - вполне возможно, что замена внешней зависимости (библиотеки) на более новую потребует значительных доработок либо в исходном коде, либо в транслированном коде, либо и там, и там.

- Вполне возможна ситуация, что новых версий какой-либо внешней зависимости (библиотеки) нет (например, ее забросили), а аналогов либо нет вообще, либо они очень сильно отличаются по дизайну АПИ и/или функционалу. В этом случае вполне допустима ситуация, что мы продолжаем использовать старую внешнюю зависимость (библиотеку), например, через некоторый промежуточный слой кода, который вводит дополнительные проверки, преобразования данных и т.д., чтобы минимизировать возможность использования уязвимостей в этой внешней зависимости (библиотеки).

Из этого можно сделать следующий вывод: внешние зависимости являются внешним фактором, который способен изменять состояние свойств безопасности после трансляции с течением времени и, как итог, приводить к “деградации” Security Fingerprint. Другими словами, с течением времени из-за внешних зависимостей семантика безопасности может становиться “слабее” уже после трансляции.

Есть еще одна потенциальная проблема с внешними зависимостями при трансляции кода: вполне возможна ситуация, что будут появляться новые сущности, либо исчезать сущности, которые есть в исходном коде при трансляции кода с одного языка на другой именно из-за несоответствия между внешними зависимостями (библиотеками) для исходного кода и для транслированного кода. Это не такая большая проблема, как проблема с возникновением уязвимостей, но это несоответствие между сущностями влияет на сравнение детализированных Security Fingerprint для исходного кода и для транслированного кода (см. [глава 3.2]).

3.7 Сохранение семантики безопасности при трансляции.

Мы видели выше из классификации (см. [глава 3.5]) свойств безопасности, что свойства безопасности, принадлежащие классу L1, автоматически сохраняются при трансляции кода, а свойства безопасности, принадлежащие классу L2, могут быть симитированы. Поэтому мы можем сказать, что семантика безопасности при трансляции кода с одного языка на другой сохраняется, если все свойства безопасности исходного кода, принадлежащие классам L1 и L2 сохраняются в коде после трансляции. Не сохранение свойств, принадлежащих классу L3, может быть крайне чувствительным при трансляции кода, но, пожалуй, единственный способ уменьшить их количество - это ручное изменение исходного кода (но даже это ручное изменение исходного кода не всегда помогает - например, с потерей инкапсуляции при трансляции кода с языка C# на язык Python мы ничего сделать не можем из-за природы языка Python). Свойства, принадлежащие классу L4, на сохранение семантики безопасности никак не влияют; однако они влияют на ее улучшение.

Исходя из написанного выше (а также с учетом сохранения функциональных требований и поведения) мы будем считать трансляцию кода успешной, если:

- 1) Все функциональные требования были сохранены при трансляции.
- 2) Поведение исходного кода и транслированного кода совпадает.
- 3) Все свойства безопасности, принадлежащие классам L1 и L2, будут сохранены.
- 4) Для нас приемлемо, что все свойства безопасности, принадлежащие классу L3, будут утеряны. Например, если потеря инкапсуляции при трансляции кода с языка C# на язык

Python для нас неприемлема (инкапсуляция при такой трансляции является свойством безопасности, принадлежащим классу L3 - см. выше [глава 3.5]), то в таком случае мы трансляцию кода будем считать неуспешной.

5) Все свойства безопасности, принадлежащие классу L4, будут для нас “приятным бонусом” - они не влияют на успешность или неуспешность трансляции.

3.8 Влияние окружения (контекста использования).

Программный код запускается не сам по себе - он существует в некотором окружении или контексте. И это окружение (контекст использования) определяет, как это код используется и может влиять в том числе и на требования безопасности, и на свойства безопасности. Окружение (контекст использования) легко учесть, когда мы его заранее знаем, но бывают случаи, когда программный код может запускаться в разных окружениях (контекстах использования) с разными требованиями к безопасности и разными свойствами безопасности.

Чтобы понять, как может влиять окружение (контекст использования) на программный код, давайте сначала посмотрим, как вообще мы можем запустить и/или использовать программный код. Варианты запуска и/или использования программного кода у нас есть следующие:

- 1) Запуск в виде standalone приложения.
- 2) Запуск в виде сервиса в некоторой программно-аппаратной конфигурации.
- 3) Использование в виде библиотеки.

Очевидно, что какой из вариантов запуска и/или использования программного кода доступен, зависит от того, как организован код и что мы с ним делаем. Если код организован в виде приложения, то его можно использовать как приложение, как сервис и, иногда, в виде библиотеки (например, в результате компиляции приложения, написанного на языке C#, мы получаем сборку, которую другое приложение может использовать как библиотечный код). Если код организован в виде библиотеки, то обычно мы можем использовать его только, как библиотеку. Сервисами обычно являются приложения, хотя в некоторых случаях сервисами могут быть и динамические библиотеки.

А сейчас давайте на примерах посмотрим в каких случаях у программного кода одно окружение (контекст использования), а в каких случаях больше одного:

- 1) Обычно, у standalone приложений одно окружение (контекст использования), в котором оно запускается и, как итог - у них один набор требований безопасности и свойств безопасности. Однако бывают случаи, когда у standalone приложений может быть больше одного окружения (контекста использования): например, когда приложение должно себя по разному ввести, когда оно запущено от имени обычного пользователя и когда оно запущено от имени администратора (обычно это свойственно системным утилитам, например, для Unix/Linux). Очевидно, что количество окружений (контекстов использования), которые должно поддерживать приложение зависит от создателя приложения и является четко определенной величиной на момент его создания.

2) У сервисов может быть как одно окружение (контекст использования), так и несколько, в которых эти сервисы могут использоваться. Например, web-сервис может использоваться как одиночный сервис, так и в составе некоторого кластера Kubernetes; и в этих двух разных сценариях использования web-сервиса у него вполне может быть два разных окружения (контекста использования) с разными наборами требований безопасности и свойств безопасности. Но также, как и в случае standalone приложений, количество окружений (контекстов использования), которые должен поддерживать сервис зависит от создателя сервиса и является четко определенной величиной на момент его создания.

3) А вот с библиотеками все уже не так однозначно, так создатель библиотеки не может предусмотреть все варианты использования его библиотеки: она может быть использована и в консольном приложении, и в приложении с GUI и в разнообразных сервисах и даже другими библиотеками. Понятно, что создатель библиотеки выбирает некоторый набор требований безопасности и свойств безопасности, которым должна удовлетворять создаваемая им библиотека. Рассмотрим, например, некоторую библиотеку. При ее создании, автор может добавить свойство безопасности “проверка входных аргументов функций” в свойства безопасности, которыми должна обладать библиотека; он даже может добавить свойство безопасности “корректная обработка ввода пользователя при формировании SQL-запросов” чтобы избежать потенциальных SQL-инъекций в коде. Но вот свойство безопасности “правильная работа с путями” в этой библиотеке вполне возможно, что отсутствует; и использование этой библиотеки в некотором web-сервисе, потенциально вносит уязвимость Path Traversal [116]. Должен ли был автор добавить свойство безопасности “правильная работа с путями” в список свойств безопасности библиотеки? Если библиотека изначально разрабатывалась для использования в среде web, то да; иначе - не обязательно. Таким образом видно, что наибольшую важность окружение (контекст использования) использования имеет для библиотек, т.к. авторы библиотек далеко не всегда могут предусмотреть все варианты их использования.

С точки зрения математической формализации (см. далее [глава 3.9]), это означает, что Security Fingerprint (отпечаток безопасности) для программы P_A зависит еще и от контекста использования этой программы (программного кода):

$$F = F(P_A, C),$$

где

C - окружение (контекст использования).

Дальше, для упрощения модели, мы будем считать, что окружение (контекст использования) у нас везде один; но при необходимости, мы всегда можем обобщить наши результаты на случай, когда окружений (контекстов использования) несколько.

3.9 Математическая формализация семантики безопасности кода.

Давайте формализуем с точки зрения математики некоторые определения, которые мы уже ввели или введем далее.

3.9.1. Формальная модель программы и трансляции.

Пусть программа P_A на языке А представлена как кортеж:

$$P_A = \left(AST_A, S_A, I_A \right),$$

где:

AST_A - синтаксическое дерево (abstract syntax tree) программы P_A ;

S_A - множество семантических свойств поведения программы P_A ;

I_A - множество свойств безопасности (инвариантов безопасности, security invariants) программы P_A .

Оператор трансляции Т программы (исходного кода) на языке А в программу (исходного кода) на языке В определяется как отображение:

$$T: P_A \rightarrow P_B,$$

где:

P_A - программа на языке А;

P_B - программа на языке В.

3.9.2. Модель свойств безопасности (инвариантов безопасности, security invariants).

Множество свойств безопасности (инвариантов безопасности, security invariants) I_A задается следующим образом:

$$I_A = \{i_1, i_2, \dots, i_n\},$$

где каждое свойство безопасности (инвариант безопасности) i_k - это функция над программой P_A (можно сказать, что это 3-значный предикат):

$$i_k: P_A \rightarrow \{0, 1/2, 1\},$$

где:

- значение 0 означает, что соответствующее свойство безопасности имеет значение “не поддерживается”;
- значение 1/2 означает, что соответствующее свойство безопасности имеет значение “поддерживается частично”;
- значение 1 означает, что соответствующее свойство безопасности имеет значение “поддерживается полностью”.

Мы считаем, что при трансляции Т свойство безопасности (инвариант безопасности) i_k сохраняется, если:

$$i_k(P_A) \neq 0 \wedge i_k(P_B) = i_k(P_A).$$

3.9.3. Классификация уровней сохранения.

Мы ввели (см. выше [глава 3.5]) следующие типы (классы) свойств безопасности:

- L1 - свойства безопасности, которые автоматически сохраняются при трансляции T;
- L2 - свойства безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, но которые мы можем симитировать при трансляции T;
- L3 - свойства безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, и которые мы не можем симитировать при трансляции T;
- L4 - свойства безопасности, поддержка которых отсутствует в исходном коде, но которые появляются в коде при трансляции T.

Тогда функция классификации может быть определена следующим образом:
 $\varphi(i_k) \in \{L1, L2, L3, L4\}$.

3.9.4. Security Fingerprint (отпечаток безопасности).

Как уже говорилось (см. выше [глава 3.2]), у нас есть два типа Security Fingerprint: агрегирующие и детализированные.

Агрегирующий Security Fingerprint (состоящий из N свойств безопасности) мы можем записать следующим образом:

$$F_a(P_A) = (i_1, i_2, \dots, i_n),$$

где:

i_k - свойство безопасности (инвариант безопасности)

Пусть e_j - некоторая сущность, определенная в исходном коде программы P_A :

$$e_j \in E(P_A),$$

где

$E(P_A)$ - множество всех сущностей, определенных в программе P_A .

Пусть $I(e_j)$ - множество всех свойств безопасности, определенных для сущности e_j

$$I(e_j) = \{i_1, i_2, \dots, i_m\},$$

где:

i_k - свойство безопасности (инвариант безопасности)

Тогда детализированный Security Fingerprint (состоящий из свойств безопасности M сущностей, определенных в исходном коде программы P_A), мы можем записать следующим образом:

$$F_d(P_A) = (I(e_1), I(e_2), \dots, I(e_m))$$

3.9.5. Метрика допустимого ослабления.

Как уже говорилось (см. выше [глава 3.7]), мы будем считать трансляцию кода успешной, если:

- Все свойства безопасности, принадлежащие классам L1 и L2, будут сохранены.
- Для нас приемлемо, что все свойства безопасности, принадлежащие классу L3, будут утеряны.

Поэтому допустимое ослабление Security Fingerprint может быть определена следующим образом:

$$\Delta(P_A, P_B) = \left| \left\{ i_k \in I_A : \varphi(i_k) = L3 \wedge i_k(P_A) \neq 0 \wedge i_k(P_B) = 0 \right\} \right|$$

При этом, обязательно должно выполняться условие:

$$\left| \left\{ i_k \in I_A : \varphi(i_k) \in \{L1, L2\} \wedge i_k(P_A) \neq 0 \wedge i_k(P_B) = 0 \right\} \right| = 0$$

Можно еще ввести метрику сохранения, которая определяется следующим образом:

$$Match(P_A, P_B) = \left| \left\{ i_k \in I_A : i_k(P_A) = i_k(P_B) \right\} \right| / |I_A|$$

На основании этой метрики, можно ввести допустимый порог потери свойств безопасности (которые принадлежат классу L3):

$$1 - Match(P_A, P_B) \leq \varepsilon,$$

где:

ε - оценка того, сколько свойств безопасности мы позволяем себе потерять.

Следует заметить, что эта оценка не учитывает важность свойств безопасности, т.к. не все свойства безопасности равнозначны (это зависит от проекта). Так, например, если для нас критически важно, чтобы инкапсуляция не терялась, а мы транслируем исходный код с языка C# на язык Python, то в этом случае для нас трансляция будет не успешной, даже если мы теряем всего одно это свойство.

3.9.6. Гипотеза симплификации

О симплификации мы будем говорить далее (см. [глава 4.6]), но формализацию гипотезы приведем здесь.

Пусть $C(AST)$ - сложность трансляции, пропорциональная количеству типов узлов, о которых должен знать транслятор (которые должен уметь обрабатывать транслятор). Очевидно (из определения того, что делает утилита для симплификации), что $C(AST)$ после симплификации всегда будет меньше или равно $C(AST)$ до симплификации:

$$C_{after}(AST) \leq C_{before}(AST)$$

Можно выдвинуть следующую гипотезу:

$$P_{after}(Error) \leq P_{before}(Error),$$

где:

$P_{before}(Error)$ - вероятность появления дефекта при трансляции до симплификации;

$P_{after}(Error)$ - вероятность появления дефекта при трансляции после симплификации.

3.10 Построение методики трансляции.

На данный момент мы умеем определять свойства безопасности, строить и сравнивать Security Fingerprint, классифицировать свойства безопасности при трансляции (классы L1, L2, L3, L4), определили, что такое семантика безопасности, как она сохраняется при трансляции и какие критерии у нас есть, чтобы считать трансляцию успешной. Мы также формализовали все это с математической точки зрения. Наш следующий шаг - это построение методики автоматической трансляции кода с сохранением семантики безопасности на основе подходов, описанных выше.

4. Методика автоматической трансляции

4.1 Теоретическая часть: что такое компиляция. Как она работает - лексический, синтаксический анализ, AST, семантика, генерация кода.

Компиляция представляет собой процесс преобразования программы, записанной на языке высокого уровня, в эквивалентную программу на более низком уровне абстракции, пригодную для непосредственного выполнения или дальнейшей обработки. Как правило, результатом компиляции является машинный код либо промежуточное представление, используемое, например, в инфраструктурах наподобие LLVM. Основной целью компиляции является сохранение функционального поведения программы при одновременном обеспечении эффективности ее выполнения. Также целью многих компиляторов является и обеспечение безопасности полученного результата (см. выше secure compilation, translation validation, verified compilation [глава 2]).

Первым этапом компиляции является лексический анализ, в ходе которого исходный текст программы разбивается на последовательность элементарных единиц - токенов. Токены представляют собой ключевые слова, идентификаторы, операторы и другие синтаксические элементы языка. На этом этапе устраняются (специальным образом обрабатываются) незначащие символы, такие как пробелы и комментарии, а также выявляются лексические ошибки.

Следующим этапом является синтаксический анализ, задача которого — установить, соответствует ли последовательность токенов грамматике языка программирования. Для этого используются формальные грамматики, чаще всего контекстно-свободные, задающие правила построения корректных программ. В процессе анализа строится синтаксическое представление программы, как правило, в виде абстрактного синтаксического дерева (AST), отражающего иерархическую структуру выражений и операторов

После построения AST выполняется семантический анализ, направленный на проверку корректности программы с точки зрения ее смысла. На данном этапе осуществляется проверка типов, областей видимости идентификаторов, корректности использования переменных и функций. Семантический анализ позволяет выявить ошибки, которые не могут быть обнаружены на уровне синтаксиса, например, использование необъявленных переменных или несоответствие типов.

Завершающим этапом первой фазы является генерация кода, в ходе которой абстрактное синтаксическое дерево преобразуется в целевое представление. Это может быть машинный код, байткод или промежуточный язык. На данном этапе часто выполняются оптимизации, направленные на повышение производительности и уменьшение объема кода. Результирующая программа должна сохранять поведение исходной, обеспечивая корректное выполнение в целевой среде.

Вторая фаза компиляции — компоновка (linking) — заключается в объединении полученных объектных модулей в единый исполняемый файл или библиотеку, предназначенные для конкретной целевой операционной системы. На этом этапе разрешаются внешние ссылки между модулями, подключаются необходимые библиотеки, а также формируется окончательная структура бинарного файла. Компоновщик учитывает особенности целевой платформы, включая формат исполняемых файлов и соглашения о вызовах функций. В результате формируется готовая программа, которая может быть загружена и выполнена в соответствующей среде.

Таким образом, компиляция представляет собой многоэтапный и двухфазный процесс, включающий трансляцию исходного кода в объектные модули и их последующую компоновку в конечный исполняемый файл. Каждый из этапов играет важную роль в обеспечении корректности, эффективности и совместимости программы с целевой системой.

Следует сказать, что на каждом шаге каждой фазы обеспечение безопасности кода играет важную роль, поэтому большинство современных компиляторов на каждом шаге проверяют состояние свойств безопасности (инвариантов безопасности) кода, с которым они работают.

4.2 Что такое автоматическая трансляция. Технические сложности, возникающие при трансляции.

С учетом определений, полученных выше при обсуждении того, что такое компиляция [глава 4.1], давайте определим, что такое автоматическая трансляция кода с одного языка на другой. Как уже говорилось выше, компиляция состоит из 2 фаз:

- 1) трансляция кода с предметно-ориентированного высокоуровневого языка (иногда, с нескольких языков) на машинно-ориентированный низкоуровневый язык (машинный код, низкоуровневый код для некоторой виртуальной машины, низкоуровневый промежуточный код для последующей JIT-компиляции);
- 2) компоновка результирующего бинарного файла (программы, библиотеки, и т.п.) для целевой ОС из сгенерированных на предыдущей фазе (фазе трансляции) объектных модулей.

Фаза 1 (фаза трансляции) обычно состоит из следующих шагов:

- 1) Лексический анализ (токенизация) - процесс разбора входной последовательности символов на распознанные группы - лексемы/токены.
- 2) Синтаксический анализ - процесс сопоставления линейной последовательности лексем/токенов формального языка с формальной грамматикой. Результатом является дерево разбора (синтаксическое дерево, AST).

3) Кодогенерация - процесс генерации последовательности инструкций, которые могут выполняться на целевой машине. Обычно, кодогенерация выполняется на синтаксическом дереве/AST.

Если на шаге кодогенерации фазы трансляции мы вместо генерирования последовательности инструкций, которые могут выполняться на целевой машине, будем генерировать сущностей кода (типы, функции, операторы, выражения и т.п.) на некотором языке, то это и будет называться автоматической трансляцией кода с исходного языка на некоторый другой язык.

Транслятором кода с исходного языка на некоторый другой язык называется программа, которая эту автоматическую трансляцию выполняет.

Существует ряд технических сложностей, с которыми неизменно придется столкнуться при создании транслятора кода - рассмотрим некоторые из них:

1) Построение AST из исходного кода. Если для платформы, на которой запускается исходный код есть библиотеки для построения AST (Roslyn для .NET [117], Clang LibTooling для C++ [119]), то делать ничего и не нужно. Если же нет - то нужно как при построении компилятора написать правила для лексики и синтаксиса, добавить действия для правил синтаксиса для генерации AST, сгенерировать лексический и синтаксический парсеры по правилам (например, с помощью ANTLR [119]), с их помощью получить AST для исходного кода. Кроме получения AST нужно также уметь получать информацию о типах. Облегчает задачу тот факт, что для большинства современных языков все же есть библиотеки для получения AST.

2) В некоторых случаях нам нужна возможность изменения AST и генерации по ней кода на исходном языке (например, при симплификации исходного кода, см. далее [глава 4.6]). Опять же, если есть библиотеки, которые предоставляют такую функциональность (Roslyn для .NET, Clang LibTooling для C++), то делать ничего и не нужно. Если же нет, то эту функциональность необходимо создавать самому.

3) Генерация необходимых файлов для сборки. Помимо трансляции исходного кода необходимо также создать необходимые файлы с данными для сборки для кода, полученного в результате трансляции (в тех случаях, когда это нужно). Например, для кода на C# - это должны быть MSBuild файлы, для C++ - CMake или Make файлы, для Java - Maven или Gradle файлы. Без подобных файлов с данными для сборки код, полученный в результате трансляции, собрать будет крайне сложно.

4.3. Неформальная классификация языков и платформ.

Проведем неформальную (полу формальную) классификацию языков, чтобы показать, какие между ними есть отличия и какие могут быть трудности при трансляции с одного языка на другой язык; в том числе и точки зрения сохранения семантики безопасности (см. [глава 3.7]). Языки можно разделить на следующие классы:

4.3.1 Подход к управлению памятью.

По подходу к управлению памятью языки можно разделить на managed и unmanaged языки:

- Managed языки - примером таких языков являются C#, Java, Golang, Python и т.д. (надо сказать, что большинство языков в современном мире - managed языки). В этих языках реализован подход, что освобождением неиспользуемой памяти занимается среда выполнения с помощью такого подхода, как сборка мусора [120]. При таком подходе, большинство проблем с памятью (например, утечки памяти), которые свойственны unmanaged языкам, даже и не возникают. Тем не менее, даже в managed языках можно сделать утечку памяти: для этого, например, мы можем создавать новые объекты и складывать их в список с глобальной областью видимости и никогда из него не удалять объекты; в этом случае потребляемая память будет все время расти, сборщик мусора в освобождении памяти от таких объектов нам не поможет и в какой-то момент времени у нас закончится доступная нам память.
- Unmanaged языки - примером таких языков являются C, C++, Rust. В этих языках временем жизни объектов в памяти (в куче) управляет сам разработчик. Конечно, в более высокоуровневых языках (таких как C++ и Rust) есть средства, упрощающие это управление - такие как, интеллектуальные указатели в C++ (*std::shared_ptr*, *std::weak_ptr*) и специальные типы в Rust (*std::boxed::Box*, *std::rc::Rc*, *std::sync::Arc*). Однако, все эти средства не спасают от того, что разработчик на ровном месте (если он будет не аккуратен) может получить утечку памяти - например, из-за проблемы циклических ссылок [121]. Есть также совершенно отличный от стандартного подхода, подход экосистемы Qt, когда родительский объект отвечает за время жизни дочерних объектов.

Следует сказать, что в managed языках тем не менее есть некоторые средства для ручного управления памятью: так, например, в языке C# можно создавать объекты и вручную освобождать память, занятую ими в unmanaged памяти, есть возможность создавать слабые (weak) ссылки [122]; в языке golang есть возможность создавать “арены” в памяти и вручную управлять временем жизни объектов внутри такой арены; в языке Python есть два механизма управления объектами памяти - сборщик мусора и механизм подсчета ссылок (который используют для целей оптимизации) и т.п.

4.3.2 Статически и динамически типизированные языки.

Один из подходов разделения языков по их типизации - это статически и динамически типизированные языки:

- Статически типизированные языки - примером таких языков является C++, C#, Java, Golang. У языков со статической типизацией тип любого выражения известен во время компиляции и не может быть изменен позже. Статическая типизация дает самый простой машинный код; к тому же многие ошибки исключаются уже на стадии компиляции.
- Динамически типизированные языки - примером таких языков являются Python, JavaScript, PHP. У языков с динамической типизацией отсутствует информация о типе любого выражения во время компиляции. Это приводит к тому, что

программы, созданные на языках с динамической типизацией обычно менее производительны, чем соответствующие программы, созданные на языках со статической типизацией. К тому же, все те проверки, которые делает компилятор языка со статической типизацией, естественно отсутствуют для программы, созданной на языке с динамической типизацией. Поэтому, по-хорошему, эти проверки необходимо реализовать в виде тестов - это увеличивает кодовую базу и время выполнения (закрытия) задач и выпуска новой функциональности (новых версий кода).

Следует также отметить, что в некоторых языках с динамической типизацией появилась возможность аннотировать типы (так, например, в языке Python они называются `type hints`); эти аннотации никак не влияют на выполнение программы и предназначены исключительно для информирования разработчика. Поэтому вполне может произойти такая ситуация, когда эти аннотации типов говорят одно, а на самом деле типы объектов совсем другие.

4.3.3 Языки с сильной и слабой типизацией.

Другой подход деления языков по их типизации - языки с сильной и слабой типизацией:

- Языки с сильной (строгой) типизацией - примером таких языков являются F#, C#, Java, Python. У языков со строгой типизацией установлены достаточно жесткие правила по возможным преобразованиям типов, особенно не явным (В этом плане, одним из самых “строгих” языков программирования является F# - в нем даже расширяющее преобразование типов в большинстве случаев должно быть явным). С другой стороны - это большой плюс таких языков: шаг влево - шаг вправо и уже компилятор не позволяет пользователю создать неожиданный для него результат; если пользователь хочет использовать какое-либо преобразование - он явно должен его использовать.
- Языки со слабой типизацией - примером таких языков являются JScript, PHP, C, C++. В языках со слабой типизацией компилятор (интерпретатор, JIT-компилятор) позволяет автоматически преобразовывать типы данных в широком диапазоне (а не только расширяющее преобразование данных). Так, например, в JScript можно сложить число со строкой `6 + “66”` и получить в результате строку `“666”` (а совсем не ошибку). Языки C и C++ такого, конечно, не позволяют, но у них есть возможность приведения объекта одного типа к объекту любого другого типа с помощью приведения типов языка C, либо оператора `reinterpret_cast` в языке C++ соответственно.

4.3.4 Парадигмы (стили), поддерживаемые языками.

Существует множество парадигм, поддерживаемых теми или иными языками; взглянуть на классификацию языков с этой точки зрения стоит обязательно:

- Процедурная парадигма (процедурный стиль) - примерами таких языков являются C, Pascal, Fortran. Это императивный подход к программированию,

при котором программа разбивается на процедуры и функции (или подпрограммы). Не смотря на все свои минусы, процедурный подход все еще широко используется в программировании, особенно при написании драйверов и скриптов (из-за своей простоты). Следует сказать, что многие современные языки поддерживают процедурную парадигму - к таким языкам относятся C++, Golang, Python; более того, даже такие ООП языки, как C# и Java, фактически поддерживают данную парадигму - с помощью статических методов.

- ООП - примерами таких языков являются C#, Java, C++, Python, Kotlin. Это парадигма разработки, при которой во главу угла ставятся объекты - сущности, объединяющие данные (атрибуты) и код/поведение (методы). Следует, сказать, что это одна из основных парадигм современных языков программирования.
- Языки с неполной поддержкой ООП - примерами таких языков являются JavaScript, Golang, Rust. В JavaScript ООП реализовано через прототипы (прототип - это объект-образец, от которого другие объекты наследуют свойства и методы). В Golang есть частичная реализация ООП - в нем есть композиция типов и интерфейсы (возможность объявлять интерфейсы и их реализовывать), но нет наследования; при этом для интерфейсов язык Golang поддерживает утиную типизацию. В Rust используется система трейтов (trait) для полиморфизма и структуры (struct, impl) для инкапсуляции данных и поведения, но также нет наследования.
- Мультипарадигменные языки - большинство современных языков программирования являются мультипарадигменными (пускай некоторые парадигмы они реализуют и не полностью). Так, например, язык C# является ООП языком, при этом он позволяет писать код в процедурной парадигме (с помощью статических членов и классов), а также реализует многие концепции из функциональной парадигмы (лямбда функции, функции высшего порядка, pattern matching и т.п.); язык Python является ООП языком (с рядом ограничений), при этом он также позволяет писать код в процедурной парадигме, и точно также реализует многие концепции из функциональной парадигмы.

Мы упомянули здесь далеко не все парадигмы, используемые при создании языков программирования: есть еще и функциональная парадигма, и метапрограммирование на этапе компиляции и ряд других парадигм. Все их мы рассматривать не будем; наша основная цель - показать насколько сильно могут различаться языки программирования, что это может повлиять как на сложность трансляции (вне зависимости от того, ручная это трансляция, автоматическая или с помощью LLM), так и на сохранение семантики безопасности при трансляции с одного языка на другой (например, смотри пример с потерей свойства безопасности “инкапсуляция” при трансляции с языка C# на язык Python - [глава 3.5])

4.4 Проблемы при трансляции кода и возможные пути их решения.

После того, как мы сделали неформальную (полу формальную) классификацию языков, давайте рассмотрим примеры проблем (очевидно, что мы рассмотрим далеко

не все возможные проблемы), которые могут возникать при трансляции кода с одного языка на другой и возможные методы их решения. Также мы поговорим о том, какие свойства безопасности могут при этих проблемах теряться.

4.4.1 Трансляция кода с managed языков на unmanaged языки.

Основной проблемой, которая возникает при трансляции кода с managed языка на unmanaged язык - это проблема очистки памяти от неиспользуемых объектов. В managed языках этим занимается сборщик мусора: он обходит граф объектов, начиная с корней (глобальные переменные и переменные в стеке), помечает каждый посещенный объект, пока не обойдет их все; объекты, которые сборщик мусора при этом не обойдет (не пометит) считаются недостижимыми объектами и могут быть удалены. В unmanaged языках ничего такого нет - в них памятью либо вообще управляют вручную (как, например, в C), либо с помощью умных указателей на основе счетчика ссылок (*std::shared_ptr* в C++, *std::rc::Rc* в Rust). Умные указатели сильно упрощают жизнь по сравнению с ручным управлением памятью, но у них есть один фундаментальный недостаток - они просто считают количество ссылок на данный объект и уничтожают его, когда оно равно 0. Соответственно, если два объекта связаны циклической ссылкой через умные указатели (т.е. когда объект А ссылается на объект В, а объект В ссылается на объект А), то эти два объекта никогда не будут удалены, даже если они ниоткуда больше не достижимы - и мы получим классическую утечку памяти. Решение такой проблемы заключается в том, что одна из ссылок должна быть сильной (через *std::shared_ptr*), а другая слабой (через *std::weak_ptr*). Однако правильная расстановка сильных и слабых ссылок совсем не тривиальная задача и далеко не всегда может быть выполнена автоматически. Особенно с учетом того факта, что многие структуры данных подразумевают внутри себя циклические зависимости: двунаправленный связанный список, дерево с ссылками на родителей (когда не только родители знают о своих детях, но и дети знают о своих родителях), граф. Рассмотрим, например, дерево с ссылками на родителей. Кажется, что наиболее естественный подход сделать сильными ссылки от родителей к детям, а ссылки от детей к родителям сделать слабыми. Это будет работать ровно до той поры, пока нам не понадобится метод, который возвращает какой-то узел дерева, и мы захотим из этого узла подниматься вверх к родительским узлам; это будет работать только тогда, когда мы сохраняем ссылку на корень дерева. Однако, на managed языках такое требование не обязательно - мы можем вернуть узел дерева не сохраняя ссылку на корень дерева и потом сможем с помощью этого узла подняться вверх до корня. Поэтому трансляция кода с managed языков на unmanaged языки должно уметь решать такие проблемы. При этом решение будет ручным - мы далеко не всегда можем выбрать в автоматическом режиме, какие ссылки делать сильными, а какие слабыми. Но и расстановка какие ссылки должны быть сильными, а какие слабыми тоже не всегда помогает - в некоторых случаях приходится “привязывать” время жизни родительского объекта ко времени жизни дочернего объекта (как это происходит в managed языках) - очевидно, что это сильно усложняет процедуру трансляции.

Следует также сказать и об альтернативном подходе - использовании внешних (в виде внешних библиотек) сборщиков мусора в unmanaged языках. Например, для C++ есть консервативный сборщик мусора, реализованный в библиотеке Boehm-Demers-Weiser [123]; для Rust есть простой сборщик мусора rust-gc [124] и т.д. Плюсы этого подхода очевидны - код с managed языка на unmanaged язык в вопросах управления объектами переносится один в один без каких-либо ухищрений. Но у этого подхода есть и очевидные минусы. Во-первых, сборка мусора - это всегда не детерминистический процесс в отличие от очистки памяти либо напрямую, либо с помощью умных указателей - мы не можем предсказать, в какой момент времени она произойдет. Во-вторых, все пользователи нашего кода должны быть использовать механизм, связанный со сборкой мусора, в том числе и для создания новых объектов. Это приемлемо, когда мы транслируем приложение; однако, когда мы транслируем библиотеку, то это накладывает слишком жесткие ограничения на ее пользователей.

Очевидно, что если мы транслируем код в unmanaged язык и используем либо обычные, либо умные указатели, то у нас есть потенциальные проблемы при работе с памятью: это либо утечки памяти (из-за проблемы циклических ссылок [121]), либо попытка использования памяти, которая уже удалена (из-за разного времени жизни объектов в исходном коде и коде, полученном в результате трансляции). Соответственно, в данном случае следующие свойства безопасности, связанные с консистентностью памяти могут быть утрачены (при неаккуратной трансляции): отсутствие утечек памяти, контроль за тем, чтобы удаленная память не использовалась и т.д.

Если же мы транслируем код в unmanaged язык и используем подход со сборкой мусора, то таких проблем практически нет (у сборщиков мусора могут быть иногда ложные срабатывания, что приводит к тому, что часть памяти очищается позже, чем это ожидается, но это достаточно редкая ситуация, которая не приводит к утечке памяти). Однако у этого подхода есть свои минусы и его используют крайне редко.

4.4.2 Трансляция кода с unmanaged языков на managed языки.

В данном случае все зависит от того, как происходит работа с памятью в unmanaged языках. Если используются обычные умные указатели, то делать ничего особо не нужно: даже не нужно расставлять сильные и слабые ссылки при формировании графа объектов. Однако, если используются, например, умные указатели с пользовательскими функциями удаления или, например, объекты с пользовательскими деструкторами (например, для реализации RAII [126]), то в данном случае трансляция становится гораздо сложнее, т.к. в managed языках деструкторов либо нет, либо они выполняются совсем не в тот момент времени, как это происходит в unmanaged языках. Однако, в большинстве современных managed языках есть специальные инструменты для очистки ресурсов - такие, как IDisposable в C#, менеджер контекста в Python - их и следует использовать.

Очевидно, что здесь есть потенциальные проблемы с утечками памяти в более сложных случаях, чем простое использование умных указателей и объектов без

деструкторов; это означает, что свойство безопасности “отсутствие утечек памяти” может быть утеряно при неаккуратной трансляции.

4.4.3 Трансляция кода с динамически типизированных языков на языки со статической типизацией.

У языков с динамической типизацией тип выражения неизвестен во время компиляции и становится известен во время выполнения. У языков же со статической типизацией тип каждого выражения должен быть известен на этапе компиляции. Это приводит к значительным трудностям при трансляции кода с языка с динамической типизацией (например, Python) на язык со статической типизацией (например, C#). Вариантов решения этой проблемы два: или использовать специальные аннотации (type hints в Python) для аннотирования типов выражений в тех языках, где они есть, или попытаться самому восстановить возможные типы выражений.

Вариант с использованием специальных аннотаций прост - например, в том же языке Python аннотации типов отображаются в AST. Но у него есть следующие недостатки:

- 1) Никто не гарантирует (как компилятор в языках со статической типизацией), что аннотации типов корректные. Никто не мешает разработчику, на функцию, которая возвращает строку навесить аннотацию типа, что данная функция возвращает число; а компилятор эту ситуацию (в отличие от компилятора для языка со статической типизацией) никак не обрабатывает.
- 2) Никто не гарантирует, что ко всем объявленным функциям, переменным и членам классов будут добавлены аннотации. Никто не мешает разработчику, например, для какой-то из функций вообще не добавлять аннотации; а компилятор эту ситуацию (в отличие от компилятора для языка со статической типизацией) также никак не обрабатывает.
- 3) Обычно никто не добавляет аннотации для переменных и полей классов; поэтому даже если ко всем функциям добавлены полные и корректные аннотации это не решает поставленную задачу.

Вариант с восстановлением возможных типов для всех выражений состоит в следующем: мы с помощью компилятора строим AST; мы можем вычислить для всех листьев, которые соответствуют литералам их типы; мы также знаем типы для аргументов и возвращаемых значений для всех функций/методов базовой библиотеки и всех используемых библиотек. Обладая этой информацией, мы можем обойти дерево, начиная с указанных выше листьев и восстановить возможные типы для всех выражений. Основная проблема в том, что для некоторых выражений невозможно однозначно восстановить его тип - возможно лишь восстановить множество типов, который может принять выражение. Так, например, возвращаемое значение следующей функции на Python

```
def do_something(source):  
    if source == 1:  
        return 666
```

```
if source == 2:
    return "iddqd"
return [13,13]
```

может быть либо целое число, либо строка, либо список. Другими словами, можно сказать, что возвращаемое значение является некоторый алгебраический тип данных (tagged union). Поэтому в языке, на который происходит трансляция кода должен быть некоторый тип для представления такого алгебраического типа - например, в C++ для этого служит тип `std::variant`. При этом код, полученный в результате трансляции, будет сложнее исходного - т.к. он должен поддерживать работу с экземплярами таких алгебраических типов.

Видно, что самая большая проблема при такой трансляции - это либо невозможность полностью восстановить используемые типы в автоматическом режиме (например, когда используется сторонняя библиотека, для которой у нас нет информации о типах/функциях), либо сложность восстановленной системы типов (когда у нас появляется много вариантных типов - алгебраический тип данных). Понятно, что такая ситуация является нормой для динамически типизированных языков программирования (например, для Python) в силу его природы, но для языков со статической типизацией это достаточно большая проблема. Как итог, при такой трансляции может быть утеряно такое свойство безопасности, как “логичность используемой системы типов” и даже, “безопасность системы типов”.

4.4.4 Трансляция кода с языков со статической типизацией на динамически типизированные языки.

Трансляция кода с языков со статической типизацией на динамически типизированные языки обычно не представляет какую-либо проблему, кроме одной - ухудшение безопасности типов (нельзя сказать, что отсутствие, т.к. даже в динамических языках есть какая-то система проверки типов). Выражается это, например, в следующем: в языке со статической типизацией в параметр метода можно передать объект только вполне определенного типа - за этим следит компилятор; а вот в динамически типизированных языках в параметр метода можно передать любой объект - самое главное, чтобы он поддерживал операции, применяемые к нему внутри метода (например, вызовы методов).

Однако, свойство безопасности “безопасность системы типов” обычно является свойством класса L2, т.е. может быть более-менее просто симитировано (по крайней мере, при трансляции с языка C# на язык Python - см. [глава 3.5]). Поэтому это не является особо большой проблемой.

Следует тут добавить следующее, отсутствие инкапсуляции и модификаторов доступа не является свойством динамически типизированных языков: в том же JavaScript есть инкапсуляция, а в PHP есть и инкапсуляция и модификаторы доступа; поэтому проблема отсутствия инкапсуляции и модификаторов доступа - это такой дизайн языка Python.

4.4.5 Трансляция кода с ООП языка на языки с частичной поддержкой ООП.

При трансляции кода с ООП языка на язык с частичной поддержкой ООП (например, при трансляции кода с C# на Golang) может возникнуть ситуация, что часть концепций вообще никак не транслируется и это ухудшает безопасность кода, полученного в результате трансляции, в целом. Так, например, предположим, что исходный код был написан на C#, содержит некоторую иерархию наследования классов и мы транслируем этот код на Golang. При трансляции, конечно, возникнет ряд проблем, т.к. Golang наследования не поддерживает. Для решения этих проблем, мы заменяем иерархию наследования при трансляции композицией, а также объявлением и реализацией интерфейсов. Однако, при таком подходе есть один принципиальный изъян - мы теряем модификаторы доступа. И если члены с публичным доступом (с модификатором доступа `public`) могут быть транслированы как экспортируемые, а члены с приватным доступом (с модификатором доступа `private`) могут быть транслированы (с очень большой натяжкой, т.к. внутри пакета инкапсуляция нарушается), как не экспортируемые, то при трансляции членов с защищенным доступом (с модификатором доступа `protected`) возникают проблемы. Они не могут быть не экспортируемыми, т.к. к ним должен быть доступ у всех производных классов, и они не могут быть экспортируемыми, т.к. к ним не должен быть доступ отовсюду. По итогу получается, что концепция защищенных членов (с модификатором доступа `protected`) отсутствует в коде, полученном в результате трансляции. Это не проблема, когда мы транслируем приложение (т.к. приложение является самодостаточным), но это является проблемой, когда мы транслируем библиотеку, которую будут использовать в других приложениях и библиотеках. Точно такие же проблемы будут, когда мы попытаемся транслировать, например, абстрактные классы: в исходном коде мы не можем создавать экземпляр абстрактного класса, а в коде после трансляции нам никто не помешает это сделать.

Следует также сказать, что подобные проблемы могут возникать и при трансляции между ООП языками. Так, например, в языке Python полностью отсутствуют модификаторы доступа: вместо них используется специальный подход к именованию объектов (имена `protected` членов должны начинаться с одного символа подчеркивания, имена `private` членов - с двух символов подчеркивания) и механизма `name mangling` (искажение имени). Очевидно, что такой подход плохая замена модификаторам доступа.

В данном случае видно, что во многих случаях будут потеряны такие свойства безопасности как “поддержка инкапсуляции” (инкапсуляция теряется не всегда - в языке Rust, например, все члены структуры по умолчанию являются приватными) и “поддержка модификаторов доступа”. Оба эти свойства по классификации являются свойствами класса L3 (см. [глава 3.5]); это означает, что никак отсутствие поддержки этих свойств со стороны языка, на который происходит трансляция, нам не симитировать.

4.4.6 Другие проблемы при трансляции между языками.

Давайте кратко упомянем еще ряд проблем, возникающих при трансляции кода с одного языка на другой. К таким проблемам относятся:

- необходимость трансляции шаблонов и метапрограммирования в C++;
- необходимость трансляции макросов, механизмов заимствования и времени жизни в Rust;
- необходимость трансляции ряда функциональных концепций в языки, в которых их поддержка отсутствует (например, трансляция выражений `pattern matching` с языка C# на язык C++);
- необходимость трансляции асинхронных конструкций в языки, в которых их поддержка отсутствует.

Очевидно, что здесь перечислены не все проблемы. Одни эти проблемы вообще невозможно решить без полной переработки исходного кода, другие - могут приводить к потере ряда свойств безопасности, например, связанных с консистентностью работы с памятью. Мы не будем рассматривать все эти проблемы в рамках этой работы более подробно, но стоит помнить, что подобные (или даже другие) проблемы могут существовать для любой пары языков.

4.4.7 Вывод.

После рассмотрения классификации языков программирования и проблем при трансляции с одного языка на другой можно сделать следующие выводы:

- Чем ближе языки друг к другу относительно их классификации (пусть и неформальной), тем легче производить трансляцию между этими языками. Близость языков друг к другу означает, что у них совпадает как минимум один класс, которому они принадлежат (чем больше, тем лучше). Идеальными языками, трансляцию между которыми достаточно легко реализовать, являются C# и Java: они оба являются managed и ООП языками со статической строгой типизацией и (т.е. совпадают по 4 классам).
- Наоборот, чем дальше языки друг от друга относительно их классификации (пусть и неформальной), тем сложнее производить трансляцию между этими языками. В некоторых случаях, даже разница в одном классе делает задачу трансляции достаточно сложной. Как, пример, трансляция из Python в C#: оба языка являются managed и ООП языками со строгой типизацией, только вот C# - язык со статической типизацией, а Python - динамически типизированный язык.
- При трансляции кода между языками, чем проще конструкция в исходном языке, тем проще ее транслировать. Точно так же, чем некоторая сущность в исходном языке как можно сильнее похожа на некоторую сущность в языке, на который происходит трансляция, тем проще трансляцию этой сущности осуществить (см. далее [глава 4.6]).
- Если осуществить трансляцию некоторой сущности достаточно сложно, то проще не изобретать какой-либо сложный механизм, а либо изменить исходный код, либо при трансляции воспользоваться возможностью ручной кастомизации, например, тела метода. Примером такой сущности является ключевое слово *yield* в C#.

Можно выдвинуть следующую гипотезу: чем больше совпадают языки, между которыми происходит трансляция кода, друг с другом, тем больше свойств безопасности будут принадлежать классу L1 (см. [глава 3.5]); чем дальше языки, между которыми происходит трансляция кода, друг от друга, тем больше свойств будут принадлежать классам L2 и L3 (см. [глава 3.5]).

4.5 Проблема согласования АПИ между языками.

Огромной проблемой при трансляции кода является несовпадение АПИ базовой библиотеки для исходного языка и для языка, на который происходит трансляция.

Проблемы здесь могут быть следующие:

- 1) Несовпадение имен функций, выполняющих одну и ту же задачу - в данном случае решение тривиально.
- 2) Несовпадение количества параметров функций и/или ожидаемых значений - здесь уже все несколько сложнее, т.к. при трансляции кода возможно понадобятся дополнительные шаги по преобразованию данных и/или созданию новых сущностей/объектов.
- 3) Вызов метода в исходном коде транслируется в последовательность шагов в коде, полученном в результате трансляции.
- 4) Аналога функциональности из исходного кода в транслируемом коде нет (очень часто этим грешит C++, у которого стандартная библиотека достаточно бедна). В этом случае необходимо использовать некоторые внешние библиотеки - это усложняет, как саму процедуру трансляции (так как появляются внешние зависимости, которые нужно учитывать), так и увеличивает проблемы с безопасностью (так как внешние библиотеки часто бывают источником уязвимостей, который нужно учитывать и с которым нужно работать).

Решение этих проблем приводит к следующим сложностям при трансляции кода:

- 1) Если в исходном коде задана цепочка вызовов функций - например

```
T result = obj.Method1(...).Method2(...).Method3(...);
```

то далеко не факт, что в транслированном коде это тоже будет такой же по структуре цепочкой вызовов соответствующих функций. Отсюда следует вывод, что перед трансляцией подобную цепочку вызовов лучше всего упрощать (симплифицировать) - в данном случае разбивать; т.е. генерировать следующий код для приведенного выше примера:

```
T1 result1 = obj.Method1();  
T2 result2 = result1.Method2();  
T result = result2.Method3();
```

Несмотря на разбухание кода после такого разбиения и появления новых переменных (особенно, когда цепочка вызовов функций сразу же используется внутри другого выражения), это упростит структуру транслятора и структуру генерируемого кода и

позволит избежать всяческих граничных случаев (например, с появлением некорректно работающего кода после трансляции).

2) Код транслятора, отвечающий за трансляцию АПИ базовой библиотеки будет самой большой частью в приложении для трансляции кода - так, например, если в исходном АПИ базовой библиотеки 1000 функций (что не очень много), то все эти 1000 функций приложение для трансляции кода должно уметь обрабатывать. Соответственно, на такой объем кода так просто не создать модульные тесты и не проверить вручную; это означает, что какое-то время эта часть транслятора будет проверена не самым лучшим образом.

Какие же есть подходы для согласования АПИ между двумя языками:

1) Как уже было написано выше - переложить одну часть работу по преобразованию АПИ на приложение для упрощения (симплификации) кода, а другую часть работы - на приложение для трансляции кода.

2) Сделать библиотеку (паттерн адаптер) для языка, на который происходит трансляция кода, которая бы копировала интерфейс АПИ базовой библиотеки исходного языка один в один. При этом реализовывалась бы вся необходимая функциональность через соответствующее АПИ базовой библиотеки языка, на который происходит трансляция кода. Плюсы этого подхода очевидны - не нужно упрощать код, связанный с цепочкой вызовов функций, и не нужно писать гигантский объем кода в приложении для трансляции кода, который бы преобразовывал вызовы АПИ базовой библиотеки - все вызовы были бы транслированы один в один. Минусы - это гигантская работа, которую необходимо выполнить (с учетом написания тестов и возможно ручного тестирования).

Следует сказать, что при первом подходе код по преобразованию вызовов АПИ базовой библиотеки можно добавлять инкрементально по мере необходимости, если для всех не поддерживаемых случаев генерировать ошибку.

4.6 Проблема сложности трансляции. Симплификация исходного кода как путь упрощения кода транслятора.

Как уже говорилось выше - транслятор, это программа, реализующая фазу трансляции компилятора, в которой на шаге кодогенерации мы вместо генерирования последовательности инструкций, которые могут выполняться на целевой машине, будем генерировать сущности кода (типы, функции, операторы, выражения и т.п.) на некотором языке. Если предположить, что сложности с построением AST у нас нет (например, за нас это делает используемая нами библиотека, как, например, в Roslyn [117]), то основная работа транслятора - это генерация сущностей кода (типы, функции, операторы, выражения и т.п.) на некотором языке на основе полученной AST. Очевидно, что чем больше различных типов узлов AST должен обрабатывать транслятор, тем он будет сложнее (тем больше цикломатическая сложность [126] транслятора). Это также приводит к тому, что такой транслятор крайне сложно как протестировать вручную, так и создать модульные тесты, которые бы покрывали все возможные случаи (все возможные потоки данных в трансляторе). С другой стороны, чем меньше различных типов узлов AST должен обрабатывать транслятор, тем он будет проще (тем больше цикломатическая сложность [126] транслятора); в таком

случае протестировать транслятор (все возможные потоки данных в трансляторе) будет проще: как с помощью ручного тестирования, так и с помощью модульных тестов.

Рассмотрим потенциальную сложность транслятора в случае, когда исходный язык - это C# и используется Roslyn [117] для построения и работы с AST (при этом язык, на который осуществляется трансляция кода - Python). Если скомпилировать и запустить следующий код под .net9:

```
Type[] knownNodeTypes = typeof(CSharpSyntaxNode)
    .Assembly
    .GetTypes()
    .Where(t => !t.IsAbstract &&
typeof(CSharpSyntaxNode).IsAssignableFrom(t))
    .OrderBy(t => t.Name)
    .ToArray();
SyntaxKind[] syntaxKindValues = Enum
    .GetValues(typeof(SyntaxKind))
    .Cast<SyntaxKind>()
    .ToArray();
Console.WriteLine($"Node types count = {knownNodeTypes.Length}");
Console.WriteLine($"SyntaxKind values count =
{syntaxKindValues.Length}");
```

то можно получить следующий результат:

```
Node types count = 241
SyntaxKind values count = 561
```

Из этого можно сделать следующие выводы:

- Количество разных типов узлов AST, определенных в библиотеке Roslyn, равно 241.
- Количество разных значений перечисления *SyntaxKind*, определенных в библиотеке Roslyn, равно 561.
- Перечисление *SyntaxKind* определяет тип данных, содержащийся в узле AST. В Roslyn некоторые узлы могут содержать разные данные, тип которых как раз определяется значением *SyntaxKind*. Так, например, выражение `++i` в AST будет представлено узлом типа *PrefixUnaryExpressionSyntax* со значением типа данных *SyntaxKind.PreIncrementExpression*, а выражение `!flag` в AST будет представлено узлом типа *PrefixUnaryExpressionSyntax* со значением типа данных *SyntaxKind.LogicalNotExpression*. Т.е., в данном случае для обоих выражений используется один и тот же тип узла AST - *PrefixUnaryExpressionSyntax* с разными типами данных *SyntaxKind*.
- Как итог - максимальное количество случаев, которое необходимо уметь обрабатывать в трансляторе - 561. Очевидно, что это достаточно много.

Поэтому базовая идея для упрощения структуры транслятора следующая: давайте мы не будем обрабатывать все возможные случаи - все возможные комбинации типов узлов AST и типов данных *SyntaxKind*. Очевидно, что для этого нам нужен

некоторый автоматизированный подход, т.к. просто договариваться, что мы не используем те или иные конструкции в исходном коде на С# достаточно “хрупкая” идея (хотя и такой подход используется в некоторых командах разработки).

Рассмотрим в качестве примера следующий фрагмент кода на С#, в котором мы создаем объект с помощью object initializer:

```
SomeData data = new SomeData
{
    Value1 = 666,
    Value2 = "IDDQD",
    Value3 = new ComplexData
    {
        Data1 = 777,
        Data2 = "IDKFA"
    }
};
```

Рассмотрим, также, эквивалентный упрощенный код на С#, в котором мы создаем тот же объект без помощи object initializer:

```
SomeData data = new SomeData();
data.Value1 = 666;
data.Value2 = "IDDQD";
data.Value3 = new ComplexData();
data.Value3.Data1 = 777;
data.Value3.Data2 = "IDKFA";
```

Эквивалентный упрощенный фрагмент кода выглядит чуть более многословным, но очевидно, что он намного проще исходного с точки зрения трансляции. Действительно, это фрагмент кода может практически один в один быть транслирован на многие языки, тогда как исходный фрагмент должен быть переработан перед трансляцией. Поэтому, один из подходов к упрощению структуры транслятора: это упрощение исходного кода к максимально простому уровню.

Рассмотрим в качестве другого примера следующее определение цикла **for** в языке С#:

```
for (int index = 0; index < 100; ++index)
{
    SomeProcess(index);
}
```

AST этого цикла **for** (полученное с помощью метода *Dump*) будет выглядеть следующим образом:

```
ForStatement
├─ForKeyword "for"
├─OpenParenToken "("
```

```

├─VariableDeclaration
│   └─PredefinedType
│       └─IntKeyword "int"
│           └─VariableDeclarator
│               └─IdentifierToken "index"
│                   └─EqualsValueClause
│                       └─EqualsToken "="
│                           └─NumericLiteralExpression
│                               └─NumericLiteralToken "0"
├─SemicolonToken ";"
├─LessThanExpression
│   └─IdentifierName "index"
│       └─LessThanToken "<"
│           └─NumericLiteralExpression
│               └─NumericLiteralToken "100"
├─SemicolonToken ";"
├─PreIncrementExpression
│   └─PlusPlusToken "++"
│       └─IdentifierName "index"
├─CloseParenToken ")"
└─Block
    └─OpenBraceToken "{"
        └─ExpressionStatement
            └─InvocationExpression
                └─IdentifierName "SomeProcess"
                    └─ArgumentList
                        └─OpenParenToken "("
                            └─Argument
                                └─IdentifierName "index"
                                    └─CloseParenToken ")"
                        └─SemicolonToken ";"
                    └─CloseBraceToken "}"

```

Давайте рассмотрим реализацию того же самого примера с помощью цикла **while**:

```

int index = 0;
while (index < 100)
{
    SomeProcess(index);
    ++index;
}

```

AST этого цикла **while** (полученное с помощью метода Dump) будет выглядеть следующим образом:

```
LocalDeclarationStatement
├─VariableDeclaration
│   ├──PredefinedType
│   │   └─IntKeyword "int"
│   └─VariableDeclarator
│       ├──IdentifierToken "index"
│       └─EqualsValueClause
│           ├──EqualsToken "="
│           └─NumericLiteralExpression
│               └─NumericLiteralToken "0"
└─SemicolonToken ";"

WhileStatement
├─WhileKeyword "while"
├─OpenParenToken "("
├─LessThanExpression
│   ├──IdentifierName "index"
│   ├──LessThanToken "<"
│   └─NumericLiteralExpression
│       └─NumericLiteralToken "100"
├─CloseParenToken ")"
└─Block
    ├──OpenBraceToken "{"
    ├──ExpressionStatement
    │   ├──InvocationExpression
    │   │   ├──IdentifierName "SomeProcess"
    │   │   └─ArgumentList
    │   │       ├──OpenParenToken "("
    │   │       ├──Argument
    │   │       │   └─IdentifierName "index"
    │   │       └─CloseParenToken ")"
    │   └─SemicolonToken ";"
    ├──ExpressionStatement
    │   ├──PreIncrementExpression
    │   │   ├──PlusPlusToken "++"
    │   │   └─IdentifierName "index"
    │   └─SemicolonToken ";"
    └─CloseBraceToken "}"
```

А теперь давайте рассмотрим реализацию той же самой функциональности на языке Python с помощью цикла **while** (в языке Python есть цикл **for**, но он является аналогом цикла **foreach** языка C#):

```

index = 0
while index < 100:
    some_process(index)
    index += 1

```

AST этого цикла **while** (полученное с помощью модуля *ast*) будет выглядеть следующим образом:

```

Assign(
    targets=[Name(id='index', ctx=Store())],
    value=Constant(value=0)),
While(
    test=Compare(
        left=Name(id='index', ctx=Load()),
        ops=[Lt()],
        comparators=[Constant(value=100)]),
    body=[
        Expr(
            value=Call(
                func=Name(id='some_process', ctx=Load()),
                args=[Name(id='index', ctx=Load())])),
        AugAssign(
            target=Name(id='index', ctx=Store()),
            op=Add(),
            value=Constant(value=1)))]

```

Если сравнить AST решения на языке Python с AST двух эквивалентных решений на языке C#, то легко можно увидеть, что AST решения на Python очень похоже на AST решения на C# с помощью цикла **while**; и менее похоже на AST решения на C# с помощью цикла **for**. Очевидно, что из-за этой похожести, трансляцию на язык Python решения на языке C# с помощью цикла **while** реализовать проще, чем трансляцию решения на языке C# с помощью цикла **for**. Отсюда возникает следующая идея - давайте преобразовывать конструкции исходного языка так, чтобы их AST максимально возможно походила на AST некоторой конструкции языка, на который мы транслируем исходный код.

Давайте также вернемся к функциональным требованиям: когда мы разбирали работу с ними в разрезе процесса транслирования кода [глава 1.9], то упоминали тот факт, что трансляция кода помимо этого должна учитывать изменение поведения тех или иных конструкций языка при транслировании. Так, например, порядок вычисления значений аргументов функций четко определен в языке C# и менее четко определен в языке C++; поэтому при трансляции кода с C# на C++ этот момент надо учитывать, чтобы поведение исходного и транслируемого кода совпадало. Следующие два фрагмента кода на C# полностью эквивалентны:

```

TResult result = SomeMethod(SomeMethod2(), SomeMethod3(),

```

```
SomeMethod4());
```

и

```
TArg1 arg1 = SomeMethod2();  
TArg2 arg2 = SomeMethod3();  
TArg3 arg3 = SomeMethod4();  
TResult result = SomeMethod(arg1, arg2, arg3);
```

Однако, с точки зрения трансляции с языка C# на язык C++ второй фрагмент гораздо более прост - в трансляторе не надо делать преобразования, эквивалентные преобразованию первого фрагмента во второй. Отсюда возникает идея, что преобразования исходного кода, учитывающие возможные изменения в поведении транслируемого кода, также упрощают структуру транслятора.

Как итог - мы можем реализовать следующие преобразования исходного кода, которые упростят трансляцию, структуру транслятора, уменьшат потенциальные ошибки в трансляторе и увеличат покрытие транслятора тестами:

- Преобразование сложных конструкций, аналога которым нет в языке, на который производится трансляция, в более простые конструкции.
- Преобразование конструкций таким образом, чтобы их AST максимально была похожей на AST некоторых конструкций языка, на который производится трансляция.
- Преобразование конструкций с учетом возможного изменения поведения этих конструкций в языке, на который производится трансляция.

Симплификатор - это программа, которые эти преобразования кода на исходном языке реализует. В результате ее работы получается код также на исходном языке, но более простой для процедуры трансляции. Этот код точно также может быть скомпилирован, запущен, в нем могут быть запущены тесты. При этом, если мы применяем симплификатор перед трансляцией, то транслятору нужно обрабатывать далеко не все возможности языка (не все валидные комбинации разных типов узлов AST и разных значений перечисления *SyntaxKind*), что упрощает его структуру и цикломатическую сложность кода.

Здесь мы можем выдвинуть следующую, одну из центральных, гипотез - если симплификатор реализован корректно, то:

- он полностью сохраняет семантику безопасности первоначального кода (никак не меняет Security Fingerprint для исходного кода);
- он уменьшает вероятность ошибки/дефекта в трансляторе из-за упрощения структуры и цикломатической сложности кода транслятора.

Симплификатор можно реализовать так, чтобы каждое упрощение исходного кода было независимо от других. При такой архитектуре, отдельные упрощения можно будет независимо включать и выключать, например, с помощью конфигурационного файла, что повышает его универсализм (возможность использования для разных пар языков перед трансляцией). Например, упрощение для фиксации порядка вычисления аргументов при вызове функций/методов нужно только при трансляции исходного кода на язык C++ и не нужно в других случаях.

4.7 Возможность ручного управления трансляцией. Зачем это нужно и как.

Далеко не все сущности в исходном коде может быть транслированы в автоматическом режиме без вмешательства разработчика - очень часто ему нужно вмешаться в процесс и сконфигурировать процесс транслирования: например, ему нужно поменять имя метода в транслированном коде, указать, какие ссылки (например, при трансляции с кода на C# в код на C++) должны быть сильными, а какие слабыми, какой-то метод или даже фрагмент кода реализовать вручную и т.д. Соответственно, эту информацию нужно как-то передать в программу для транслирования кода. И тут есть несколько возможностей:

1) Добавление метаинформации в исходный код с помощью атрибутов (например, в C#). Плюсы этого подхода в том, что он поддерживается компилятором, легкий, информативный, всегда расположен рядом с сущностью, к которой он применяется. Минусы - не всеми языками поддерживается, в атрибуты можно передать только информацию, вычисляемую во время компиляции и не во всех случаях этот подход применим (например, если мы захотим реализовать часть метода вручную, то мы не сможем с помощью атрибутов сделать пометку внутри кода метода).

2) С помощью комментариев в коде, сформированных особым образом. Плюсы - можно использовать с любым языком и в любом месте (в том числе и в середине метода), всегда расположен с сущностью или с местом (если мы говорим о середине метода), к которому он применяется. Минусы - нужно соблюдать специальный формат (если в нем ошибиться, то не будет распознан), не поддерживается компилятором (для компилятора любой комментарий просто комментарий), для передачи сложных данных необходимо выполнять дополнительные шаги (например, десериализовать данные). Также стоит упомянуть, что обычно обработка комментариев в исходном коде более сложна, чем работа с обычными конструкциями языка.

3) В специальном конфигурационном файле с указанием номера строки сущности (начала сущности, либо начала и конца сущности в случае фрагмента кода). Плюсы - можно использовать с любым языком и для ссылки на любое место; т.к. все подобные определения находятся в одном файле, то легко увидеть общую картину. Минусы - Каждое такое определение находится не рядом с кодом, к которому оно применяется (для того, чтобы понять, применяется ли какое-либо определение к сущности или к блоку кода нужно смотреть в этот конфигурационный файл, что очень неудобно); определения, связанные с номерами строк очень хрупкие - стоит вставить даже пустую строку в файл и определения для этого файла после этой строки сразу же становятся недействительными.

Очевидно, что несмотря на все свои плюсы, подход с использованием конфигурационного файла самый хрупкий, в первую очередь из-за того, что даже случайное изменение исходного файла (например, добавление пустой строки) все может сломать.

4.8 Общая последовательность шагов при трансляции.

Давайте соберем все вместе и посмотрим, как должна выглядеть общая последовательность шагов при трансляции:

1) Анализ исходного кода на предмет наличия свойств безопасности. Мы можем его выполнять либо на основе исходного кода, либо с помощью AST, предварительно построив ее. Очевидно, если проводить такой анализ вручную, то его проще выполнять на основе исходного кода; если использовать какие-либо автоматизированные средства, то такой анализ проще выполнять на основе уже построенной AST. В результате этого анализа у нас будет список свойств безопасности (или Security Fingerprint) для данного кода, который мы собираемся транслировать.

2) Классификация полученных свойств безопасности. Каждому свойству безопасности, полученному на предыдущем шаге (см. [глава 3.5]), мы сопоставляем один из типы (классы) свойств безопасности L1-L4. Здесь класс L1 соответствует свойствам безопасности, которые автоматически сохраняются при трансляции; класс L2 соответствует свойствам безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, но которые мы можем симитировать (с помощью дополнительного кода, генерируемого при трансляции); класс L3 соответствует свойствам безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, и которые мы не можем симитировать (либо их имитация крайне сложна); класс L4 соответствует свойствам безопасности, поддержка которых отсутствует в исходном коде, но которые появляются в коде после трансляции, причем не важно из-за действий транслятора или из-за свойств языка/платформы.

3) После получения свойств безопасности исходного кода и их классификации необходимо принять решение устраивает ли нас то, как изменится набор свойств безопасности кода после его трансляции. А именно: устраивает ли нас, что свойства безопасности, которые принадлежат классу L3 будут утеряны; а также устраивает ли нас цена компенсирующих мер для свойств безопасности, которые принадлежат классу L2. Если ответ нет, то в данном случае мы прекращаем работы по трансляции кода на данном этапе; в данном случае, мы можем только менять исходный код так, чтобы нас устраивало изменение набора свойств безопасности кода после его трансляции (либо мы можем изменить подход к оценке, какие изменения набора свойств безопасности кода после его трансляции нас устраивают). Но даже переработка исходного кода не гарантирует, что удастся избавиться от утери того или иного свойства безопасности (которое принадлежит классу L3); так, например, при трансляции кода с языка C# на язык Python, как бы мы не пытались изменить исходный код на языке C#, мы все равно не сможем избавиться от утери свойство безопасности “инкапсуляция” (т.к. это связано с фундаментальными особенностями языка C#).

4) Упрощение исходный код с помощью симплификатора. В результате получаем код на том же исходном языке, но который стал проще для процедуры трансляции (см. [глава 4.6]). Полученный упрощенный код должен компилироваться; все тесты, которые были в исходном коде должны быть и в коде после симплификации и все эти тесты должны выполняться успешно (как и в исходном коде до симплификации). Если эти условия не выполняются, то мы считаем, что симплификация исходного кода выполняется неудачно - в этом случае необходимо исправлять, либо дорабатывать симплификатор.

5) Проверка гипотезы о том, что симплификатор полностью сохраняет семантику безопасности первоначального кода (никак не меняет Security Fingerprint для исходного кода). Это опциональный шаг, т.е. его выполнять совсем не обязательно. Если же в результате этого шага мы обнаруживаем, что симплификатор полностью не сохраняет семантику безопасности первоначального кода (т.е. происходит изменение Security Fingerprint для исходного кода), то мы считаем, что симплификация исходного кода выполняется неудачно - в этом случае необходимо исправлять, либо дорабатывать симплификатор.

6) Трансляция исходного кода с генерацией компенсирующих механизмов для сохранения свойств безопасности, которые принадлежат классу L2. На данном этапе происходит трансляция исходного кода с одного языка на другой. Во время этой трансляции происходит генерация компенсирующих механизмов для сохранения свойств безопасности, которые принадлежат классу L2. Так, например, если мы транслируем код с языка C# на язык Python и хотим, чтобы свойство безопасности “типобезопасность” было сохранено (это свойство безопасности принадлежит классу L2 - см. [глава 3.5]), то именно в момент трансляции кода с языка C# на язык Python должны быть сгенерированы дополнительные проверки. После трансляции мы проверяем полученный код на предмет того, что он компилируется и все полученные (в результате трансляции) тесты выполняются успешно. Не для всех случаев необходимо проверять, что полученный в результате трансляции код компилируется; например, для случая трансляции кода с языка C# на язык Python, проверка кода, полученного в результате трансляции, на то что он компилируется не нужна, т.к. Python - это преимущественно интерпретируемый язык, хотя при запуске кода на Python происходит Just-In-Time (JIT) компиляция для ускорения работы.

7) Анализ полученного в результате трансляции кода и построение набора свойств безопасности (Security Fingerprint) для него. Также, как и для исходного кода (см. пункт 1), мы должны провести анализ полученного в результате трансляции кода. В результате этого анализа у нас будет список свойств безопасности (или Security Fingerprint) для кода, полученного в результате трансляции.

8) Сравнение набора свойств безопасности (Security Fingerprint) исходного кода и кода полученного в результате трансляции. Мы сопоставляем два набора свойств безопасности (Security Fingerprint), как это описано в главе 3.3 и получаем результат их сравнения для дальнейшего анализа; причем мы можем сравнивать как агрегирующие наборы свойств безопасности (Security Fingerprint), так и детализированные (и по всем результатам сравнения мы можем провести дальнейший анализ).

9) Анализ результатов сравнения набора свойств безопасности (Security Fingerprint) исходного кода и кода полученного в результате трансляции. У нас есть классификация свойств безопасности относительно трансляции: согласно этой классификации свойства безопасности, которые принадлежат классам L1 и L2 должны сохраняться. Соответственно, в первую очередь необходимо проверить именно это. Если это не так, то очевидно, что трансляция кода была выполнена не успешно, т.к. не были сохранены те свойства безопасности, которые должны быть сохранены. Также, с помощью анализа результатов сравнения, мы можем найти свойства безопасности, которые принадлежат классу L4 (см. [глава 3.5]); это свойства безопасности, поддержка которых отсутствует

в исходном коде, но которые появляются в коде после трансляции, причем не важно из-за действий транслятора или из-за свойств языка/платформы. Появление таких свойств безопасности мы обычно просто отслеживаем.

4.9 Корректность и полнота реализации транслятора.

Следует сказать, что когда мы говорим о классификации свойств безопасности (см. [глава 3.5]) относительно трансляции, мы всегда предполагаем, что транслятор реализован корректно и его реализация является полной (например, с языка C# на язык Python). Под корректностью и полнотой реализации понимается следующее:

- 1) Транслятор должен создавать такой код, который удовлетворяет функциональным требованиям и соответствует поведению исходного кода (см. [глава 1.9]).
- 2) Вся функциональность, определенная на исходном языке, может быть обработана и транслирована в соответствующие конструкции на языке, на который происходит трансляция. Этим мы сохраняем свойства безопасности кода, которые принадлежат классу безопасности L1.
- 3) Генерируются соответствующие компенсирующие механизмы (функциональность) для всех потенциальных свойств безопасности, которые принадлежат классу безопасности L2.

Если реализация трансляции в чем-то не является корректной и/или полной, то мы можем прийти к одной (или даже нескольким) из следующих ситуаций:

- 1) Полученный в результате трансляции код не удовлетворяет функциональным требованиям и/или не соответствует поведению исходного кода. В этом случае мы говорим о полном провале трансляции, останавливаемся и идем доделывать транслятор и/или симплификатор (для того, чтобы транслятору некоторые конструкции вообще не попадались - см. [глава 4.6]).
- 2) Полученный в результате трансляции код теряет одно или несколько свойств безопасности, которые принадлежат классу безопасности L1 (но при этом удовлетворяет функциональным требованиям и соответствует поведению исходного кода). Это приводит к тому, что такие свойства безопасности меняют свой класс, которому они принадлежат, с L1 на L3.
- 3) Для одного или нескольких свойств безопасности, которые принадлежат классу безопасности L2, не генерируются соответствующие компенсирующие механизмы (функциональность) при трансляции исходного кода. Это приводит к тому, что такие свойства безопасности меняют свой класс, которому они принадлежат, с L2 на L3.

Таким образом, если у нас ситуация, когда код удовлетворяет функциональным требованиям и соответствует поведению исходного кода, но одно или несколько свойств безопасности меняют свой класс, которому они принадлежат, на класс L3, то мы можем либо временно принять такую деградацию свойств безопасности и работать с полученным транслированным кодом, либо ждать доработок в трансляторе (такая ситуация возможна, например, когда транслятор активно разрабатывается и не вся необходимая функциональность уже реализована).

5. Реализация

5.1 Общие положения, принятые для реализации.

Как уже говорилось выше (см. [глава 1.10]), главная пара языков, на которой сконцентрировано внимание в данной работе - это C# и Python. В рамках данной работы мы главным образом будем изучать трансляцию кода, написанного на языке C#, в код на языке Python, и сохранение свойств безопасности при такой трансляции, а все остальные пары языков используются для примеров при обсуждении теории. Так как, исходным языком является C#, то для создания AST и работы с ней будет использоваться функциональность компилятора Roslyn (см. [117]). Следует отметить, что AST, которую мы создаем с помощью компилятора Roslyn, является неизменяемым объектом, поэтому все попытки ее изменения всегда приводят к созданию новой AST. При реализации функциональности для трансляции кода мы будем использовать Unix-like подход - когда для решения отдельной задачи создается отдельное приложение (отдельная утилита). В рамках данной работы это означает, что симплификатор и транслятор будут двумя разными приложениями (утилитами), каждое из которых при запуске должно будет создавать свой экземпляр AST для выполнения своей работы.

Также следует сказать следующее о процессе сбора свойств безопасности и формировании Security Fingerprint (см. [главы 3.1-3.2]) - в рамках данной работы мы это будем делать вручную, как для исходного кода, так и для кода, полученного в результате трансляции. Кажется, что этот процесс можно автоматизировать на основе получения AST для кода и последующего ее анализа, но это достаточно большая работа, которая заслуживает отдельного внимания.

5.2 Минимальная последовательность шагов для запуска трансляции.

Минимальная последовательность шагов для трансляции кода с языка C# на язык Python имеет следующий вид (см. Рисунок [номер]):

1. Анализ исходного кода на предмет свойств безопасности. Может выполняться как вручную, так и с помощью автоматизированных средств. На данный момент выполняется вручную.
2. Анализ полученного набора свойств безопасности (Security Fingerprint) исходного кода и предсказание итогового набора свойств безопасности (Security Fingerprint) кода после трансляции.
3. Запуск симплификатора. На входе: проект с исходным кодом на языке C#. На выходе: проект с упрощенным кодом на языке C#.
4. Запуск транслятора. На входе: проект с упрощенным кодом на языке C#. На выходе: пакет с транслированным кодом на языке Python.
5. Анализ кода, полученного в результате трансляции, на предмет свойств безопасности. Может выполняться как вручную, так и с помощью автоматизированных средств. На данный момент выполняется вручную.
6. Сравнение ожидаемого набора (полученного на шаге 2) свойств безопасности (Security Fingerprint), для кода после трансляции, с полученным набором свойств

безопасности (Security Fingerprint) в результате трансляции. Принятие решения об успехе или неудаче процесса трансляции.

Здесь будет картинка.

5.3 Максимальная (полная) последовательность шагов для запуска трансляции.

Максимально возможная последовательность шагов для трансляции кода с языка C# на язык Python имеет следующий вид (см. Рисунок [номер]):

1. Компиляция исходного кода, проверка на предупреждения (warnings). Запуск тестов.
2. Запуск инструментов для анализа исходного кода и зависимостей: SAST, DAST, SCA.
3. Запуск утилиты для анализа исходного кода на возможность трансляции (не рассматривается в рамках данной работы, но используется в промышленном решении). Это дает возможность понять возможна ли трансляция еще до фазы самой трансляции. Например, если транслятор с языка C# на язык Python не поддерживает трансляцию ключевого **yield**, а оно есть в исходном коде на C#, то, очевидно, что трансляция будет неудачной. На входе: проект с исходным кодом на языке C#. На выходе: информация о том, возможна ли трансляция или нет.
4. Анализ исходного кода на предмет свойств безопасности. Может выполняться как вручную, так и с помощью автоматизированных средств. На данный момент выполняется вручную.
5. Анализ полученного набора свойств безопасности (Security Fingerprint) исходного кода и предсказание итогового набора свойств безопасности (Security Fingerprint) кода после трансляции.
6. Запуск симплификатора. На входе: проект с исходным кодом на языке C#. На выходе: проект с упрощенным кодом на языке C#.
7. Запуск утилиты для генерации обязательного кода, например, проверок входных аргументов публичных методов, если они отсутствуют (не рассматривается в рамках данной работы, но используется в промышленном решении). На входе: проект с упрощенным кодом на языке C#. На выходе: проект с упрощенным кодом на языке C# с возможными улучшениями кода.
8. Запуск транслятора. На входе: проект с упрощенным кодом на языке C# с возможными улучшениями кода. На выходе: пакет с транслированным кодом на языке Python.
9. Запуск инструментов для анализа кода, полученного в результате трансляции, и его зависимостей: SAST, DAST, SCA.
10. Анализ кода, полученного в результате трансляции, на предмет свойств безопасности. Может выполняться как вручную, так и с помощью автоматизированных средств. На данный момент выполняется вручную.
11. Сравнение ожидаемого набора (полученного на шаге 4) свойств безопасности (Security Fingerprint), для кода после трансляции, с полученным набором свойств безопасности (Security Fingerprint) в результате трансляции. Принятие решения об успехе или неудаче процесса трансляции.

Здесь будет картинка.

5.4 Симплификатор. Детали реализации.

Как уже говорилось выше (см. [глава 4.6]), симплификатор - это программа, которая реализует ряд преобразований кода на исходном языке таким образом, что в итоге получается код также на исходном языке, но более простой для процедуры трансляции. Для этого симплификатор получает AST исходного кода, анализирует ее, модифицирует, если это необходимо и восстанавливает обратно код из AST; если в результате работы симплификатора было выполнено какие-либо изменения AST, эти изменения попадут в код в момент восстановления кода обратно из AST.

Как уже говорилось выше (см. [глава 5.1]), AST, которые мы создаем с помощью компилятора Roslyn, являются неизменяемыми объектами, поэтому любое изменение AST реализуется через создание нового объекта AST на базе AST до изменений и соответственно, самих изменений (реализуется это с помощью функциональности, реализованной в классах *Microsoft.CodeAnalysis.CSharp.CSharpSyntaxRewriter* и *Microsoft.CodeAnalysis Editing.DocumentEditor*). Может возникнуть вопрос, что если мы на каждое изменение пересоздаем AST, то такой подход может занимать очень много времени. Здесь есть одна хитрость: мы можем обрабатывать проект с исходным кодом на C# пофайлово и создавать AST только для текущего обрабатываемого файла (если у нас в коде нет partial-классов - см. [127]); вся остальная информация о типах и методах, которой нет в AST для текущего обрабатываемого файла, получается из информации о семантической модели, которая создается при компиляции проекта. Поэтому получается, что многократные изменения AST для текущего обрабатываемого файла не так уж сильно влияют на общее время работы симплификатора.

Давайте рассмотрим возможные варианты изменений в AST в рамках симплификации и как они влияют на процесс обработки AST при симплификации:

1) Замена некоторого узла в AST на другой узел, который будет принадлежать измененной AST (т.е. в пропорции 1:1). Это самый простой случай, который реализуется с помощью создания класса, родителем которого является класс *Microsoft.CodeAnalysis.CSharp.CSharpSyntaxRewriter*, перегрузки методов, соответствующих типу узлов AST, которые необходимо заменить и возврат из этих методов новых узлов, если замену для данного узла необходимо сделать. С помощью такого подхода можно заменить оператор *nameof* на его значение в виде строкового литерала.

2) Замена одного множества узлов в AST на другое множество узлов в AST (т.е. в пропорции M:N). Это более сложный случай замены, при котором нужно сначала найти исходные узлы, потом создать новые узлы, после чего создать новую AST с заменой исходных узлов на новые с помощью методов класса *Microsoft.CodeAnalysis Editing.DocumentEditor*. С помощью такого подхода можно упростить такую функциональность, как объявление **out** переменной внутри метода. Другими словами преобразовать, например, такой вызов метода (фрагмент кода на языке C#):

```
SomeMethod(value1, value2, out int i);
```

в следующий набор конструкций:

```
int i;  
SomeMethod(value1, value2, out i);
```

В данном примере, нам необходимо преобразовать один узел AST - вызов метода ***SomeMethod*** в два узла AST - объявление переменной ***i*** и вызов метода ***SomeMethod*** (т.е. преобразование узлов в пропорции 1:2).

3) Замены, которые приводят к созданию новых сущностей в AST (типы, члены классов). Основная проблема в этом варианте изменений в том, что информации о семантической модели ничего не знает об этой новой сущности. Поэтому, после создания этой новой сущности необходимо либо пересоздавать семантическую модель (т.е. перекомпилировать проект), либо кэшировать информацию о новых сущностях где-то отдельно. К счастью, изменения этого типа приходится делать редко при симплификации исходного кода.

Симплификатор реализован в виде отдельных, независимых задач по симплификации, которые упрощают отдельные конструкции в исходном коде независимо друг от друга. За счет этого можно гибко конфигурировать симплификатор: включать и выключать отдельные задачи на симплификацию в зависимости от того, какие конструкции и функциональность необходимо упростить (например, в зависимости от языка, на который будет производиться дальнейшая трансляция). Таким образом, симплификатор получается достаточно универсальным инструментом.

На данный момент созданы следующие задачи на симплификацию в рамках симплификатора (Таблица [номер]):

Задача на симплификацию	Описание
Выражения nameof	Заменяет выражение <i>nameof(name)</i> на выражение <i>"name"</i> . Построена на основе подхода 1. По умолчанию включена.
Интерполяция строк	Заменяет выражения интерполяции строк на вызовы метода <i>System.String.Format</i> . Построена на основе подхода 1. По умолчанию выключена (т.к. используется для трансляции кода с языка C# на язык Python для версий млпдше 3.6).
Инициализаторы объектов	Заменяет выражения, использующие инициализаторы объектов более простыми версиями. Построена на основе подхода 2. По умолчанию включена.
Объявление out переменной внутри метода	Выносит объявление out переменной за пределы метода. Построена на основе подхода 2.

	По умолчанию включена.
Null-conditional операторы ?. и ?[]	Заменяет выражения, использующие Null-conditional операторы более простыми версиями. Построена на основе подхода 2. По умолчанию включена.
Преобразователь циклов for в циклы while	Преобразует циклы for (циклы for в языке Python являются аналогом циклов foreach в языке C#) в циклы while . Построена на основе подхода 2. По умолчанию включена.

5.5 Транслятор. Детали реализации.

Как уже говорилось выше (см. [глава 4.2] и [глава 5.1]), транслятор - это программа, которая преобразует код, написанный на языке C#, в код, написанный на языке Python. Для этого транслятор получает AST исходного кода, анализирует ее, после чего на основе результатов анализа генерирует код на языке Python, оформляя его в виде модулей и пакетов на основе имен файлов с исходным кодом на C# и используемых в этом исходном коде пространств имен.

При этом нам приходится решать ряд технических задач, т.к. между языками Python и C# нет полной совместимости. Давайте рассмотрим некоторые из них.

5.5.1 Проблема с перегруженными методами.

В языке C# есть поддержка перегруженных методов; в языке Python такой поддержки нет. Это является серьезной проблемой при реализации транслятора, поэтому ее необходимо уметь решать. Перегруженными называются методы, которые определены в одном классе, имеют одинаковые имена, но разные сигнатуры. Соответственно, с перегруженными методами есть две проблемы, которые необходимо уметь решать: это выбор лучшей (наиболее подходящей) перегрузки и обработка ситуаций, когда выбор между перегруженными методами неоднозначен. Давайте рассмотрим следующий пример на C#:

```
namespace SomeNamespace
{
    public class ClassA
    {
    }
    public class ClassB : ClassA
    {
    }
    public class Program
    {
```



```

    public static void M(String obj)
    {
        Console.WriteLine("In M(String)");
    }
    public static void M(ClassA obj)
    {
        Console.WriteLine("In M(ClassA)");
    }
    public static void M(ClassB obj)
    {
        Console.WriteLine("In M(ClassB)");
    }
    public static void Main(string[] args)
    {
        M("");
        M(new ClassA());
        M(new ClassB());
        //M(null);
    }
}

```

Если мы запустим этот код, то на экране увидим следующее:

```

In M(String)
In M(ClassA)
In M(ClassB)

```

Видно, что для вызова *M("")* компилятор C# выбирает перегруженную версию *M(String)*, для вызова *M(new ClassA())* компилятор C# выбирает перегруженную версию *M(ClassA)*, для вызова *M(new ClassB())* компилятор C# выбирает перегруженную версию *M(ClassB)*, а вот для вызова *M(null)* компилятор C# выбрать перегруженную версию метода *M* не может (именно поэтому этот вызов закомментирован, т.к. в противном случае мы бы получили ошибку компиляции).

Для решения этой проблемы с перегруженными методами на языке Python мы будем делать следующее:

1. Если метод в коде на языке C# не является перегруженным, то мы его транслируем как есть - т.е. с обычными обязательными позиционными аргументами; имена этих аргументов в коде на языке Python совпадают с именами аргументов в исходном коде на языке C# (см. глава [5.5.3]).
2. Если два или три метода в коде на языке C# являются перегруженными, но имеют разные модификаторы доступа (public, protected, private), то из-за изменения имен методов с модификаторами доступа protected и private в языке Python (см. глава [5.5.3]), такие методы перестают быть перегруженными.

3. В остальных случаях перегруженные методы есть. Предположим, что в исходном коде на языке C# у класса *ClassA* есть несколько перегруженных методов с именем *SomeMethod* и одним модификатором доступа - для простоты пусть это будет *public*. В этом случае, в коде на Python у класса *ClassA* мы определим только один метод с именем *SomeMethod*, с переменным количеством позиционных аргументов (*args), а мы будем проверять по этим позиционным аргументам все возможные комбинации, соответствующие перегруженным методам. При этом нам нужно решить две задачи (как говорилось выше): выбор лучшей (наиболее подходящей) перегрузки и обработка ситуаций, когда выбор между перегруженными методами неоднозначен. Первая задача решается с помощью сортировки всех комбинаций типов аргументов, по которым производится поиск, при этом комбинации с более конкретными типами аргументов должны идти раньше комбинации с более общими типами аргументов. Вторая задача - это убедиться, что лучшая (наиболее подходящая) перегрузка только одна. Рассмотрим еще раз приведенный выше пример: для вызова *M(new ClassB())* подходят следующие перегруженные версии функции *M* - *M(ClassA)* и *M(ClassB)*, но компилятор C# выбирает *M(ClassB)*, т.к. она подходит лучше (т.к. тип аргумента наиболее близок к типу значения). Для вызова *M(null)* уже подходят все перегруженные версии функции *M* - *M(String)*, *M(ClassA)* и *M(ClassB)*, причем лучшими (наиболее подходящими) перегрузками являются две - *M(String)*, и *M(ClassB)*; в этом случае компилятор C# выбрать перегруженную версию метода *M* не может. Вторая задача несколько более сложная, чем первая, поэтому для простоты в рамках данной работы мы ее решать не будем (т.к. ее решение нам принципиально ничего нового не даст), хотя в промышленном коде она успешно решается. Поэтому для свойства безопасности “правильная обработка перегруженных методов” значение будет равно “поддерживается частично” (см. таблица [номер]).

Давайте еще раз вернемся к приведенному выше примеру и покажем, как он будет выглядеть после трансляции на язык Python:

```
class ClassA:
    pass
class ClassB(ClassA):
    pass
class Program:
    @staticmethod
    def M(*args):
        if (len(args) == 1) and ((args[0] is None) or
isinstance(args[0], str)):
            print("In M(String)")
            return
        if (len(args) == 1) and ((args[0] is None) or
isinstance(args[0], ClassB)):
            print("In M(ClassB)")
            return
```

```

        if (len(args) == 1) and ((args[0] is None) or
isinstance(args[0], ClassA)):
            print("In M(ClassA)")
            return
        raise ValueError("Unknown combination of arguments")
    @staticmethod
    def Main(args):
        Program.M("")
        Program.M(ClassA())
        Program.M(ClassB())
        Program.M(None)
        #Program.M(666)
if __name__ == "__main__":
    Program.Main([])

```

Если мы запустим этот пример, то мы увидим следующий вывод на экране:

```

In M(String)
In M(ClassA)
In M(ClassB)
In M(String)

```

Видно, что наша реализация на Python для перегруженного метода *M* выбирает лучший (наиболее подходящий) вариант для обработки аргументов в тех случаях, когда неоднозначности нет (точно так же, как это работает и в исходном коде на языке C#); однако, когда есть неоднозначность, в реализации на Python происходит выбор первого подходящего варианта, тогда как в исходном коде на языке C# будет ошибка компиляции (именно поэтому для свойства безопасности “правильная обработка перегруженных методов” значение будет равно “поддерживается частично”). Также стоит обратить внимание на закомментированный вариант вызова метода *M* с неподдерживаемым типом значения (в данном случае - это целое число 666); в этом случае мы получим ошибку при вызове метода *M* (что эквивалентно поведению исходного кода на языке C# - здесь мы получим ошибку времени компиляции).

4. Еще следует сказать о такой проблеме, как перегрузка, например, по разным числовым типам. В языке Python только два числовых типа - **int** для целых чисел и **float** для действительных чисел. Поэтому мы не сможем транслировать с языка C# на язык Python, например, две перегрузки только по целочисленным типам. В данной работе в таком случае мы будем генерировать ошибку во время трансляции, но в промышленном коде можно выбирать, например, перегрузку для самого большого целочисленного типа и транслировать только ее (либо конфигурировать трансляцию таких ситуаций вручную - см. глава [4.7]).

5.5.2 Проблема согласования АПИ между языками.

В данной реализации транслятора мы используем подход с созданием библиотеки (паттерн адаптер) для языка Python (на который происходит трансляция кода), чтобы она копировала интерфейс .NET Framework (базовой библиотеки для языка C#) один в один (см. [глава 4.5]). Однако для ряда типов сделано исключение и вместо использования реализаций из библиотеки адаптера используются нативные типы языка Python. Описание соответствия между типами языка C# и типами языка Python приведено в следующей таблице (Таблица [номер]):

Тип для языка C#	Тип для языка Python
System.Boolean	bool
Все целочисленные числовые типы: System.SByte, System.Byte и т.д.	int
Все типы действительных (вещественных) чисел: System.Single, System.Double и т.д.	float
Строки: System.String	str
Символы: System.Char	str
Массивы	list
System.Object	object
Все остальные типы	Соответствующий тип из библиотеки адаптера.

Так, например, для типа **System.Text.StringBuilder** языка C# будет создан тип **StringBuilder** в пакете **system.text** для языка Python.

5.5.3 Правила именования объектов.

У языка Python есть ключевой документ, регламентирующий правила именования объектов в Python - PEP 8 - Style Guide for Python Code (см. [128]). И в коммерческих системах трансляции кода с языка C# на язык Python мы должны придерживаться этого набора правил. Однако, этот набор правил ничего нам не дает для нашей проблемы сохранения набора свойств безопасности (Security Fingerprint) при трансляции кода с языка C# на язык Python, но при этом усложняет код транслятора, т.к. при этом нам нужно хранить информацию для всего транслируемого проекта о соответствии между именами сущностей для языка C# и именами сущностей для языка Python. Поэтому для транслятора, который применяется в данной работе, мы не будем придерживаться этого набора правил и будем использовать следующий очень простой набор правил, который приведен в следующей таблице (Таблица [номер]):

Имя сущности для языка C#	Имя сущности для языка Python
Пространство имен - например,	Имя пакета в нижнем регистре -

System.Text	например, system.text
Член класса с модификатором доступа protected - например SomeMethod	Член класса, имя которого начинается с одного подчеркивания, после чего следует оригинальное имя без изменений - например, _SomeMethod
Член класса с модификатором доступа private - например SomeMethod	Член класса, имя которого начинается с двух подчеркиваний, после чего следует оригинальное имя без изменений - например, __SomeMethod
Все остальные случаи	Оригинальное имя без изменений

6. Эксперимент

В качестве эксперимента мы будем сравнивать трансляцию кода, приведенного ниже, с языка C# на язык Python с помощью следующих подходов:

- 1) Автоматическая трансляция с помощью симплификатора и транслятора
- 2) Ручная трансляция, как это обычно делается при разработке.
- 3) Автоматическая трансляция с помощью LLM.

6.1 Исходный код.

Исходный код, с которым мы будем работать (проводить эксперимент) выглядит следующим образом (это упрощенная и переработанная часть кода реального проекта для работы с графическими примитивами):

```
namespace ExperimentExample1
{
    public abstract class Step
    {
        protected abstract Boolean Check();
        public abstract String GetRepresentation();
    }

    public class Move : Step
    {
        public Move(Int32 xPosValue, Int32 yPosValue)
        {
            xPos = xPosValue;
            yPos = yPosValue;
        }
        protected override Boolean Check()
        {
```

```

        return (xPos >= 0) && (yPos >= 0);
    }
    public override String GetRepresentation()
    {
        if (!Check())
            throw new InvalidOperationException("Bad data");
        return $"move({xPos},{yPos})";
    }
    private readonly Int32 xPos;
    private readonly Int32 yPos;
}

public class Color : Step
{
    public Color(String colorValue)
    {
        color = colorValue;
    }
    protected override Boolean Check()
    {
        return (color != null) && (color.Length > 0);
    }
    public override String GetRepresentation()
    {
        if (!Check())
            throw new InvalidOperationException("Bad data");
        return $"color({color})";
    }
    private readonly String color;
}

public class Point : Step
{
    public Point(Int32 xPosValue, Int32 yPosValue)
    {
        xPos = xPosValue;
        yPos = yPosValue;
    }
    protected override Boolean Check()
    {
        return (xPos >= 0) && (yPos >= 0);
    }
}

```

```

        public override String GetRepresentation()
        {
            if (!Check())
                throw new InvalidOperationException("Bad data");
            return $"point({xPos},{yPos})";
        }
        private readonly Int32 xPos;
        private readonly Int32 yPos;
    }

    public class Rectangle : Step
    {
        public Rectangle(Int32 xLeftValue, Int32 yTopValue, Int32
xRightValue,Int32 yBottomValue)
        {
            xLeft = xLeftValue;
            yTop = yTopValue;
            xRight = xRightValue;
            yBottom = yBottomValue;
        }
        public Rectangle(Int32 xLeftValue, Int32 yTopValue, Int32
size)
        {
            xLeft = xLeftValue;
            yTop = yTopValue;
            xRight = xLeftValue + size;
            yBottom = yTopValue + size;
        }
        protected override Boolean Check()
        {
            return (xLeft >= 0) && (yTop >= 0) && (xRight >= 0) &&
(yBottom >= 0) && (xRight > xLeft) && (yBottom > yTop);
        }
        public override String GetRepresentation()
        {
            if (!Check())
                throw new InvalidOperationException("Bad data");
            return
$"rectangle({xLeft},{yTop},{xRight},{yBottom})";
        }
        private readonly Int32 xLeft;
        private readonly Int32 yTop;
    }

```

```

        private readonly Int32 xRight;
        private readonly Int32 yBottom;
    }

    public class Circle : Step
    {
        public Circle(Int32 xCenterValue, Int32 yCenterValue,
Int32 radiusValue)
        {
            xCenter = xCenterValue;
            yCenter = yCenterValue;
            radius = radiusValue;
        }
        protected override Boolean Check()
        {
            Int32 minX = xCenter - radius;
            Int32 minY = yCenter - radius;
            return (xCenter >= 0) && (yCenter >= 0) && (radius >
0) && (minX >= 0) && (minY >= 0);
        }
        public override String GetRepresentation()
        {
            if (!Check())
                throw new InvalidOperationException("Bad data");
            return $"circle({xCenter},{yCenter},{radius})";
        }
        private readonly Int32 xCenter;
        private readonly Int32 yCenter;
        private readonly Int32 radius;
    }

    public class StepStorage
    {
        public StepStorage()
        {
            steps = new List<Step>();
        }
        public IList<Step> GetSteps()
        {
            return steps;
        }
        public void AddMove(Move move)
    }

```



```

{
    if (move == null)
        throw new ArgumentNullException("move");
    steps.Add(move);
}
public void AddMove(Int32 x, Int32 y)
{
    steps.Add(new Move(x, y));
}
public void AddColor(Color color)
{
    if (color == null)
        throw new ArgumentNullException("color");
    steps.Add(color);
}
public void AddColor(String color)
{
    if (color == null)
        throw new ArgumentNullException("color");
    steps.Add(new Color(color));
}
public void AddPoint(Point point)
{
    if (point == null)
        throw new ArgumentNullException("point");
    steps.Add(point);
}
public void AddPoint(Int32 x, Int32 y)
{
    steps.Add(new Point(x, y));
}
public void AddRectangle(Rectangle rectangle)
{
    if (rectangle == null)
        throw new ArgumentNullException("rectangle");
    steps.Add(rectangle);
}
public void AddRectangle(Int32 xLeft, Int32 yTop, Int32
xRight, Int32 yBottom)
{
    steps.Add(new Rectangle(xLeft, yTop, xRight,
yBottom));
}

```

```

    }
    public void AddRectangle(Int32 xLeft, Int32 yTop, Int32
size)
    {
        steps.Add(new Rectangle(xLeft, yTop, size));
    }
    public void AddCircle(Circle circle)
    {
        if (circle == null)
            throw new ArgumentNullException("circle");
        steps.Add(circle);
    }
    public void AddCircle(Int32 xCenter, Int32 yCenter, Int32
radius)
    {
        steps.Add(new Circle(xCenter, yCenter, radius));
    }
    private IList<Step> steps;
}

public interface IPainter
{
    void Paint(IList<Step> steps);
}

public class SimplePainter : IPainter
{
    public void Paint(IList<Step> steps)
    {
        if (steps == null)
            throw new ArgumentNullException("steps");
        foreach (Step step in steps)
        {
            Console.WriteLine(step.GetRepresentation());
        }
    }
}

public class Program
{
    public static void Main(String[] args)
    {

```

```

        StepStorage storage = new StepStorage();
        storage.AddMove(3, 5);
        storage.AddColor("red");
        storage.AddPoint(10, 10);
        storage.AddMove(100,100);
        storage.AddRectangle(12,14,55);
        storage.AddCircle(16,16,13);
        IPainter simplePainter = new SimplePainter();
        simplePainter.Paint(storage.GetSteps());
    }
}
}

```

Давайте проведем анализ этого кода и найдем его свойства безопасности (и, сразу классифицируем их - см. [глава 3.5]):

- Проверка входных аргументов; это свойство безопасности принадлежит классу L1.
- Сильная гарантия состояния программы к возникновению исключений; это свойство безопасности принадлежит классу L1.
- Корректная работа с абстрактными типами и методами - невозможность создания объекта абстрактного типа и вызова абстрактного метода; это свойство безопасности принадлежит классу L1, т.к., мы и не можем создать экземпляр абстрактного класса в языке Python и тем самым не можем вызвать абстрактный метод.
- Типобезопасность (т.е. невозможность передачи объектов другого типа в качестве параметра метода или возвращаемого значения); это свойство безопасности принадлежит классу L2 (см. [глава 3.5]).
- Корректная поддержка перегруженных методов; это безопасности принадлежит классу L2, т.к. в самом языке Python перегрузка методов не поддерживается.
- Инкапсуляция и поддержка модификаторов доступа; это свойство безопасности принадлежит классу L3 (см. [глава 3.5]).
- Поддержка readonly полей: это свойство безопасности принадлежит классу L3.

Также, у нас есть следующие функциональные и нефункциональные требования (помимо сохранения свойств безопасности, принадлежащих классам L1 и L2):

- Поведение кода, полученного в результате трансляции, должно полностью совпадать с поведением исходного кода.
- Интерфейс кода (интерфейс типов), полученного в результате трансляции должен по максимуму совпадать с интерфейсом исходного кода. Допускается только изменение имен методов с модификаторами доступа `protected` и `private` в соответствии с правилами языка Python.

Данный исходный код является законченной консольной программой; это было сделано для простоты проверки его работоспособности. В результате работы этого исходного кода мы получаем следующий результат (на который будем ориентироваться при проверке работоспособности кода, полученного в результате трансляции):

```
move(3,5)
color(red)
point(10,10)
move(100,100)
rectangle(12,14,67,69)
circle(16,16,13)
```

6.2 Результат трансляции кода с использованием симплификатора и транслятора.

В первую очередь рассмотрим, что же у нас получается, если транслировать исходный код с помощью симплификатора и транслятора:

```
# -*- coding: utf-8 -*-

import system
import system.collections.generic
import abc

class Step(abc.ABC):
    def __init__(self):
        raise system.InvalidOperationException("Abstract method
call")

    @abc.abstractmethod
    def _Check(self):
        raise system.InvalidOperationException("Abstract method
call")

    @abc.abstractmethod
    def GetRepresentation(self):
        raise system.InvalidOperationException("Abstract method
call")

class Move(Step):
    def __init__(self, xPosValue, yPosValue):
        self.__xPos = 0
        self.__yPos = 0
        if isinstance(xPosValue, int) and isinstance(yPosValue,
int):
            self.__xPos = xPosValue
            self.__yPos = yPosValue
```

```

        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

def _Check(self):
    return (self.__xPos >= 0) and (self.__yPos >= 0)

def GetRepresentation(self):
    if not self._Check():
        raise system.InvalidOperationException("Bad data")
    return f"move({self.__xPos},{self.__yPos})"

class Color(Step):
    def __init__(self, colorValue):
        self.__color = ""
        if (isinstance(colorValue, str) or colorValue is None):
            self.__color = colorValue
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def _Check(self):
        return (self.__color != None) and (len(self.__color) > 0)

    def GetRepresentation(self):
        if not self._Check():
            raise system.InvalidOperationException("Bad data")
        return f"color({self.__color})"

class Point(Step):
    def __init__(self, xPosValue, yPosValue):
        self.__xPos = 0
        self.__yPos = 0
        if isinstance(xPosValue, int) and isinstance(yPosValue,
int):
            self.__xPos = xPosValue
            self.__yPos = yPosValue
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def _Check(self):

```

```

        return (self.__xPos >= 0) and (self.__yPos >= 0)

    def GetRepresentation(self):
        if not self._Check():
            raise system.InvalidOperationException("Bad data")
        return f"point({self.__xPos},{self.__yPos})"

class Rectangle(Step):
    def __init__(self, *args):
        self.__xLeft = 0
        self.__yTop = 0
        self.__xRight = 0
        self.__yBottom = 0
        if (len(args) == 3) and isinstance(args[0], int) and
isinstance(args[1], int) and isinstance(args[2], int):
            xLeftValue = args[0]
            yTopValue = args[1]
            size = args[2]
            self.__xLeft = xLeftValue
            self.__yTop = yTopValue
            self.__xRight = xLeftValue + size
            self.__yBottom = yTopValue + size
        elif (len(args) == 4) and isinstance(args[0], int) and
isinstance(args[1], int) and isinstance(args[2], int) and
isinstance(args[3], int):
            xLeftValue = args[0]
            yTopValue = args[1]
            xRightValue = args[2]
            yBottomValue = args[3]
            self.__xLeft = xLeftValue
            self.__yTop = yTopValue
            self.__xRight = xRightValue
            self.__yBottom = yBottomValue
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def _Check(self):
        return (self.__xLeft >= 0) and (self.__yTop >= 0) and
(self.__xRight >= 0) and (self.__yBottom >= 0) and (self.__xRight
> self.__xLeft) and (self.__yBottom > self.__yTop)

```

```

    def GetRepresentation(self):
        if not self._Check():
            raise system.InvalidOperationException("Bad data")
        return
f"rectangle({self.__xLeft},{self.__yTop},{self.__xRight},{self.__y
Bottom})"

class Circle(Step):
    def __init__(self, xCenterValue, yCenterValue, radiusValue):
        self.__xCenter = 0
        self.__yCenter = 0
        self.__radius = 0
        if isinstance(xCenterValue, int) and
isinstance(yCenterValue, int) and isinstance(radiusValue, int):
            self.__xCenter = xCenterValue
            self.__yCenter = yCenterValue
            self.__radius = radiusValue
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def _Check(self):
        minX = self.__xCenter - self.__radius
        minY = self.__yCenter - self.__radius
        return (self.__xCenter >= 0) and (self.__yCenter >= 0) and
(self.__radius > 0) and (minX >= 0) and (minY >= 0)

    def GetRepresentation(self):
        if not self._Check():
            raise system.InvalidOperationException("Bad data")
        return
f"circle({self.__xCenter},{self.__yCenter},{self.__radius})"

class StepStorage:
    def __init__(self):
        self.__steps = system.collections.generic.List()

    def AddCircle(self, *args):
        if (len(args) == 1) and (isinstance(args[0], Circle) or
args[0] is None):
            circle = args[0]
            if circle == None:

```

```

        raise system.ArgumentNullException("circle")
    self.__steps.Add(circle)
    elif (len(args) == 3) and isinstance(args[0], int) and
isinstance(args[1], int) and isinstance(args[2], int):
        xCenter = args[0]
        yCenter = args[1]
        radius = args[2]
        self.__steps.Add(Circle(xCenter, yCenter, radius))
    else:
        raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def AddColor(self, *args):
        if (len(args) == 1) and (isinstance(args[0], Color) or
args[0] is None):
            color = args[0]
            if color == None:
                raise system.ArgumentNullException("color")
            self.__steps.Add(color)
        elif (len(args) == 1) and (isinstance(args[0], str) or
args[0] is None):
            color = args[0]
            if color == None:
                raise system.ArgumentNullException("color")
            self.__steps.Add(Color(color))
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def AddMove(self, *args):
        if (len(args) == 1) and (isinstance(args[0], Move) or
args[0] is None):
            move = args[0]
            if move == None:
                raise system.ArgumentNullException("move")
            self.__steps.Add(move)
        elif (len(args) == 2) and isinstance(args[0], int) and
isinstance(args[1], int):
            x = args[0]
            y = args[1]
            self.__steps.Add(Move(x, y))
        else:

```



```

        raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def AddPoint(self, *args):
        if (len(args) == 1) and (isinstance(args[0], Point) or
args[0] is None):
            point = args[0]
            if point == None:
                raise system.ArgumentNullException("point")
            self.__steps.Add(point)
        elif (len(args) == 2) and isinstance(args[0], int) and
isinstance(args[1], int):
            x = args[0]
            y = args[1]
            self.__steps.Add(Point(x, y))
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def AddRectangle(self, *args):
        if (len(args) == 1) and (isinstance(args[0], Rectangle) or
args[0] is None):
            rectangle = args[0]
            if rectangle == None:
                raise system.ArgumentNullException("rectangle")
            self.__steps.Add(rectangle)
        elif (len(args) == 3) and isinstance(args[0], int) and
isinstance(args[1], int) and isinstance(args[2], int):
            xLeft = args[0]
            yTop = args[1]
            size = args[2]
            self.__steps.Add(Rectangle(xLeft, yTop, size))
        elif (len(args) == 4) and isinstance(args[0], int) and
isinstance(args[1], int) and isinstance(args[2], int) and
isinstance(args[3], int):
            xLeft = args[0]
            yTop = args[1]
            xRight = args[2]
            yBottom = args[3]
            self.__steps.Add(Rectangle(xLeft, yTop, xRight,
yBottom))
        else:

```

```

        raise system.InvalidOperationException("Unrecognized
combination of arguments")

    def GetSteps(self):
        return self.__steps

class IPainter(abc.ABC):
    @abc.abstractmethod
    def Paint(self, steps):
        if (isinstance(steps, system.collections.generic.ICollection) or
steps is None):
            raise system.InvalidOperationException("Abstract
method call")
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

class SimplePainter(IPainter):
    def __init__(self):
        pass

    def Paint(self, steps):
        if (isinstance(steps, system.collections.generic.ICollection) or
steps is None):
            if steps == None:
                raise system.ArgumentNullException("steps")
            for step in steps:
                system.Console.WriteLine(step.GetRepresentation())
        else:
            raise system.InvalidOperationException("Unrecognized
combination of arguments")

class Program:
    def __init__(self):
        pass

    @staticmethod
    def Main(args):
        if (isinstance(args, list) or args is None):
            storage = StepStorage()
            storage.AddMove(3, 5)
            storage.AddColor("red")

```

```

        storage.AddPoint(10, 10)
        storage.AddMove(100, 100)
        storage.AddRectangle(12, 14, 55)
        storage.AddCircle(16, 16, 13)
        simplePainter = SimplePainter()
        simplePainter.Paint(storage.GetSteps())
    else:
        raise system.InvalidOperationException("Unrecognized
combination of arguments")

if __name__ == "__main__":
    Program.Main([])

```

Если запустить этот результат, то мы получим следующий вывод (который совпадает с выводом исходного кода):

```

move(3,5)
color(red)
point(10,10)
move(100,100)
rectangle(12,14,67,69)
circle(16,16,13)

```

Можно сделать следующие выводы о результате трансляции исходного кода с помощью симплификатора и транслятора:

- Все свойства безопасности, относящиеся к классам L1 и L2, сохраняются.
- Все свойства безопасности, относящиеся к классу L3, теряются. Мы считаем, что для нас это приемлемо (из-за особенностей языка Python).
- Свойств безопасности, относящихся к классу L4, нет.
- Результат выполнения полученного кода совпадает с результатом выполнения исходного кода.
- Интерфейс кода (интерфейс типов), полученного в результате трансляции совпадает с интерфейсом исходного кода, за исключением изменения имен методов с модификаторами доступа `protected` и `private` в соответствии с правилами языка Python.

Таким образом, мы можем считать данную трансляцию кода успешной (см. [глава 3.7]).

6.3 Результат ручной трансляции кода.

Давайте рассмотрим, что же у нас получается, если транслировать исходный код вручную. Следует сказать, что для чистоты эксперимента данная ручная трансляция кода осуществлялась не автором (конечно, для статистической чистоты необходимо, чтобы таких вариантов было несколько и, желательно, как можно больше, но тогда нужно уметь определять, что такое “средний” код среди всех возможных вариантов).

```
# -*- coding: utf-8 -*-

import abc

class Step(abc.ABC):
    @abc.abstractmethod
    def _Check(self):
        pass

    @abc.abstractmethod
    def GetRepresentation(self):
        pass

class Move(Step):
    def __init__(self, xPosValue, yPosValue):
        self.__xPos = xPosValue
        self.__yPos = yPosValue

    def _Check(self):
        return (self.__xPos >= 0) and (self.__yPos >= 0)

    def GetRepresentation(self):
        if not self._Check():
            raise ValueError("Bad data")
        return f"move({self.__xPos},{self.__yPos})"

class Color(Step):
    def __init__(self, colorValue):
        self.__color = colorValue

    def _Check(self):
        return (self.__color != None) and (len(self.__color) > 0)

    def GetRepresentation(self):
        if not self._Check():
            raise system.ValueError("Bad data")
        return f"color({self.__color})"

class Point(Step):
    def __init__(self, xPosValue, yPosValue):
        self.__xPos = xPosValue
```

```

        self.__yPos = yPosValue

    def _Check(self):
        return (self.__xPos >= 0) and (self.__yPos >= 0)

    def GetRepresentation(self):
        if not self._Check():
            raise ValueError("Bad data")
        return f"point({self.__xPos},{self.__yPos})"

class Rectangle(Step):
    def __init__(self, xLeftValue, yTopValue, size=None,
xRightValue=None, yBottomValue=None):
        self.__xLeft = xLeftValue
        self.__yTop = yTopValue
        if size is not None:
            self.__xRight = xLeftValue + size
            self.__yBottom = yTopValue + size
        elif (xRightValue is not None) and (yBottomValue is not
None):
            self.__xRight = xRightValue
            self.__yBottom = yBottomValue
        else:
            raise ValueError("Bad data")

    def _Check(self):
        return (self.__xLeft >= 0) and (self.__yTop >= 0) and
(self.__xRight >= 0) and (self.__yBottom >= 0) and (self.__xRight
> self.__xLeft) and (self.__yBottom > self.__yTop)

    def GetRepresentation(self):
        if not self._Check():
            raise system.InvalidOperationException("Bad data")
        return
f"rectangle({self.__xLeft},{self.__yTop},{self.__xRight},{self.__y
Bottom})"

class Circle(Step):
    def __init__(self, xCenterValue, yCenterValue, radiusValue):
        self.__xCenter = xCenterValue
        self.__yCenter = yCenterValue
        self.__radius = radiusValue

```

```

def _Check(self):
    minX = self.__xCenter - self.__radius
    minY = self.__yCenter - self.__radius
    return (self.__xCenter >= 0) and (self.__yCenter >= 0) and
(self.__radius > 0) and (minX >= 0) and (minY >= 0)

def GetRepresentation(self):
    if not self._Check():
        raise ValueError("Bad data")
    return
f"circle({self.__xCenter},{self.__yCenter},{self.__radius})"

class StepStorage:
    def __init__(self):
        self.__steps = []

    def AddCircle(self, circle=None, xCenter=None, yCenter=None,
radius=None):
        if circle is not None:
            self.__steps.append(circle)
        elif (xCenter is not None) and (yCenter is not None) and
(radius is not None):
            self.__steps.append(Circle(xCenter, yCenter, radius))
        else:
            raise ValueError("Bad data")

    def AddColor(self, color=None, colorString=None):
        if color is not None:
            self.__steps.append(color)
        elif colorString is not None:
            self.__steps.append(Color(colorString))
        else:
            raise ValueError("Bad data")

    def AddMove(self, move=None, x=None, y=None):
        if move is not None:
            self.__steps.append(move)
        elif (x is not None) and (y is not None):
            self.__steps.append(Move(x, y))
        else:
            raise ValueError("Bad data")

```

```

def AddPoint(self, point=None, x=None, y=None):
    if point is not None:
        self.__steps.append(point)
    elif (x is not None) and (y is not None):
        self.__steps.append(Point(x, y))
    else:
        raise ValueError("Bad data")

    def AddRectangle(self, rectangle=None, xLeft=None, yTop=None,
size=None, xRight=None, yBottom=None):
        if rectangle is not None:
            self.__steps.append(rectangle)
        elif (xLeft is not None) and (yTop is not None) and (size
is not None):
            self.__steps.append(Rectangle(xLeft, yTop, size=size))
        elif (xLeft is not None) and (yTop is not None) and
(xRight is not None) and (yBottom is not None):
            self.__steps.append(Rectangle(xLeft, yTop,
xRightValue=xRight, yBottomValue=yBottom))
        else:
            raise ValueError("Bad data")

    def GetSteps(self):
        return self.__steps

class IPainter(abc.ABC):
    @abc.abstractmethod
    def Paint(self, steps):
        pass

class SimplePainter(IPainter):
    def Paint(self, steps):
        for step in steps:
            print(step.GetRepresentation())

class Program:
    @staticmethod
    def Main():
        storage = StepStorage()
        storage.AddMove(x=3, y=5)
        storage.AddColor(colorString="red")

```

```

storage.AddPoint(x=10, y=10)
storage.AddMove(x=100, y=100)
storage.AddRectangle(xLeft=12, yTop=14, size=55)
storage.AddCircle(xCenter=16, yCenter=16, radius=13)
simplePainter = SimplePainter()
simplePainter.Paint(storage.GetSteps())

if __name__ == "__main__":
    Program.Main()

```

Если запустить этот результат, то мы получим следующий вывод (который совпадает с выводом исходного кода):

```

move(3,5)
color(red)
point(10,10)
move(100,100)
rectangle(12,14,67,69)
circle(16,16,13)

```

Можно сделать следующие выводы о результате трансляции исходного кода вручную:

- Все свойства безопасности, относящиеся к классу L1, сохраняются.
- Свойство безопасности типобезопасность, которое принадлежит классу L2, не сохраняется.
- Свойство безопасности корректная поддержка перегруженных методов, которое принадлежит классу L2, поддерживается частично за счет именованных параметров; поведение такого подхода отличается от поведения перегруженных методов в исходном коде. Более того, если использовать обычные параметры вместо именованных (например, по неосторожности), то можно получить неожиданное поведение. Поэтому считаем, что это свойство безопасности не сохраняется.
- Все свойства безопасности, относящиеся к классу L3, теряются. Мы считаем, что для нас это приемлемо (из-за особенностей языка Python).
- Свойств безопасности, относящихся к классу L4, нет.
- Результат выполнения полученного кода совпадает с результатом выполнения исходного кода.
- Интерфейс кода (интерфейс типов), полученного в результате трансляции не совпадает с интерфейсом исходного кода как раз из-за реализации поддержки перегруженных методов.

Таким образом, мы не можем считать данную трансляцию кода успешной, т.к. хотя функциональные требования полностью выполнены, нефункциональные требования выполнены лишь частично, особенно в области безопасности кода (см. [глава 3.7]).

6.4 Результат трансляции кода с помощью LLM.

В первую очередь, следует сделать следующие замечания об использовании и взаимодействии с LLM:

- Будет использоваться только deepseek; никаких дополнительных исследований и сравнений результатов работы разных LLM в рамках данной работы производиться не будет.
- Будет использоваться ограниченное число промтов для LLM; никаких дополнительных исследований и сравнений результатов работы для разных промтов (с целью поиска наилучшего) производиться не будет.
- Периодически LLM может выдавать не работающий код (при запуске которого возникает ошибка парсинга или ошибка времени выполнения); для решения подобных проблем используются уточняющие промпы. Здесь они и промежуточные некорректные результаты не будут приведены.
- Все приведенные ниже варианты при запуске генерируют вывод, совпадающий с выводом исходного кода; т.е. с соблюдением функциональных требований нет проблем у всех вариантов.

Попытка номер 1.

Использовался следующий промпт:

Тебе нужно транслировать следующий код с языка C# на язык Python. Делай так, чтобы транслированный код был по максимуму похож на исходный.
Не используй комментарии в коде.

В результате был получен следующий код:

```
class Step:
    def Check(self):
        raise NotImplementedError()

    def GetRepresentation(self):
        raise NotImplementedError()

class Move(Step):
    def __init__(self, xPosValue, yPosValue):
        self._xPos = xPosValue
        self._yPos = yPosValue

    def Check(self):
        return (self._xPos >= 0) and (self._yPos >= 0)

    def GetRepresentation(self):
        if not self.Check():
            raise InvalidOperationExpection("Bad data")
        return f"move({self._xPos},{self._yPos})"
```

```

class Color(Step):
    def __init__(self, colorValue):
        self._color = colorValue

    def Check(self):
        return (self._color is not None) and (len(self._color) >
0)

    def GetRepresentation(self):
        if not self.Check():
            raise InvalidOperationExpection("Bad data")
        return f"color({self._color})"

class Point(Step):
    def __init__(self, xPosValue, yPosValue):
        self._xPos = xPosValue
        self._yPos = yPosValue

    def Check(self):
        return (self._xPos >= 0) and (self._yPos >= 0)

    def GetRepresentation(self):
        if not self.Check():
            raise InvalidOperationExpection("Bad data")
        return f"point({self._xPos},{self._yPos})"

class Rectangle(Step):
    def __init__(self, xLeftValue, yTopValue, xRightValue,
yBottomValue=None):
        if yBottomValue is None:
            size = xRightValue
            self._xLeft = xLeftValue
            self._yTop = yTopValue
            self._xRight = xLeftValue + size
            self._yBottom = yTopValue + size
        else:
            self._xLeft = xLeftValue
            self._yTop = yTopValue
            self._xRight = xRightValue
            self._yBottom = yBottomValue

```

```

    def Check(self):
        return (self._xLeft >= 0) and (self._yTop >= 0) and
        (self._xRight >= 0) and (self._yBottom >= 0) and (self._xRight >
        self._xLeft) and (self._yBottom > self._yTop)

    def GetRepresentation(self):
        if not self.Check():
            raise InvalidOperationExpection("Bad data")
        return
        f"rectangle({self._xLeft},{self._yTop},{self._xRight},{self._yBottom})"

class Circle(Step):
    def __init__(self, xCenterValue, yCenterValue, radiusValue):
        self._xCenter = xCenterValue
        self._yCenter = yCenterValue
        self._radius = radiusValue

    def Check(self):
        minX = self._xCenter - self._radius
        minY = self._yCenter - self._radius
        return (self._xCenter >= 0) and (self._yCenter >= 0) and
        (self._radius > 0) and (minX >= 0) and (minY >= 0)

    def GetRepresentation(self):
        if not self.Check():
            raise InvalidOperationExpection("Bad data")
        return
        f"circle({self._xCenter},{self._yCenter},{self._radius})"

class StepStorage:
    def __init__(self):
        self._steps = []

    def GetSteps(self):
        return self._steps

    def AddMove(self, move):
        if isinstance(move, Move):
            if move is None:
                raise ArgumentNullException("move")
            self._steps.append(move)

```

```

        else:
            x, y = move
            self._steps.append(Move(x, y))

    def AddColor(self, color):
        if isinstance(color, Color):
            if color is None:
                raise ArgumentNullException("color")
            self._steps.append(color)
        else:
            if color is None:
                raise ArgumentNullException("color")
            self._steps.append(Color(color))

    def AddPoint(self, point):
        if isinstance(point, Point):
            if point is None:
                raise ArgumentNullException("point")
            self._steps.append(point)
        else:
            x, y = point
            self._steps.append(Point(x, y))

    def AddRectangle(self, rectangle):
        if isinstance(rectangle, Rectangle):
            if rectangle is None:
                raise ArgumentNullException("rectangle")
            self._steps.append(rectangle)
        else:
            if len(rectangle) == 3:
                xLeft, yTop, size = rectangle
                self._steps.append(Rectangle(xLeft, yTop, size))
            else:
                xLeft, yTop, xRight, yBottom = rectangle
                self._steps.append(Rectangle(xLeft, yTop, xRight,
yBottom))

    def AddCircle(self, circle):
        if isinstance(circle, Circle):
            if circle is None:
                raise ArgumentNullException("circle")
            self._steps.append(circle)

```

```

        else:
            xCenter, yCenter, radius = circle
            self._steps.append(Circle(xCenter, yCenter, radius))

class IPainter:
    def Paint(self, steps):
        raise NotImplementedError()

class SimplePainter(IPainter):
    def Paint(self, steps):
        if steps is None:
            raise ArgumentNullException("steps")
        for step in steps:
            print(step.GetRepresentation())

class Program:
    @staticmethod
    def Main(args):
        storage = StepStorage()
        storage.AddMove((3, 5))
        storage.AddColor("red")
        storage.AddPoint((10, 10))
        storage.AddMove((100, 100))
        storage.AddRectangle((12, 14, 55))
        storage.AddCircle((16, 16, 13))
        simplePainter = SimplePainter()
        simplePainter.Paint(storage.GetSteps())

if __name__ == "__main__":
    class InvalidOperationException(Exception):
        pass

    class ArgumentNullException(Exception):
        pass

    Program.Main([])

```

Можно сделать следующие выводы о результате трансляции для данного варианта:

- Свойство безопасности проверка входных аргументов, которое принадлежит классу L1, сохраняется.
- Свойство безопасности сильная гарантия состояния программы к возникновению исключений, которое принадлежит классу L1, сохраняется.

- Свойство безопасности корректная работа с абстрактными типами и методами, которое принадлежит классу L1, не сохраняется из-за того, что в результате нет вообще абстрактных типов и методов.
- Свойство безопасности типобезопасность, которое принадлежит классу L2, не сохраняется.
- Свойство безопасности корректная поддержка перегруженных методов, которое принадлежит классу L2, поддерживается частично за счет изменения интерфейса кода. Поэтому считаем, что это свойство безопасности не сохраняется.
- Все свойства безопасности, относящиеся к классу L3, теряются. Мы считаем, что для нас это приемлемо (из-за особенностей языка Python).
- Свойств безопасности, относящихся к классу L4, нет.
- Результат выполнения полученного кода совпадает с результатом выполнения исходного кода.
- Интерфейс кода (интерфейс типов), полученного в результате трансляции не совпадает с интерфейсом исходного кода из-за подхода к реализации поддержки перегруженных методов.
- Некоторые сущности (например, конструктор класса ***Rectangle***) реализованы не самым лучшим способом.

Таким образом, мы не можем считать данную трансляцию кода успешной, т.к. хотя функциональные требования полностью выполнены, нефункциональные требования выполнены лишь частично, особенно в области безопасности кода (см. [глава 3.7]).

Попытка номер 2.

Использовался следующий промпт:

Тебе нужно транслировать следующий код с языка C# на язык Python. Делай так, чтобы транслированный код был по максимуму похож на исходный.

Не используй комментарии в коде.

Есть такое понятие как свойства безопасности в коде.

Мы их можем классифицировать следующим образом относительно операции трансляции кода со следующим обозначением этих типов (классов) свойств безопасности:

L1 - свойства безопасности, которые автоматически сохраняются при трансляции;

L2 - свойства безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, но которые мы можем симитировать (с помощью дополнительного кода, генерируемого при трансляции);

L3 - свойства безопасности, поддержка которых отсутствует в языке/платформе, на который мы транслируем код, и которые мы не можем симитировать (либо их имитация крайне сложна);

L4 - свойства безопасности, поддержка которых отсутствует в исходном коде, но которые появляются в коде после трансляции, причем не важно из-за действий транслятора или из-за свойств языка/платформы. Необходимо, чтобы свойства безопасности, относящиеся к классам L1 и L2 были сохранены при трансляции.

В результате был получен следующий код:

```
import sys
from typing import List

class Step:
    def _check(self) -> bool:
        raise NotImplementedError

    def get_representation(self) -> str:
        raise NotImplementedError

class Move(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        self._x_pos = x_pos_value
        self._y_pos = y_pos_value

    def _check(self) -> bool:
        return (self._x_pos >= 0) and (self._y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationExpection("Bad data")
        return f"move({self._x_pos},{self._y_pos})"

class Color(Step):
    def __init__(self, color_value: str):
        self._color = color_value

    def _check(self) -> bool:
        return (self._color is not None) and (len(self._color) >
0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationExpection("Bad data")
        return f"color({self._color})"
```

```

class Point(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        self._x_pos = x_pos_value
        self._y_pos = y_pos_value

    def _check(self) -> bool:
        return (self._x_pos >= 0) and (self._y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return f"point({self._x_pos},{self._y_pos})"

class Rectangle(Step):
    def __init__(self, x_left_value: int, y_top_value: int,
x_right_value: int = None, y_bottom_value: int = None, size: int =
None):
        if size is not None:
            self._x_left = x_left_value
            self._y_top = y_top_value
            self._x_right = x_left_value + size
            self._y_bottom = y_top_value + size
        else:
            self._x_left = x_left_value
            self._y_top = y_top_value
            self._x_right = x_right_value
            self._y_bottom = y_bottom_value

    def _check(self) -> bool:
        return (self._x_left >= 0) and (self._y_top >= 0) and
(self._x_right >= 0) and (self._y_bottom >= 0) and (self._x_right
> self._x_left) and (self._y_bottom > self._y_top)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return
f"rectangle({self._x_left},{self._y_top},{self._x_right},{self._y_
bottom})"

class Circle(Step):

```



```

    def __init__(self, x_center_value: int, y_center_value: int,
radius_value: int):
        self._x_center = x_center_value
        self._y_center = y_center_value
        self._radius = radius_value

    def _check(self) -> bool:
        min_x = self._x_center - self._radius
        min_y = self._y_center - self._radius
        return (self._x_center >= 0) and (self._y_center >= 0) and
(self._radius > 0) and (min_x >= 0) and (min_y >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationExpection("Bad data")
        return
f"circle({self._x_center},{self._y_center},{self._radius})"

class StepStorage:
    def __init__(self):
        self._steps = []

    def get_steps(self) -> List[Step]:
        return self._steps

    def add_move(self, move):
        if isinstance(move, Move):
            if move is None:
                raise ArgumentNullException("move")
            self._steps.append(move)
        else:
            x, y = move
            self._steps.append(Move(x, y))

    def add_color(self, color):
        if isinstance(color, Color):
            if color is None:
                raise ArgumentNullException("color")
            self._steps.append(color)
        else:
            if color is None:
                raise ArgumentNullException("color")

```

```

        self._steps.append(Color(color))

def add_point(self, point):
    if isinstance(point, Point):
        if point is None:
            raise ArgumentNullException("point")
        self._steps.append(point)
    else:
        x, y = point
        self._steps.append(Point(x, y))

def add_rectangle(self, *args):
    if len(args) == 1:
        rectangle = args[0]
        if rectangle is None:
            raise ArgumentNullException("rectangle")
        self._steps.append(rectangle)
    elif len(args) == 4:
        x_left, y_top, x_right, y_bottom = args
        self._steps.append(Rectangle(x_left, y_top, x_right,
y_bottom))
    elif len(args) == 3:
        x_left, y_top, size = args
        self._steps.append(Rectangle(x_left, y_top,
size=size))

def add_circle(self, circle):
    if isinstance(circle, Circle):
        if circle is None:
            raise ArgumentNullException("circle")
        self._steps.append(circle)
    else:
        x_center, y_center, radius = circle
        self._steps.append(Circle(x_center, y_center, radius))

class IPainter:
    def paint(self, steps: List[Step]) -> None:
        raise NotImplementedError

class SimplePainter(IPainter):
    def paint(self, steps: List[Step]) -> None:
        if steps is None:

```

```

        raise ArgumentNullException("steps")
    for step in steps:
        print(step.get_representation())

class Program:
    @staticmethod
    def main(args: List[str]) -> None:
        storage = StepStorage()
        storage.add_move((3, 5))
        storage.add_color("red")
        storage.add_point((10, 10))
        storage.add_move((100, 100))
        storage.add_rectangle(12, 14, 55)
        storage.add_circle((16, 16, 13))
        simplePainter = SimplePainter()
        simplePainter.paint(storage.get_steps())

class InvalidOperationException(Exception):
    pass

class ArgumentNullException(Exception):
    pass

if __name__ == "__main__":
    Program.main(sys.argv[1:])

```

Можно сделать следующие выводы о результате трансляции для данного варианта:

- Свойство безопасности проверка входных аргументов, которое принадлежит классу L1, сохраняется.
- Свойство безопасности сильная гарантия состояния программы к возникновению исключений, которое принадлежит классу L1, сохраняется.
- Свойство безопасности корректная работа с абстрактными типами и методами, которое принадлежит классу L1, не сохраняется из-за того, что в результате нет вообще абстрактных типов и методов.
- Свойство безопасности типобезопасность, которое принадлежит классу L2, не сохраняется.
- Свойство безопасности корректная поддержка перегруженных методов, которое принадлежит классу L2, поддерживается частично за счет изменения интерфейса кода. Поэтому считаем, что это свойство безопасности не сохраняется.
- Все свойства безопасности, относящиеся к классу L3, теряются. Мы считаем, что для нас это приемлемо (из-за особенностей языка Python).
- Свойств безопасности, относящихся к классу L4, нет.

- Результат выполнения полученного кода совпадает с результатом выполнения исходного кода.
- Интерфейс кода (интерфейс типов), полученного в результате трансляции не совпадает с интерфейсом исходного кода из-за подхода к реализации поддержки перегруженных методов, а также из-за изменения имен всех методов.

Таким образом, мы не можем считать данную трансляцию кода успешной, т.к. хотя функциональные требования полностью выполнены, нефункциональные требования выполнены лишь частично, особенно в области безопасности кода (см. [глава 3.7]).

После нескольких прототипов с уточнениями был получен следующий результат:

```
import sys
from typing import List, Union, Optional

class Step:
    def _check(self) -> bool:
        raise NotImplementedError()

    def get_representation(self) -> str:
        raise NotImplementedError()

class Move(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        self.__x_pos = x_pos_value
        self.__y_pos = y_pos_value

    @property
    def x_pos(self) -> int:
        return self.__x_pos

    @property
    def y_pos(self) -> int:
        return self.__y_pos

    def _check(self) -> bool:
        return (self.__x_pos >= 0) and (self.__y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationExpection("Bad data")
        return f"move({self.__x_pos},{self.__y_pos})"

class Color(Step):
    def __init__(self, color_value: str):
```

```

        if not isinstance(color_value, str):
            raise TypeError("color_value must be str")
        self.__color = color_value

    @property
    def color(self) -> str:
        return self.__color

    def _check(self) -> bool:
        return (self.__color is not None) and (len(self.__color) >
0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationExpection("Bad data")
        return f"color({self.__color})"

class Point(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        if not isinstance(x_pos_value, int) or not
isinstance(y_pos_value, int):
            raise TypeError("x_pos_value and y_pos_value must be
int")
        self.__x_pos = x_pos_value
        self.__y_pos = y_pos_value

    @property
    def x_pos(self) -> int:
        return self.__x_pos

    @property
    def y_pos(self) -> int:
        return self.__y_pos

    def _check(self) -> bool:
        return (self.__x_pos >= 0) and (self.__y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationExpection("Bad data")
        return f"point({self.__x_pos},{self.__y_pos})"

```

```

class Rectangle(Step):
    def __init__(self, x_left_value: int, y_top_value: int,
x_right_value: int = None, y_bottom_value: int = None, size: int =
None):
        if not isinstance(x_left_value, int) or not
isinstance(y_top_value, int):
            raise TypeError("x_left_value and y_top_value must be
int")
        if size is not None:
            if not isinstance(size, int):
                raise TypeError("size must be int")
            self.__x_left = x_left_value
            self.__y_top = y_top_value
            self.__x_right = x_left_value + size
            self.__y_bottom = y_top_value + size
        elif x_right_value is not None and y_bottom_value is not
None:
            if not isinstance(x_right_value, int) or not
isinstance(y_bottom_value, int):
                raise TypeError("x_right_value and y_bottom_value
must be int")
            self.__x_left = x_left_value
            self.__y_top = y_top_value
            self.__x_right = x_right_value
            self.__y_bottom = y_bottom_value
        else:
            raise ValueError("Invalid constructor arguments")

    @property
    def x_left(self) -> int:
        return self.__x_left

    @property
    def y_top(self) -> int:
        return self.__y_top

    @property
    def x_right(self) -> int:
        return self.__x_right

    @property
    def y_bottom(self) -> int:

```

```

        return self.__y_bottom

    def _check(self) -> bool:
        return (self.__x_left >= 0) and (self.__y_top >= 0) and
            (self.__x_right >= 0) and (self.__y_bottom >= 0) and
            (self.__x_right > self.__x_left) and (self.__y_bottom >
            self.__y_top)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return
        f"rectangle({self.__x_left},{self.__y_top},{self.__x_right},{self.
        __y_bottom})"

class Circle(Step):
    def __init__(self, x_center_value: int, y_center_value: int,
radius_value: int):
        if not isinstance(x_center_value, int) or not
isinstance(y_center_value, int) or not isinstance(radius_value,
int):
            raise TypeError("x_center_value, y_center_value,
radius_value must be int")
        self.__x_center = x_center_value
        self.__y_center = y_center_value
        self.__radius = radius_value

    @property
    def x_center(self) -> int:
        return self.__x_center

    @property
    def y_center(self) -> int:
        return self.__y_center

    @property
    def radius(self) -> int:
        return self.__radius

    def _check(self) -> bool:
        min_x = self.__x_center - self.__radius
        min_y = self.__y_center - self.__radius

```

```

        return (self.__x_center >= 0) and (self.__y_center >= 0)
and (self.__radius > 0) and (min_x >= 0) and (min_y >= 0)

def get_representation(self) -> str:
    if not self._check():
        raise InvalidOperationException("Bad data")
    return
f"circle({self.__x_center},{self.__y_center},{self.__radius})"

class StepStorage:
    def __init__(self):
        self.__steps = []

    def get_steps(self) -> List[Step]:
        return self.__steps

    def add_move(self, param: Union[Move, int, tuple]) -> None:
        if isinstance(param, Move):
            if param is None:
                raise ArgumentNullException("move")
            self.__steps.append(param)
        elif isinstance(param, tuple) and len(param) == 2:
            x, y = param
            if not isinstance(x, int) or not isinstance(y, int):
                raise TypeError("x and y must be int")
            self.__steps.append(Move(x, y))
        else:
            raise TypeError("Invalid parameter type for add_move")

    def add_color(self, param: Union[Color, str]) -> None:
        if isinstance(param, Color):
            if param is None:
                raise ArgumentNullException("color")
            self.__steps.append(param)
        elif isinstance(param, str):
            if param is None:
                raise ArgumentNullException("color")
            self.__steps.append(Color(param))
        else:
            raise TypeError("Invalid parameter type for
add_color")

```



```

def add_point(self, param: Union[Point, int, tuple]) -> None:
    if isinstance(param, Point):
        if param is None:
            raise ArgumentNullException("point")
        self.__steps.append(param)
    elif isinstance(param, tuple) and len(param) == 2:
        x, y = param
        if not isinstance(x, int) or not isinstance(y, int):
            raise TypeError("x and y must be int")
        self.__steps.append(Point(x, y))
    else:
        raise TypeError("Invalid parameter type for
add_point")

def add_rectangle(self, *args) -> None:
    if len(args) == 1:
        rectangle = args[0]
        if rectangle is None:
            raise ArgumentNullException("rectangle")
        if not isinstance(rectangle, Rectangle):
            raise TypeError("Expected Rectangle object")
        self.__steps.append(rectangle)
    elif len(args) == 4:
        x_left, y_top, x_right, y_bottom = args
        if not all(isinstance(arg, int) for arg in args):
            raise TypeError("All arguments must be int")
        self.__steps.append(Rectangle(x_left, y_top, x_right,
y_bottom))
    elif len(args) == 3:
        x_left, y_top, size = args
        if not all(isinstance(arg, int) for arg in args):
            raise TypeError("All arguments must be int")
        self.__steps.append(Rectangle(x_left, y_top,
size=size))
    else:
        raise TypeError("Invalid number of arguments for
add_rectangle")

def add_circle(self, param: Union[Circle, int, tuple]) ->
None:
    if isinstance(param, Circle):
        if param is None:

```

```

        raise ArgumentNullException("circle")
    self.__steps.append(param)
elif isinstance(param, tuple) and len(param) == 3:
    x_center, y_center, radius = param
    if not all(isinstance(arg, int) for arg in param):
        raise TypeError("x_center, y_center, radius must
be int")
    self.__steps.append(Circle(x_center, y_center,
radius))
else:
    raise TypeError("Invalid parameter type for
add_circle")

class IPainter:
    def paint(self, steps: List[Step]) -> None:
        raise NotImplementedError()

class SimplePainter(IPainter):
    def paint(self, steps: List[Step]) -> None:
        if steps is None:
            raise ArgumentNullException("steps")
        if not isinstance(steps, list):
            raise TypeError("steps must be list")
        for step in steps:
            if not isinstance(step, Step):
                raise TypeError("Each element in steps must be
Step instance")
            print(step.get_representation())

class Program:
    @staticmethod
    def main(args: List[str]) -> None:
        storage = StepStorage()
        storage.add_move((3, 5))
        storage.add_color("red")
        storage.add_point((10, 10))
        storage.add_move((100, 100))
        storage.add_rectangle(12, 14, 55)
        storage.add_circle((16, 16, 13))
        simplePainter = SimplePainter()
        simplePainter.paint(storage.get_steps())

```

```

class InvalidOperationException(Exception):
    pass

class ArgumentNullException(Exception):
    pass

if __name__ == "__main__":
    Program.main(sys.argv[1:])

```

Также LLM выдала следующее заключение о найденных свойствах безопасности и их классах :

Сохранённые L1 свойства:
 Инкапсуляция (двойное подчёркивание `__` для `private` полей)
 Наследование (Step как базовый класс)
 Проверка на `null` (`ArgumentNullException`)
 Логика проверки через метод `_check()`
 Абстрактные методы
 Сохранённые L2 свойства:
 Protected методы → метод `_check()` с одним подчёркиванием (конвенция `protected`)
 Readonly поля → `property` без `setter`, приватные поля `__field`
 Перегрузка методов → проверка типов через `isinstance()` и `*args`
 Строгая типизация → runtime проверки типов через `isinstance()` и `TypeError`
 Приватные поля → `name mangling` с `__` для эмуляции `private`

Можно сделать следующие выводы о результате трансляции для данного варианта:

- Не все заключения о найденных свойствах и/или их классах корректны: например, свойства безопасности инкапсуляция и `protected` методы принадлежат классу L3 (см [глава 3.5]), наследование вообще не является свойством безопасности, поддержка `readonly` полей является свойством безопасности, которое принадлежит классу L3.
- Свойство безопасности проверка входных аргументов, которое принадлежит классу L1, сохраняется.
- Свойство безопасности сильная гарантия состояния программы к возникновению исключений, которое принадлежит классу L1, сохраняется.
- Свойство безопасности корректная работа с абстрактными типами и методами, которое принадлежит классу L1, не сохраняется из-за того, что в результате нет вообще абстрактных типов и методов.
- Свойство безопасности типобезопасность, которое принадлежит классу L2, частично сохраняется; например, в конструкторе класса `Move` нет проверок на соответствие аргументов ожидаемым типам.
- Свойство безопасности корректная поддержка перегруженных методов, которое принадлежит классу L2, поддерживается частично за счет изменения

интерфейса кода. Поэтому считаем, что это свойство безопасности не сохраняется.

- Все свойства безопасности, относящиеся к классу L3, теряются. Мы считаем, что для нас это приемлемо (из-за особенностей языка Python).
- Свойств безопасности, относящихся к классу L4, нет.
- Результат выполнения полученного кода совпадает с результатом выполнения исходного кода.
- Интерфейс кода (интерфейс типов), полученного в результате трансляции не совпадает с интерфейсом исходного кода из-за подхода к реализации поддержки перегруженных методов, а также из-за изменения имен всех методов.

Таким образом, мы не можем считать данную трансляцию кода успешной, т.к. хотя функциональные требования полностью выполнены, нефункциональные требования выполнены лишь частично (см. [глава 3.7]). Следует сказать, что результат этой трансляции уже сильно близок к результату, который мы ожидаем.

Попытка номер 3.

Использовался следующий промпт:

Тебе нужно транслировать следующий код с языка C# на язык Python. Делай так, чтобы транслированный код был по максимуму похож на исходный.

Не используй комментарии в коде.

Есть такое понятие как свойства безопасности в коде.

В данном коде есть следующие свойства безопасности, которые необходимо сохранить: проверка входных аргументов
сильная гарантия состояния программы к возникновению исключений
типобезопасность

корректная работа перегруженных методов

корректная работа абстрактных типов и методов

В результате был получен следующий код:

```
import sys
from typing import List

class Step:
    def _check(self) -> bool:
        raise NotImplementedError

    def get_representation(self) -> str:
        raise NotImplementedError

class Move(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        self._x_pos = x_pos_value
```

```

        self._y_pos = y_pos_value

    def _check(self) -> bool:
        return (self._x_pos >= 0) and (self._y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return f"move({self._x_pos},{self._y_pos})"

class Color(Step):
    def __init__(self, color_value: str):
        self._color = color_value

    def _check(self) -> bool:
        return (self._color is not None) and (len(self._color) >
0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return f"color({self._color})"

class Point(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        self._x_pos = x_pos_value
        self._y_pos = y_pos_value

    def _check(self) -> bool:
        return (self._x_pos >= 0) and (self._y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return f"point({self._x_pos},{self._y_pos})"

class Rectangle(Step):
    def __init__(self, x_left_value: int, y_top_value: int,
x_right_value: int = None, y_bottom_value: int = None, size: int =
None):
        if x_right_value is not None and y_bottom_value is not
None:

```

```

        self._x_left = x_left_value
        self._y_top = y_top_value
        self._x_right = x_right_value
        self._y_bottom = y_bottom_value
    elif size is not None:
        self._x_left = x_left_value
        self._y_top = y_top_value
        self._x_right = x_left_value + size
        self._y_bottom = y_top_value + size
    else:
        raise ArgumentException("Invalid constructor
parameters")

    def _check(self) -> bool:
        return (self._x_left >= 0) and (self._y_top >= 0) and
        (self._x_right >= 0) and (self._y_bottom >= 0) and (self._x_right
        > self._x_left) and (self._y_bottom > self._y_top)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return
        f"rectangle({self._x_left},{self._y_top},{self._x_right},{self._y_
        bottom})"

class Circle(Step):
    def __init__(self, x_center_value: int, y_center_value: int,
radius_value: int):
        self._x_center = x_center_value
        self._y_center = y_center_value
        self._radius = radius_value

    def _check(self) -> bool:
        min_x = self._x_center - self._radius
        min_y = self._y_center - self._radius
        return (self._x_center >= 0) and (self._y_center >= 0) and
        (self._radius > 0) and (min_x >= 0) and (min_y >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return

```

```

f"circle({self._x_center},{self._y_center},{self._radius})"

class StepStorage:
    def __init__(self):
        self._steps: List[Step] = []

    def get_steps(self) -> List[Step]:
        return self._steps

    def add_move(self, move_or_x, y=None):
        if isinstance(move_or_x, Move):
            if move_or_x is None:
                raise ArgumentNullException("move")
            self._steps.append(move_or_x)
        elif isinstance(move_or_x, int) and isinstance(y, int):
            self._steps.append(Move(move_or_x, y))
        else:
            raise ArgumentException("Invalid arguments")

    def add_color(self, color_or_value):
        if isinstance(color_or_value, Color):
            if color_or_value is None:
                raise ArgumentNullException("color")
            self._steps.append(color_or_value)
        elif isinstance(color_or_value, str):
            if color_or_value is None:
                raise ArgumentNullException("color")
            self._steps.append(Color(color_or_value))
        else:
            raise ArgumentException("Invalid arguments")

    def add_point(self, point_or_x, y=None):
        if isinstance(point_or_x, Point):
            if point_or_x is None:
                raise ArgumentNullException("point")
            self._steps.append(point_or_x)
        elif isinstance(point_or_x, int) and isinstance(y, int):
            self._steps.append(Point(point_or_x, y))
        else:
            raise ArgumentException("Invalid arguments")

    def add_rectangle(self, rectangle_or_x_left, y_top=None,

```

```

x_right=None, y_bottom=None, size=None):
    if isinstance(rectangle_or_x_left, Rectangle):
        if rectangle_or_x_left is None:
            raise ArgumentNullException("rectangle")
        self._steps.append(rectangle_or_x_left)
    elif isinstance(rectangle_or_x_left, int) and
isinstance(y_top, int) and isinstance(x_right, int) and
isinstance(y_bottom, int):
        self._steps.append(Rectangle(rectangle_or_x_left,
y_top, x_right, y_bottom))
    elif isinstance(rectangle_or_x_left, int) and
isinstance(y_top, int) and isinstance(size, int):
        self._steps.append(Rectangle(rectangle_or_x_left,
y_top, size=size))
    else:
        raise ArgumentException("Invalid arguments")

    def add_circle(self, circle_or_x_center, y_center=None,
radius=None):
        if isinstance(circle_or_x_center, Circle):
            if circle_or_x_center is None:
                raise ArgumentNullException("circle")
            self._steps.append(circle_or_x_center)
        elif isinstance(circle_or_x_center, int) and
isinstance(y_center, int) and isinstance(radius, int):
            self._steps.append(Circle(circle_or_x_center,
y_center, radius))
        else:
            raise ArgumentException("Invalid arguments")

class IPainter:
    def paint(self, steps: List[Step]) -> None:
        raise NotImplementedError

class SimplePainter(IPainter):
    def paint(self, steps: List[Step]) -> None:
        if steps is None:
            raise ArgumentNullException("steps")
        for step in steps:
            print(step.get_representation())

class ArgumentNullException(Exception):

```



```

pass

class ArgumentException(Exception):
    pass

class InvalidOperationException(Exception):
    pass

class Program:
    @staticmethod
    def main(args: List[str]) -> None:
        storage = StepStorage()
        storage.add_move(3, 5)
        storage.add_color("red")
        storage.add_point(10, 10)
        storage.add_move(100, 100)
        storage.add_rectangle(12, 14, size=55)
        storage.add_circle(16, 16, 13)
        simplePainter = SimplePainter()
        simplePainter.paint(storage.get_steps())

if __name__ == "__main__":
    Program.main(sys.argv)

```

Можно сделать следующие выводы о результате трансляции для данного варианта:

- Свойство безопасности проверка входных аргументов, которое принадлежит классу L1, сохраняется.
- Свойство безопасности сильная гарантия состояния программы к возникновению исключений, которое принадлежит классу L1, сохраняется.
- Свойство безопасности корректная работа с абстрактными типами и методами, которое принадлежит классу L1, не сохраняется из-за того, что в результате нет вообще абстрактных типов и методов.
- Свойство безопасности типобезопасность, которое принадлежит классу L2, не сохраняется.
- Свойство безопасности корректная поддержка перегруженных методов, которое принадлежит классу L2, поддерживается частично за счет изменения интерфейса кода. Поэтому считаем, что это свойство безопасности не сохраняется.
- Все свойства безопасности, относящиеся к классу L3, теряются. Мы считаем, что для нас это приемлемо (из-за особенностей языка Python).
- Свойств безопасности, относящихся к классу L4, нет.
- Результат выполнения полученного кода совпадает с результатом выполнения исходного кода.

- Интерфейс кода (интерфейс типов), полученного в результате трансляции не совпадает с интерфейсом исходного кода из-за подхода к реализации поддержки перегруженных методов, а также из-за изменения имен всех методов.
- Странная попытка использовать один и тот же аргумент функции для параметров разных типов, в результате чего появляются аргументы с подобными именами: *move_or_x*

Таким образом, мы не можем считать данную трансляцию кода успешной, т.к. хотя функциональные требования полностью выполнены, нефункциональные требования выполнены лишь частично, особенно в области безопасности кода (см. [глава 3.7]).

После нескольких промов с уточнениями был получен следующий результат:

```
from typing import List
from abc import ABC, abstractmethod

class Step(ABC):
    @abstractmethod
    def _check(self) -> bool:
        pass

    @abstractmethod
    def get_representation(self) -> str:
        pass

class Move(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        if not isinstance(x_pos_value, int) or not
isinstance(y_pos_value, int):
            raise TypeError("Arguments must be of type int")
        self._x_pos = x_pos_value
        self._y_pos = y_pos_value

    def _check(self) -> bool:
        return (self._x_pos >= 0) and (self._y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationExpection("Bad data")
        return f"move({self._x_pos},{self._y_pos})"

class Color(Step):
    def __init__(self, color_value: str):
        if not isinstance(color_value, str):
            raise TypeError("Argument must be of type str")
```

```

        self._color = color_value

    def _check(self) -> bool:
        return (self._color is not None) and (len(self._color) >
0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return f"color({self._color})"

class Point(Step):
    def __init__(self, x_pos_value: int, y_pos_value: int):
        if not isinstance(x_pos_value, int) or not
isinstance(y_pos_value, int):
            raise TypeError("Arguments must be of type int")
        self._x_pos = x_pos_value
        self._y_pos = y_pos_value

    def _check(self) -> bool:
        return (self._x_pos >= 0) and (self._y_pos >= 0)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return f"point({self._x_pos},{self._y_pos})"

class Rectangle(Step):
    def __init__(self, x_left_value: int, y_top_value: int,
x_right_value: int = None, y_bottom_value: int = None, size: int =
None):
        if not isinstance(x_left_value, int) or not
isinstance(y_top_value, int):
            raise TypeError("Arguments must be of type int")
        if x_right_value is not None and y_bottom_value is not
None:
            if not isinstance(x_right_value, int) or not
isinstance(y_bottom_value, int):
                raise TypeError("Arguments must be of type int")
            self._x_left = x_left_value
            self._y_top = y_top_value
            self._x_right = x_right_value

```

```

        self._y_bottom = y_bottom_value
    elif size is not None:
        if not isinstance(size, int):
            raise TypeError("Argument must be of type int")
        self._x_left = x_left_value
        self._y_top = y_top_value
        self._x_right = x_left_value + size
        self._y_bottom = y_top_value + size
    else:
        raise ArgumentException("Invalid constructor
parameters")

    def _check(self) -> bool:
        return (self._x_left >= 0) and (self._y_top >= 0) and
(self._x_right >= 0) and (self._y_bottom >= 0) and (self._x_right
> self._x_left) and (self._y_bottom > self._y_top)

    def get_representation(self) -> str:
        if not self._check():
            raise InvalidOperationException("Bad data")
        return
f"rectangle({self._x_left},{self._y_top},{self._x_right},{self._y_
bottom})"

class Circle(Step):
    def __init__(self, x_center_value: int, y_center_value: int,
radius_value: int):
        if not isinstance(x_center_value, int) or not
isinstance(y_center_value, int) or not isinstance(radius_value,
int):
            raise TypeError("Arguments must be of type int")
        self._x_center = x_center_value
        self._y_center = y_center_value
        self._radius = radius_value

    def _check(self) -> bool:
        min_x = self._x_center - self._radius
        min_y = self._y_center - self._radius
        return (self._x_center >= 0) and (self._y_center >= 0) and
(self._radius > 0) and (min_x >= 0) and (min_y >= 0)

    def get_representation(self) -> str:

```

```

        if not self._check():
            raise InvalidOperationException("Bad data")
        return
f"circle({self._x_center},{self._y_center},{self._radius})"

class StepStorage:
    def __init__(self):
        self._steps: List[Step] = []

    def get_steps(self) -> List[Step]:
        return self._steps

    def add_move(self, move_or_x, y=None):
        if isinstance(move_or_x, Move):
            if move_or_x is None:
                raise ArgumentNullException("move")
            if not isinstance(move_or_x, Move):
                raise TypeError("Expected Move type")
            self._steps.append(move_or_x)
        elif isinstance(move_or_x, int) and isinstance(y, int):
            self._steps.append(Move(move_or_x, y))
        else:
            raise ArgumentException("Invalid arguments")

    def add_color(self, color_or_value):
        if isinstance(color_or_value, Color):
            if color_or_value is None:
                raise ArgumentNullException("color")
            if not isinstance(color_or_value, Color):
                raise TypeError("Expected Color type")
            self._steps.append(color_or_value)
        elif isinstance(color_or_value, str):
            if color_or_value is None:
                raise ArgumentNullException("color")
            self._steps.append(Color(color_or_value))
        else:
            raise ArgumentException("Invalid arguments")

    def add_point(self, point_or_x, y=None):
        if isinstance(point_or_x, Point):
            if point_or_x is None:
                raise ArgumentNullException("point")

```

```

        if not isinstance(point_or_x, Point):
            raise TypeError("Expected Point type")
        self._steps.append(point_or_x)
    elif isinstance(point_or_x, int) and isinstance(y, int):
        self._steps.append(Point(point_or_x, y))
    else:
        raise ArgumentException("Invalid arguments")

    def add_rectangle(self, rectangle_or_x_left, y_top=None,
x_right=None, y_bottom=None, size=None):
        if isinstance(rectangle_or_x_left, Rectangle):
            if rectangle_or_x_left is None:
                raise ArgumentNullException("rectangle")
            if not isinstance(rectangle_or_x_left, Rectangle):
                raise TypeError("Expected Rectangle type")
            self._steps.append(rectangle_or_x_left)
        elif isinstance(rectangle_or_x_left, int) and
isinstance(y_top, int) and isinstance(x_right, int) and
isinstance(y_bottom, int):
            self._steps.append(Rectangle(rectangle_or_x_left,
y_top, x_right, y_bottom))
        elif isinstance(rectangle_or_x_left, int) and
isinstance(y_top, int) and isinstance(size, int):
            self._steps.append(Rectangle(rectangle_or_x_left,
y_top, size=size))
        else:
            raise ArgumentException("Invalid arguments")

    def add_circle(self, circle_or_x_center, y_center=None,
radius=None):
        if isinstance(circle_or_x_center, Circle):
            if circle_or_x_center is None:
                raise ArgumentNullException("circle")
            if not isinstance(circle_or_x_center, Circle):
                raise TypeError("Expected Circle type")
            self._steps.append(circle_or_x_center)
        elif isinstance(circle_or_x_center, int) and
isinstance(y_center, int) and isinstance(radius, int):
            self._steps.append(Circle(circle_or_x_center,
y_center, radius))
        else:
            raise ArgumentException("Invalid arguments")

```

```

class IPainter:
    def paint(self, steps: List[Step]) -> None:
        raise NotImplementedError

class SimplePainter(IPainter):
    def paint(self, steps: List[Step]) -> None:
        if steps is None:
            raise ArgumentNullException("steps")
        if not isinstance(steps, list):
            raise TypeError("Expected list type")
        for step in steps:
            if not isinstance(step, Step):
                raise TypeError("Expected Step type")
            print(step.get_representation())

class ArgumentNullException(Exception):
    pass

class ArgumentException(Exception):
    pass

class InvalidOperationException(Exception):
    pass

class Program:
    @staticmethod
    def main(args: List[str]) -> None:
        storage = StepStorage()
        storage.add_move(3, 5)
        storage.add_color("red")
        storage.add_point(10, 10)
        storage.add_move(100, 100)
        storage.add_rectangle(12, 14, size=55)
        storage.add_circle(16, 16, 13)
        simplePainter = SimplePainter()
        simplePainter.paint(storage.get_steps())

if __name__ == "__main__":
    Program.main([])

```

Можно сделать следующие выводы о результате трансляции для данного варианта:

- Свойство безопасности проверка входных аргументов, которое принадлежит классу L1, сохраняется.
- Свойство безопасности сильная гарантия состояния программы к возникновению исключений, которое принадлежит классу L1, сохраняется.
- Свойство безопасности корректная работа с абстрактными типами и методами, которое принадлежит классу L1, сохраняется частично, т.к. почему то класс *Step* и его методы сделаны абстрактными, а класс *IPainter* и его методы не сделаны абстрактными.
- Свойство безопасности типобезопасность, которое принадлежит классу L2, сохраняется.
- Свойство безопасности корректная поддержка перегруженных методов, которое принадлежит классу L2, поддерживается частично за счет изменения интерфейса кода. Также, реализация поведения перегруженных методов в коде на языке Python отличается от поведения перегруженных методов в коде на языке C#: в коде на языке C# я могу, например, передать *null* в метод *StepStorage.AddMove*, но не могу в коде на языке Python передать *None* в метод *StepStorage.add_move*. Поэтому считаем, что это свойство безопасности не сохраняется.
- Все свойства безопасности, относящиеся к классу L3, теряются. Мы считаем, что для нас это приемлемо (из-за особенностей языка Python).
- Свойств безопасности, относящихся к классу L4, нет.
- Результат выполнения полученного кода совпадает с результатом выполнения исходного кода.
- Интерфейс кода (интерфейс типов), полученного в результате трансляции не совпадает с интерфейсом исходного кода из-за подхода к реализации поддержки перегруженных методов, а также из-за изменения имен всех методов.
- Странная попытка использовать один и тот же аргумент функции для параметров разных типов, в результате чего появляются аргументы с подобными именами: *move_or_x*

Таким образом, мы не можем считать данную трансляцию кода успешной, т.к. хотя функциональные требования полностью выполнены, нефункциональные требования выполнены лишь частично (см. [глава 3.7]). Следует сказать, что результат этой трансляции уже сильно близок к результату, который мы ожидаем.

6.5 Выводы из результатов экспериментов.

Из полученных результатов экспериментов можно сделать следующие выводы:

- Трансляция кода с помощью симплификатора и транслятора позволяет получить нам ожидаемый результат. Следует сказать, что это полностью зависит от того, насколько полно и корректно реализован транслятор (см. [глава 4.9]).
- Трансляция кода вручную показывает, что разработчики не всегда занимаются выявлением и анализом свойств безопасности при решении подобных задач. Это по итогу приводит к тому, что свойства безопасности, которые принадлежат классу L2, теряются, т.к. для их сохранения необходимо создавать специальные

компенсирующие механизмы, как, например, в случае такого свойства безопасности, как типобезопасность (см. [глава 3.5]). Про анализ кода, созданного вручную следует сказать следующее: с одной стороны с точки зрения статистики нужно, чтобы вариантов решения было несколько и чем больше, тем лучше, с другой стороны - для этого нам нужно ввести понятие “средний” код и алгоритм, как его получить из всех возможных вариантов.

- Результаты трансляции с помощью LLM достаточно неоднозначны. Во-первых можно получить неработающий вариант кода. Во-вторых, во многих случаях недостаточно начального простота, чтобы получить от LLM нужный результат - приходится использовать дополнительные промпты с уточнениями. Возможно, что при другом начальном промпте, чем те, которые использовались в попытка 2 и 3, использование дополнительных промптов с уточнениями было бы не нужно. Тем не менее, до конца устраивающий вариант с помощью LLM получить не удалось. Скорее всего, если использовать неограниченное количество уточнений, то рано или поздно можно прийти к ожидаемому результату; однако достичь его за ограниченное количество использования LLM (начальный промпт и 1-2 промпта с уточнениями) не удалось.
- Непредсказуемость работы LLM: периодически LLM может выдавать некорректный результат. Даже в данном эксперименте дважды LLM выдавал неработающий код. Данный код очень простой, поэтому все проблемы достаточно легко найти, однако в случае большого проекта это будет не так и тогда встает вопрос о контроле качества результата, получаемого с помощью LLM.
- При этом стоит сказать о качестве кода, генерируемом с помощью LLM: оно несколько хуже, чем код, который либо получается непосредственно в результате работы человека, либо получается в результате использования транслятора; например, использование одних и тех же именованных аргументов метода для объектов разных типов (обычно все таки используют разные именованные аргументы метода для объектов разных типов).
- При работе с автоматизированным транслятором мы можем с помощью тестов как проверять корректность работы транслятора, так и документировать особенности его работы (например, как транслируется та или иная конструкция). Однако, для трансляции с помощью LLM возникает вопрос о возможности существования подобных тестов, т.к. нет никакой гарантии, что на один и тот же запрос к LLM мы всегда будем получать один и тот же ответ; к тому же, если запросы к LLM платные, то идея подобных тестов полностью теряет свой смысл.

Также следует сказать о том, какие в будущем возможны дальнейшие эксперименты (в рамках уже других работ):

- Трансляция кода одного или нескольких web-приложений с проверкой с помощью SAST и DAST как исходного кода, так и кода полученного в результате трансляции с последующим сравнением результатов. Основной проблемой здесь является создание обертки над АПИ (см. [глава 4.5] и [глава 5.5.2]), т.к. это достаточно трудоемкий процесс: в одной только базовой

библиотеке BCL (Base Class Library) для .NET содержится примерно 10000 - 15000 типов, у каждого из которых в среднем 8-12 методов, а есть еще ASP.NET Core, в которой примерно 20000-30000 типов; конечно нам не все типы будут нужны сразу, но тем не менее это достаточно трудоемкий процесс.

- Выбор различных LLM и различных промптов для них с тем, чтобы оценить насколько хорошо и стабильно можно решать задачу автоматической трансляции кода между языками с сохранением семантики безопасности с помощью LLM, какая из LLM лучше для этого подходит и насколько детально должны быть начальные промпты, чтобы минимизировать количество промптов с уточнениями. Эти эксперименты следует проводить для разных проектов с проверкой результатов работы, прохождения тестов, а также с помощью SAST и DAST проверок. Также, эти эксперименты необходимо проводить вместе с трансляцией кода с помощью транслятора для сравнения результатов, полученных с помощью транслятора и с помощью LLM.

7. Ограничения

Процесс автоматической трансляции, несмотря на свои возможности, не является полностью универсальным решением, т.к. у него есть немало ограничений. Давайте посмотрим на некоторые из них:

- 1) Мы далеко не для каждой пары языков можем в автоматическом режиме транслировать код между этими языками. Например, если исходный код написан на C++ и широко использует шаблоны и метапрограммирование, то такой код мы скорее всего не сможем транслировать на тот же C#, в котором шаблонов и метапрограммирования нет (см глава [4.4]).
- 2) Свойства безопасности, принадлежащие классам L1 и L2, сильно зависят от их поддержки в трансляторе (см. глава [4.9]). Как уже говорилось (см. глава [3.4]), свойства безопасности, которые принадлежат классу L1, автоматически сохраняются при трансляции, для сохранения свойств безопасности, которые принадлежат классу L2 необходимо сгенерировать соответствующие компенсирующие механизмы (с помощью дополнительного кода, генерируемого при трансляции). Это все справедливо только в случае “идеального транслятора”; достаточно часто бывают ситуации, что транслятор умеет переносить не весь код с одного языка на другой, при этом такой его урезанной функциональности хватает для сохранения функциональных требований и поведения исходного кода (см. глава [1.9]). Если транслятор не поддерживает функциональность, от которой зависит некоторое свойство безопасности, которое принадлежит классу L1, то это свойство безопасности не будет автоматически сохранено при трансляции. Если транслятор не поддерживает генерацию соответствующих компенсирующих механизмов для некоторого свойства, которое принадлежит классу L2, то это свойство также не будет сохранено при трансляции.
- 3) Влияние качества исходного кода (особенно в случае legacy-кода). Мы формируем набор свойств безопасности (Security Fingerprint), с которым работаем на протяжении всей процедуры трансляции, на основе анализа исходного кода (см главы [3.1-3.2]). Однако, если исходный код имеет достаточно низкое качество, то его набор свойств

безопасности будет достаточно беден (особенно часто это справедливо для случая legacy-кода). Например, если в исходном коде проверка входных аргументов (для не частных членов) полностью отсутствует, то соответствующего свойства безопасности не будет в наборе свойств безопасности (Security Fingerprint); если же в исходном коде проверка входных аргументов выполняется не везде, то соответствующее свойство безопасности будет в наборе свойств безопасности (Security Fingerprint), но со значением “поддерживается частично” (в агрегирующем наборе свойств безопасности). Ни транслятор, ни симплификатор повлиять на это никак не могут - они выполняют совершенно другие задачи (в промышленном решении могут использоваться специальные утилиты для генерации дополнительного кода, но в рамках данной работы они не рассматриваются - см. глава [5.3]).

4) Некорректное сопоставление свойства из двух наборов свойств безопасности (Security Fingerprint) во время их сравнения (см. глава [3.3]). Обычно, мы сравниваем свойства из следующих двух наборов свойств безопасности (Security Fingerprint): первый набор получен при анализе свойств безопасности для исходного кода, а другой - при анализе свойств безопасности для кода, полученного в результате трансляции. При этом нам нужно уметь сопоставлять свойства безопасности из этих двух наборов свойств безопасности (Security Fingerprint) для того, чтобы отслеживать то, как свойства безопасности изменяются во время трансляции кода. Если в наборе свойств безопасности (Security Fingerprint) для исходного кода и наборе свойств безопасности (Security Fingerprint) для кода, полученного в результате трансляции, есть свойства с одним и тем же именем, то в этом случае все тривиально - свойства с одинаковыми именами всегда могут быть сопоставлены. Однако, не для всех свойств безопасности, которые присутствуют в наборе свойств безопасности (Security Fingerprint) для исходного кода, есть свойства безопасности с совпадающими именами, которые присутствуют в наборе свойств безопасности (Security Fingerprint) для кода, полученного в результате трансляции: одна часть свойств безопасности просто не сохраняется при трансляции (это свойства безопасности, которые принадлежат классу L3), а другая часть свойств безопасности появляется в результате трансляции (это свойства безопасности, которые принадлежат классу L4). Оказывается, что достаточно часто мы можем семантически сопоставить свойство безопасности исходного кода, которое принадлежит классу L3 со свойством безопасности кода, полученного в результате трансляции, которое принадлежит классу L4. Это связано с тем, что языки (и платформы) могут отличаться по той функциональности, которую они содержат (поддерживают). Например, в языке C# для обработки ошибочных ситуаций используют исключения, тогда как в языке Golang исключений нет (в языке Golang есть паника, но это далеко не то, что исключения языка C#) и используются ошибки (объекты, которые возвращаются из функции). Предположим, что мы транслируем код с языка C# на язык Golang. Предположим, что у нас есть свойство безопасности “сильная гарантия к возникновению исключений” в исходном коде на языке C#, а в коде на языке Golang, полученном в результате трансляции, есть свойство безопасности “сильная гарантия к возникновению ошибок”. Очевидно, что эти два свойства безопасности можно семантически сопоставить (причем первое формально принадлежит классу L3, а второе формально принадлежит классу L4). И вот при таком

семантическом сопоставлении есть опасность, что мы сопоставим не те свойства и получим при сравнении двух наборов свойств безопасности (Security Fingerprint) совсем не тот результат.

5) Невозможность сравнения двух свойств безопасности со значением “поддерживается частично”. Мы ввели свойства безопасности кода, их значения и правила сравнения двух свойств безопасности, которые можно семантически сопоставить (см. главы [3.1-3.3]). Однако при сравнении двух свойств безопасности со значением “поддерживается частично” результат не определен. Но как показывает практика, значение свойств безопасности “поддерживается частично” встречается достаточно часто (в некоторых проектах это вообще самое частое значение для свойств безопасности). Но разумного способа сравнения двух свойств безопасности со значением “поддерживается частично” нет. Пусть, например, у нас есть метод *M* с пятью аргументами; если в исходном коде у него проверяются первый и второй аргументы, а в коде, полученном в результате трансляции - третий и четвертый, то свойство безопасности для метода *M* имеет значение “поддерживается частично” как для исходного кода, так и для кода, полученного в результате трансляции. Очевидно, что невозможно сравнить эти два свойства безопасности. Особенно сильно это касается сравнения двух агрегирующих наборов свойств безопасности (Security Fingerprint), т.к. кажется, что именно для агрегирующих наборов свойств безопасности (Security Fingerprint) гораздо чаще будут встречаться свойства безопасности со значением “поддерживается частично”.

6) Получение свойств безопасности в автоматизированном режиме. Свойства безопасности можно получать как в ручную, так и в автоматическом режиме. Однако, при получении свойств безопасности в автоматическом режиме есть ряд сложностей. Во-первых, необходимо уметь получать AST не только для исходного кода, но и для кода, полученного в результате трансляции (это не проблема, например, для примеров в данной работе, т.к. мы транслируем код с языка C# на язык Python, но может быть проблемой для другой пары языков). Во-вторых, для получения свойств безопасности в автоматическом режиме необходимо уметь анализировать AST специальным образом (например, для поиска определенных паттернов и конструкций); это достаточно большая задача, по объему и сложности не уступающая данной работе.

7) Влияние внешних зависимостей. Как уже говорилось ранее (см. глава [3.6]), внешние зависимости являются внешним фактором, который способен изменять состояние свойств безопасности после трансляции с течением времени и, как итог, приводить к “деградации” Security Fingerprint. Другими словами, с течением времени из-за внешних зависимостей семантика безопасности может становиться “слабее” уже после трансляции. И это никак не исправить без вмешательства разработчиков, причем вносить правки потенциально нужно и в исходный код, и в код транслятора (или в библиотеку, используемую для согласования АПИ между исходным языком и языком, на который выполняется трансляция - см. глава [4.5]); после чего нужно будет получать новую версию кода с помощью трансляции.

8) Влияние ручного управления трансляцией на набор свойств безопасности (Security Fingerprint) кода, полученного в результате трансляции. В результате ручного управления трансляцией мы можем получить код, который слабо связан с исходным

кодом; у нас могут даже появиться новые сущности и исчезнуть сущности, которые есть в исходном коде. Как итог, наше предсказание эволюции свойств безопасности при трансляции может сильно отличаться от реальной картины (особенно это может касаться детализированных наборов свойств безопасности).

8. Заключение.

Давайте посмотрим на то, что у нас в рамках данной работы получилось сделать. Мы показали, какие бывают свойства безопасности кода и как их можно получить для текущей кодовой базы с помощью ее анализа, используя списки Common Weakness Enumeration (CWE) и спецификации исходного языка. Было показано как из этих свойств безопасности формировать набор свойств безопасности (Security Fingerprint) как агрегирующий, так и детализированный, а также как сравнивать два набора свойств безопасности (Security Fingerprint) - до процедуры трансляции и после. Была определена классификация свойств безопасности (классы L1, L2, L3, L4) относительно процедуры трансляции кода с одного языка на другой, после чего, на основании этой классификации были разработаны правила трансформации свойств безопасности во время процедуры трансляции кода. Мы показали на примерах, что для сохранения свойств безопасности, которые относятся к классу L2, необходимо генерировать компенсирующие механизмы (дополнительный код), а свойства безопасности, которые относятся к классу L3, вообще сохранить нельзя, и это является фундаментальными особенностями таких свойств безопасности, а не какими-то недоработками в процедуре трансляции. Мы определили, при каких условиях при трансляции у нас сохраняется семантика безопасности: мы можем определить эти условия еще до проведения процедуры трансляции исходя из функциональных (и нефункциональных) требований, а также из свойств безопасности, присущих нашему коду и их классификации. Был рассмотрен ряд дополнительных вопросов о влиянии внешних зависимостей и окружения на набор свойств безопасности кода (Security Fingerprint). После этого была выполнена математическая формализация нашего подхода к работе со свойствами безопасности (с наборами свойств безопасности) при трансляции кода с одного языка на другой. Далее мы определили, что такое автоматическая трансляция кода между языками, после чего провели неформальную классификацию некоторых языков программирования и показали проблемы, возникающие при трансляции кода между языками (при промышленном использовании транслятора между разными парами языков), которые относятся к разным классам. Было сделано наблюдение, что, чем больше совпадают языки, между которыми происходит трансляция кода, друг с другом, тем проще процесс трансляции; и наоборот, чем дальше языки, между которыми происходит трансляция кода, друг от друга, тем сложнее процесс трансляции. Отсюда была выдвинута следующая гипотеза: чем больше совпадают языки, между которыми происходит трансляция кода, друг с другом, тем больше свойств безопасности будут принадлежать классу L1; чем дальше языки, между которыми происходит трансляция кода, друг от друга, тем больше свойств будут принадлежать классам L2 и L3. После этого были рассмотрены проблемы согласования АПИ при трансляции между языками и предложены методы

решения этих проблем; также были рассмотрены возможности ручного влияния на процесс автоматической трансляции. Далее было показано, что мы можем упростить код транслятора за счет выделения отдельного шага перед трансляцией - симплификации, т.е. упрощение исходного кода так, чтобы транслятору было проще его обрабатывать. В качестве одной из центральных гипотез была выдвинута гипотеза о том, что симплификация полностью сохраняет семантику безопасности исходного кода, упрощает код транслятора и уменьшает вероятность ошибки/дефекта в трансляторе. Далее было показано, как процесс трансляции вместе с симплификацией реализован; также были показаны некоторые детали этой реализации. Был проведен эксперимент по сравнению разных подходов к трансляции кода: с использованием транслятора (и симплификатора), вручную, а также с использованием LLM. И, наконец, были описаны ограничения, которые возникают при использовании автоматической трансляции с сохранением семантики безопасности.

Из данной работы можно сделать следующий вывод: был разработан метод автоматической трансляции кода между языками с сохранением семантики безопасности; был проведен эксперимент, который продемонстрировал (по сравнению с другими методами), что данный метод работает. Также этот метод позволяет определить в каких случаях трансляция считается успешной, а в каких нет и какие компенсирующие меры необходимо применить, чтобы трансляция кода между языками считалась успешной (относительно функциональных и нефункциональных требований, а также сохранения семантики безопасности). Но несмотря на это, у метода автоматической трансляции кода между языками есть свой ряд ограничений, который не позволяет во всех случаях успешно транслировать код между языками, особенно если есть достаточно жесткие требования на сохранение тех или иных свойств безопасности.

9. Литература.

9.1 Книги и статьи

- 1) “Компиляторы: принципы, технологии и инструменты” Альфред Ахо, Моника С. Лам, Рави Сети, Джеффри Ульман. ISBN 978-5845913494.
- 2) “Трансляция языков программирования” Ф. Вайнгартен ISBN 978-5-0000-0000-0.
- 3) “Языки программирования и методы трансляции” Опалева Э. А., Самойленко В. П. ISBN 5-94157-327-8.
- 4) “Конструирование компиляторов” Мёссенбёк Х. ISBN 978-5-93700-391-1.
- 5) “Построение компиляторов” Никлаус Вирт ISBN 978-5-97060-219-5.
- 6) “Типы в языках программирования” Пирс Бенджамин ISBN 978-5-7913-0082-9.
- 7) “Practical Foundations for Programming Languages” Robert Harper ISBN 978-1107150300.
- 8) Ховард М., Лебланк Д. "Защищённый код" ISBN 5-7502-0238-0.
- 9) Marco Patrignani, Amal Ahmed, and Dave Clarke. 2018. Formal Approaches to Secure Compilation: A survey of fully abstract compilation and related work. ACM Comput. Surv. 1, Article 1 (January 2018), 46 pages. <https://doi.org/10.1145/3280984>

10) ГОСТ Р 71206-2024 НАЦИОНАЛЬНЫЙ СТАНДАРТ РОССИЙСКОЙ ФЕДЕРАЦИИ Защита информации РАЗРАБОТКА БЕЗОПАСНОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ. БЕЗОПАСНЫЙ КОМПИЛЯТОР ЯЗЫКОВ С/С++ Общие требования (Information protection. Secure software development. Safe C/C++ compiler. General requirements)

11) Translation Validation of Optimizing Compilers by Yi Fang A dissertation submitted in partial fulfillment of the requirements for the degree of Doctor of Philosophy Department of Computer Science Courant Institute of Mathematical Sciences New York University December, 2005

9.2 Онлайн документация

101) Common Weakness Enumeration (CWE) <https://cwe.mitre.org/>

102) Capability Hardware Enhanced RISC Instructions (CHERI)

<https://www.cl.cam.ac.uk/research/security/ctsrds/cheri/>

103) Intel Software Guard Extensions (Intel SGX) <https://eprint.iacr.org/2016/086.pdf>

104) Proof-carrying code (PCC) <https://www.cs.princeton.edu/~appel/papers/fpcc.pdf>

105) CompCert <https://compcert.org/>

106) seL4 microkernel <https://sel4.systems/>

107) WebAssembly (Wasm) <https://webassembly.org/>

108) Strategy pattern https://en.wikipedia.org/wiki/Strategy_pattern

109) CWE-787: Out-of-bounds Write <https://cwe.mitre.org/data/definitions/787.html>

110) Python isinstance function <https://docs.python.org/3/library/functions.html#isinstance>

111) .NET Reflection API

<https://learn.microsoft.com/en-us/dotnet/fundamentals/reflection/overview>

112) Python name mangling <https://docs.python.org/3/tutorial/classes.html#private-variables>

113) Python inspect module <https://docs.python.org/3/library/inspect.html>

114) C standard string library <https://en.cppreference.com/c/header/string>

115) Software Composition Analysis

<https://github.com/resources/articles/what-is-software-composition-analysis>

116) Path Traversal Vulnerabilities

https://owasp.org/www-community/attacks/Path_Traversal

117) The .NET Compiler Platform SDK (Roslyn)

<https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/>

118) Clang LibTooling <https://clang.llvm.org/docs/LibTooling.html>

119) ANTLR <https://www.antlr.org/>

120) Garbage collection

[https://en.wikipedia.org/wiki/Garbage_collection_\(computer_science\)](https://en.wikipedia.org/wiki/Garbage_collection_(computer_science))

121) Reference cycles <https://doc.rust-lang.org/book/ch15-06-reference-cycles.html>

122) Weak reference https://en.wikipedia.org/wiki/Weak_reference

123) A garbage collector for C and C++

<https://www.cs.princeton.edu/~appel/modern/c/software/boehm/gc.html>

124) Simple tracing (mark and sweep) garbage collector for Rust

<https://github.com/Manishearth/rust-gc>

125) Resource acquisition is initialization (RAII)

https://en.wikipedia.org/wiki/Resource_acquisition_is_initialization

126) Cyclomatic complexity

<https://www.sonarsource.com/resources/library/cyclomatic-complexity/>

127) Partial Classes and members

<https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/partial-classes-and-methods>

128) PEP 8 – Style Guide for Python Code <https://peps.python.org/pep-0008/>